The Coefficient Is an Agreement

Cost drivers, step changes, and the operating discipline that turns capacity plans into commitments

Guillermo Martinez

Version 7.9 · October 2026 · a working draft

How this paper is built

Each layer stands on its own and goes one level deeper. The abstract is for the expert, and the executive summary for a busy leader in finance, engineering or supply chain. Then come a one-page version to pin up and the deep dive (Sections 1–14) for planners. Appendix A holds a worksheet with a full walk-through; Appendix B, the public record on supply behind Section 6. Each appendix stands on its own and can be printed or handed over separately.

The two claims are the part meant to last. The method, the thirteen mechanisms, the stages and every illustrative number are expected to change as people use them.

What changed in this version

For readers who saw an earlier draft, these are the main changes since version 6.

Item	What happened	Why
Adjacent fields and new evidence (v7.9)	Telephone engineering added to the discipline's lineage. New evidence from supply chains, power grids, risk management and AI platforms. It covers why not to split a shortage pro rata, why to name shared drivers, and measured AI demand. One labeled exploration on AI demand declared as a policy. New sources are listed as [A1]–[A9] until the next renumbering.	To test the method against fields that solved neighbouring problems first
Credit, re-checked (v7.8)	More of the frame is credited to Hixson and Guliani: blow-up factors as an early α, the fast lane, one plan for several products' upside, and the limits small companies face. The 2026 SRE chapter's link to error budgets is quoted again, and its *stockout resiliency* is named. Several reference links were repaired.	So a reader can tell which parts are the sources' and which are this paper's.
Prior art, re-checked (v7.7)	The measuring loop's eight steps are labeled as this paper's arrangement of the 2026 SRE chapter's advice. A few readings of the chapter were softened, and Hixson and Guliani's "costs" are read as an early form of the coefficient	Accuracy and fair credit
Softer where experts push back (v7.7)	Coefficients as a straight-line reading near today's load; *k* set per cell, with uneven failover; the marginal job pulls the next buy earlier; buffers multiply only if latency must hold during a failure	Precision
Layers	An abstract, an executive summary and a one-page version now come before the deep dive	Each reader can stop at the depth they need

Item	What happened	Why
Two claims and a method	The paper now states two claims, that a coefficient is an agreement and that approval is not allocation, and offers one method	To say plainly what it argues and what it offers
The commit-point method (Section 8)	New: eight steps for one launch, what each side owes the other, and a walk-through	To give planners something to try, not only an argument
Supply (Sections 6.2 and 6.7)	Section 6.2 rewritten around public values; Section 6.7, on supply's own step changes, expanded	Supply runs on its own clock, with its own lead times and contracts
Appendices A and B	New: a worksheet with a full walk-through, and the public record on supply	Each can be printed or handed over on its own
Figures	Added throughout: clocks, scenes, method cards, flowcharts, swimlanes and charts of public values; 60 in all	So the figures and captions carry the argument on their own
Corrections	Hixson and Guliani's five years is quoted in full, as their example, not a maximum; the AEP Ohio tariff figures and the Section 9.1 quote are corrected	Accuracy
Tests on the mechanisms (Section 9.2)	Each test moved from Section 9.4 to the mechanism it belongs to, under *You'll know it's working when…*	So each mechanism carries its own check
Mechanism 3	Renamed the coefficient register and versioned rate card	It's the practice that keeps the first claim true, not a second statement of it
Sentences	Most long sentences split into shorter ones; a few that hold quotes are left whole	Easier to read
The 2026 SRE chapter	Read in full; credited in Sections 1, 3, 5.2, 5.4, 6, 7, 8.5, 9, 10, 11, 12 and 13, the Terms and Appendix B.5 wherever it got there first, and every "may cover" resolved	Its authors include Hixson, and much of this paper's ground is theirs first
New public sources	Added on pooling, stranding, quota versus capacity, phased supply, the Meet case and recent operator statements; lineage of the discipline in Section 3	Each supports a claim already made, or softens one
Eight explorations	Labeled ideas from setting the paper beside the SRE chapter (Sections 5.2, 6.3, 6.6, 6.7, 9.3, 10.2, 12 and 13)	Offered as ways to think, not findings
The measuring loop (Section 5.2, Figure 10)	The SRE chapter's business-driver loop is now laid out step by step and credited in full; the paper adds the price reading and a labeled idea that the chapter's two alerts route each term to its owner. A second labeled idea makes re-baselining a routine step that scores the gap by owner. Sections 5.4 and 7 each gain one line	The paper's loop showed only the governance half
The handshake (Section 10, Figure 41)	Section 10 opens with capital, supply and demand as a handshake the planner brokers, and what the planner trades in; Figure 41 becomes the triangle, with the three seams on its edges	The people half lacked one frame

Item	What happened	Why
People practice (Sections 6.1, 6.4, 9.1, 9.3, 10.1, 10.2, 10.4, 12 and 14)	Short, general notes on where the right to refuse comes from, talking before the intake, no surprises in the room, shocks, finance’s distrust, translation, freezes, onboarding, work between charters, re-baselining, brokering and re-shaping	Practice the mechanisms depend on, written as practice, not as a story

Abstract

A team can forecast its demand on a shared service within a few percent and still see its bill come in 30% over plan. The bill multiplies three numbers: how much demand the team sends, how much resource each unit uses (the *coefficient*), and what that resource costs (the *rate*). The arithmetic is published; the team owns only the first number. This paper makes two claims and offers one method. A coefficient is an agreement between the suppliers and users of shared capacity; it holds only while someone owns it, versions it and restates it when hardware or measurement changes. Approval is not allocation. An ask can be approved in a common unit that makes unlike requests comparable, and still lack machines of the right shape, in the right place, on time. That gap needs an owner. Supply runs on its own clock. Power, buildings and machines are ordered long before launches firm up, under contracts that turn forecasts into liabilities. And supply slips in steps of its own. The commit-point method joins demand's clock (how firm a launch is) to supply's (when each layer must be ordered). For one launch, it asks when each layer must be ordered in a slow case, what to commit then and in what form, and who owns the rest. The paper also separates three prices one number is often asked to serve. It is a practitioner synthesis, not a model, and untested.

Executive summary

Backdrop. Shared infrastructure (a database tier, a storage service) is paid for centrally and used by product teams. A product's cost on a shared service is *volume* × *coefficient* × *rate*: how much demand it sends, how much resource each unit uses, and what that resource costs. Large operators have published how to compute it at scale, though attributing some work is still open (Sections 2 and 3), and plans still miss.

The problem. Take Section 7's invented year-end review. A team forecast 500,000 requests per second and landed at 480,000, within 4%. Its bill came in at $683,280 against a plan of $525,600, 30% over. Split the miss by term and each piece goes to whoever moved it:

- **Volume:** −$21,024, owned by the product team.
- **Coefficient:** +$126,144, owned by the shared service, whose release raised CPU per call.
- **Rate:** +$52,560, owned by the service with finance, after its power was re-priced.

The two terms the team didn't own add up to +$178,704, more than the whole +$157,680 overrun. The split turns an argument about fault into a meeting about two fixes; who *pays* is a separate rule, best agreed before the miss (Sections 5.2 and 7). The same example misses a second way. The ask was approved as 4 units of an invented common unit, because the intake asked only about CPU. Once its memory was counted, it needed 4.8, and the high-memory machines arrived a quarter after launch. Figure 1 shows both misses.

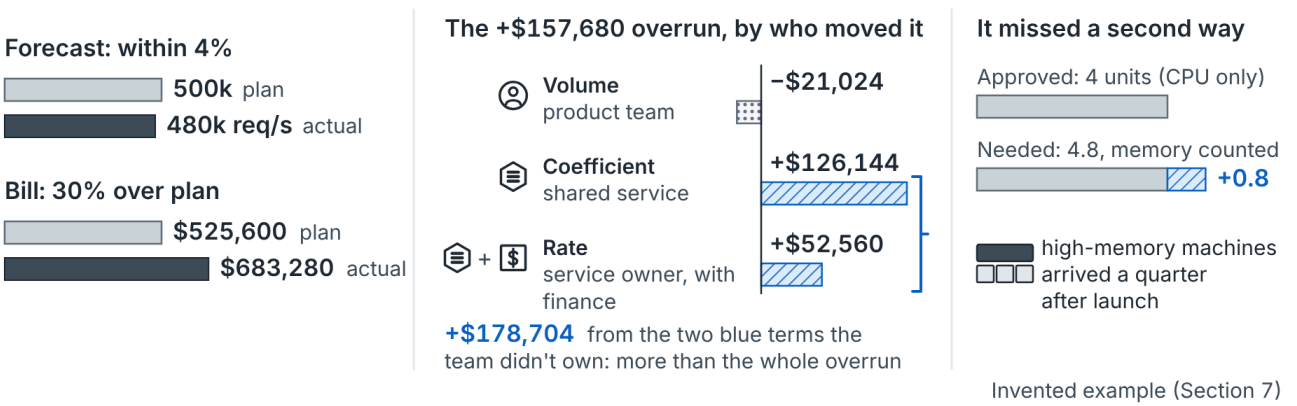


Figure 1. The same invented year-end review, two ways it missed: a bill 30% over a forecast within 4%, split by who moved each piece, and an approval that needed more machines, later, than it bought.

What the paper offers: two claims and a method.

1. **A coefficient is an agreement.** One number forecasts the service's load and prices the caller's growth, so it needs an owner, versions and restating when hardware or the meter changes (Sections 5.1–5.3). Launches with no history enter as *declarations*: signed statements of expected demand that firm up in stages (Section 5.4).
2. **Approval is not allocation, in shape, place and time.** The gap to "running" needs a named owner and a ledger (Section 6.1). Supply runs on its own clock: years for power and buildings, quarters for machines, and weeks to install and test them (called *turn-up*). It runs under contracts that become liabilities once their cancel windows (the last date to walk away cheaply) close. And it slips in steps of its own (Sections 6.2 and 6.7).

3. **The commit-point method**, below (Section 8).

Two ideas support them. An **operating discipline** of thirteen mechanisms, each an artifact with an owner, keeps the numbers true each cycle (Section 9). And **three prices** answer three questions, as in a hotel (Section 4): who pays for what already runs, whether to build, and who gets scarce capacity now. In hotel terms, those are a room-night's average cost, a new wing's cost per room, and the last room's worth on a sold-out night. One number for all three misleads on at least one. Figure 2 maps the claims, the method and their supports to the sections that carry them.

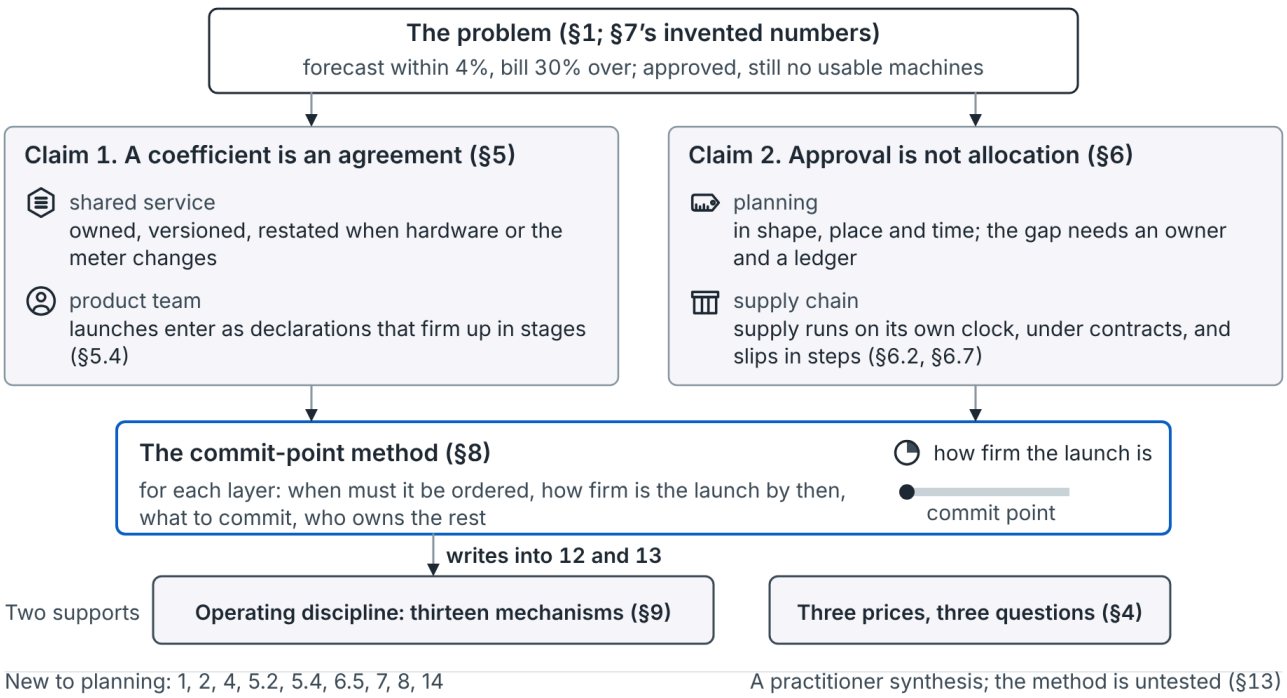


Figure 2. Two claims and one method, resting on an operating discipline and a clear answer to "which price?". Section numbers show where each is argued.

The method. For each launch, work back from the date capacity must be live to each supply layer's *commit point*. That's the last date its order can still be placed, on a slow-case lead time. At each commit point, read how firm the launch will be and whether what drives that layer (memory, region) is known yet. Cover more of the range of likely demand where running short costs more than holding spare. For lasting demand in a growing business, the paper argues (not a published result) that this usually means the slow layers, held largely as options (Section 6.5). Net what's on hand or on the way, and buy the firm part. Hold the next slice as options or a reserve, and name a stopgap (a *bridge*) and an owner for the rest. Agree how much the ask may still change at each stage (*change bands*), and have supply answer every line with a delivery date or a named shortfall (a *named gap*). In the example, the memory question moves to eleven months out, for about 1.0 extra unit held above the middle estimate (Section 8.4; illustrative). It's a way to think it through, not a model.

What to do Monday.

- *Planning and product:* walk the next launch through the worksheet (Appendix A); put last year's ask beside its actual; ask for slow-supply attributes first (Section 9.3).
- *Finance should co-own* a few rules, agreed in advance. One is what "fully loaded" contains. Two more are the landing rule (who keeps capacity a team saves) and the shortage-to-idle ratios (which set how far up the range each

layer buys). Last are the order of cuts when budgets move, and who pays when a service's own coefficient regresses or an ask moves outside its change bands (Sections 9.2 and 14).

- *Supply should own* a dated commit or named gap for every approved ask, and the link from each order to its demand. It also owns a contract register with liabilities and cancel windows, and, with planning, the change bands and the commit-point map. And it owns its own step changes, declared in stages. It is scored on promised against actual, as demand is scored on how much it changed inside supply's lead time (Sections 6.2, 6.4, 8.3, 9.2 and 14).

What's unknown. This is argument from sources and practice, not a measured study. In the sources I could read, I found no measurement of coefficient drift, of how well declarations are calibrated by stage, or of how often launches slip. I also found no published order-to-dock times, turn-up durations or hardware contract terms (nor does the 2026 SRE capacity chapter, Section 3). The method is untested (Section 13).

If you want…	Read
Why bills miss	1, 2, 5.2
What's published, and what's new	3
Which price for which question	4
Governing a coefficient	5.2–5.3
Launches with no history	5.4
Approval versus supply; buffers, bridges	6.1, 6.3, 6.4
Lead times, contracts, options	6.2
How far up the range to buy	6.5
Slips, size misses, supply's own steps	6.6, 6.7
The worked example	7
The method and worksheet	8, Appendix A
The thirteen mechanisms	9
People, overrides, gaming	10
AI fleets; mid-sized companies	11, 12
Limits and unknowns	13
A checklist by role	14

One page

The problem. A team can forecast its demand on a shared service within 4% and still see its bill 30% over plan. That's because two of the three numbers behind the bill belong to someone else: how much resource each request uses, and what that resource costs. And an approved ask can still leave a launch without machines it can use.

Two claims and a method

- **A coefficient is an agreement.** One service release raised CPU per call and moved a caller's bill more than the caller's own forecast miss did (Section 7).
- **Approval is not allocation, in shape, place and time.** An ask was approved as four units of capacity, because the intake asked only about CPU. It needed 4.8 once its memory was counted, and the machines arrived a quarter after launch (Sections 6 and 7).
- **The commit-point method.** Ask each supply layer's question at the date it must be ordered (its *commit point*), not at launch (Section 8).

The method, for one launch

1. List every supply layer and its lead time in a slow case: for example, if machines usually take one quarter but one order in five takes two, plan on two.
2. Work back from the date capacity must be live to each layer's commit point.
3. At each commit point, note how firm the launch will be, and whether what drives that layer is known yet.
4. Cover more of the range of likely demand where running short costs more than holding spare.
5. Subtract what's on hand, on order or due back, each with a date and an owner.
6. Split each layer: firm orders, options or a reserve, and a named stopgap (a *bridge*) with an owner.
7. Agree how much size and date may still move at each stage.
8. Write it down, and get supply's answer: a delivery date or a named shortfall (a *named gap*).

Figure 3 draws the eight steps as decisions.

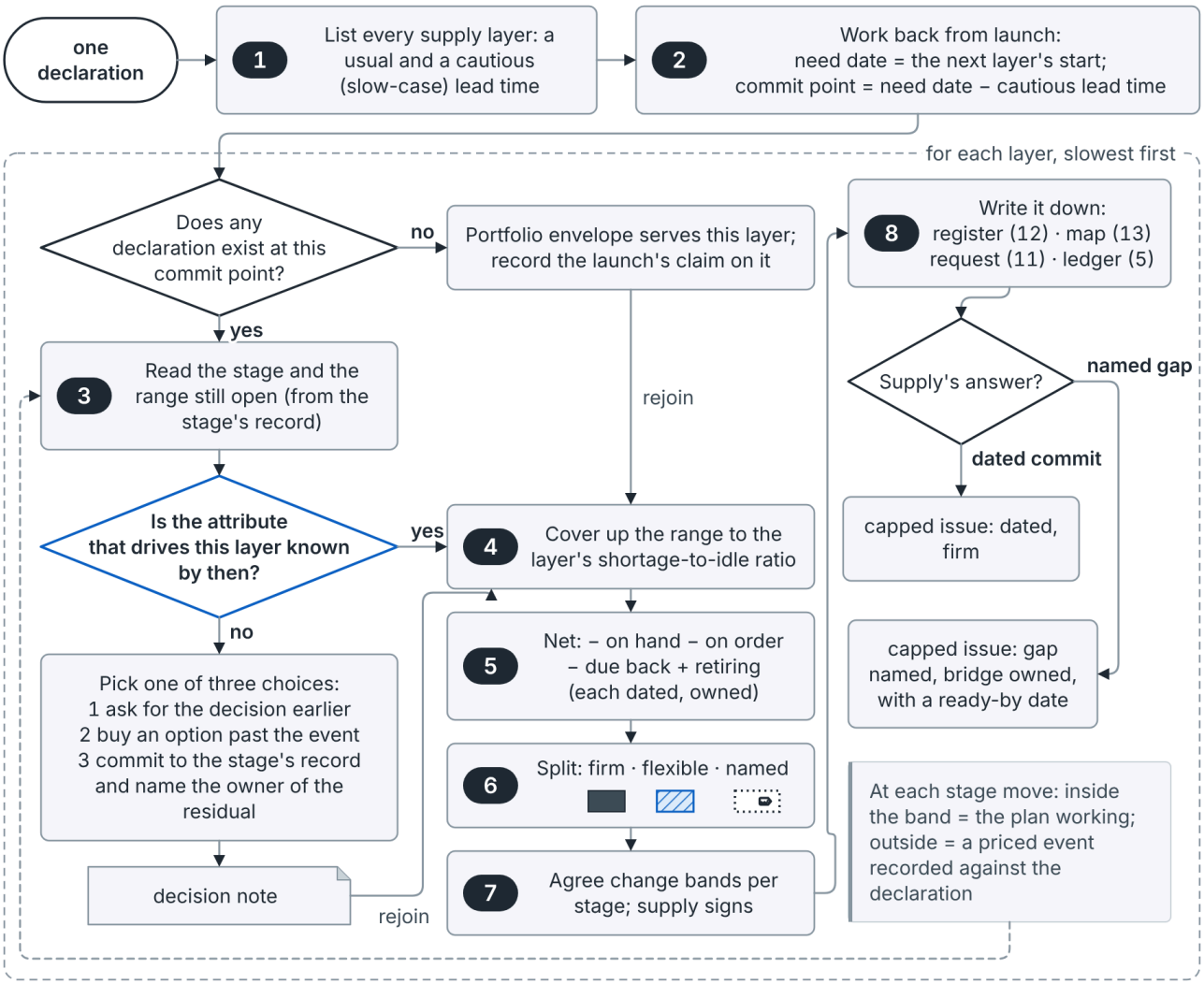


Figure 3. The commit-point method for one launch. Two questions decide the path at each layer: does a declaration exist yet, and is what drives this layer known by its commit point? Every path ends in a written answer from supply, and either answer makes the approval capped issue. A way to think it through, not a model; the method is untested (Section 13).

Five questions for your own planning

1. Who owns each coefficient behind our largest bills, and when did it last change?
2. When a bill missed, did we split the miss into volume, coefficient and rate?
3. Has supply answered every approved ask with a date or a named gap?
4. For our next launch, what is the slowest layer’s commit point, and will we know what drives it by then?
5. What did each team ask for last year, and what did it use?

What it doesn’t claim. It’s a practitioner synthesis, not a model; its numbers are illustrative unless cited, and the method hasn’t been tested (Sections 8.6 and 13).

1. Introduction: a close forecast and a wide miss

In short: a team can land within 4% of its forecast and still be 30% over its bill. That's because two of the three numbers behind the bill belong to someone else.

Picture the year-end review. A product team forecast its demand on a shared service within 4%, yet its bill came in 30% over plan. A release had raised the service's CPU cost per call, and a re-priced power contract had moved its rate. No process surfaced either change in advance (Section 7 works the case with invented figures).

The arithmetic isn't the hard part: requests × calls per request × CPU per call, summed over callers at the shared peak, plus buffers, times a rate. Each coefficient in it is a straight-line reading of a curve, true near today's load, which is why it carries a valid range (Section 5). Large operators have published systems that do it at scale [1, 2]. Plans built on them still miss, in my experience more often because of the agreements around the formula than the formula itself. Hence two claims:

1. **A coefficient is an agreement** between the parties who supply and consume shared capacity, and it holds only while the agreement is kept. Someone declares demand in agreed units, owns the coefficient when it drifts, and signs the forecast. Someone carries the gap when approved demand can't become machines, and owns the cost nobody can attribute.
2. **Approval is not allocation** (*allocation* here means placing demand on physical machines). Approving demand in a common unit prices it, so unlike asks can be compared, but doesn't source it. The distance between "approved" and "live" (Figure 15), in shape, place and time, needs a named owner. Much of the largest demand arrives as step changes (launches, events, new products) that must be supplied before they firm up. And supply has step changes of its own (Sections 5.4 and 6.5–6.7).

Neither claim is new in its parts, and Sections 3 and 6.1 credit where each part comes from. What this paper adds is naming who owns each part, and when, at a finer grain than the engineering, product and finance roles in Hixson and Guliani's article (Section 3).

Section 8 turns the two claims into a method for one launch, the commit-point method. Both claims come down to a handshake among three parties: capital, which funds; supply, which builds; and demand, which uses. The planner brokers it, usually with no capacity or budget of its own (Section 10).

How to read this paper. Sections 1 to 4 lay the foundations: why bills miss, why cost follows the caller, what's already published and capacity's three prices. Read Section 3, above all on Hixson and Guliani, before judging what's new. Sections 5 to 8 are the mechanics: the coefficient, the declaration and the supply order. Then come one invented service (Section 7) and a method built from it (Section 8, with a worksheet in Appendix A). Sections 9, 10, 12 and 14 cover running it, and Sections 11 and 13 cover AI fleets and what's unmeasured. Appendix B collects the public record on supply. New to planning? Read 1, 2, 4, 5.2, 5.4, 6.5, 7, 8 and 14. Each section opens with an *In short* line.

Contributions. The additions are operating mechanisms, each with an owner, a failure mode and an indicator; Section 9.2 lists all thirteen. Six carry the argument:

- a coefficient register that keeps each coefficient versioned and open to challenge (mechanism 3);
- an owner and a ledger for the gap between approval and usable machines (5);

- prior actuals in the intake (2), extending published intake and scoring practice (Section 3) by scoring step changes by stage;
- a rule, agreed in advance, for who keeps capacity a team saves (8);
- a register of launches and how firm each is (12);
- a map of when each kind of supply must be ordered (13).

The method of Section 8 writes its answers into the last two.

None of the parts is new. Nesting commitments by lead time comes from hierarchical planning [3]; rewarding recovered cost, from gain-sharing [4]; mapping service costs to consumers, from IT financial management [5]. Hixson and Guliani's capacity planning article supplies the frame: organic versus inorganic growth, and a planning horizon at least as long as resource order time. Its third piece is capacity built on company-wide trends and assigned to a product late (*fungible*: able to serve whichever product turns out to need it) [6]. The 2026 SRE chapter, which Hixson co-wrote, adds the fullest public account of demand types, supply dimensions and pooling [7]. Consensus forecasting [8] and reference class forecasting [9, 10] supply the inside and outside views (Sections 3 and 5.4); supply-chain practice supplies netting, time fences, quantity-flexibility contracts and postponement (Sections 6 and 8). This is a practitioner synthesis, and it describes no particular organization. Where the paper says "in my experience," it marks judgment drawn from practice and the published accounts, not any one organization's process.

Terms. Three carry the paper. The **driver** (volume, *V*) is what causes load, and the **coefficient** (α) is resource used per unit of driver. The **rate** (*r*) is cost per unit of resource, by default the average loaded rate (Section 4). A few more are listed after Section 14. Figure 4 shows who owns each.

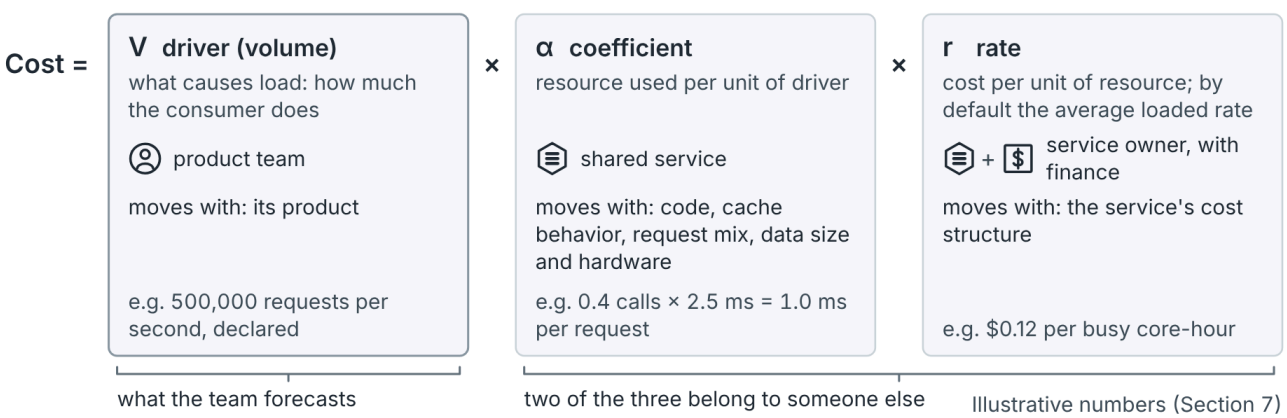


Figure 4. The bill multiplies three numbers. The team that forecasts its demand owns only the first.

2. Why cost drivers exist: the service doesn't cause its own load

In short: a shared service doesn't cause its own load; its callers do, so the cost signal has to reach them.

Tell a shared service team to cut costs. It tunes its code, and consumption keeps growing, because its callers cause the load. Economists call this **derived demand**, and it's why cost drivers exist.

A cost driver is a factor whose change changes an activity's cost. Activity-based costing (ABC) uses drivers in two hops [11]. In infrastructure, bills for equipment, power, space and staff land first on the shared services that use them (*resource*

drivers). Then they pass to the products that call each service (*activity drivers*).

Network shows it most clearly. Inter-region bandwidth is consumed by replication and cross-region calls no product owns directly, and needs its own drivers: bytes replicated per byte written, and cross-region calls per request.

Tracing is one way to find the caller. Dapper's authors could "point to the causal trace root" of expensive inter-cluster traffic rather than stop at the two peer machines [12]. Flux models the capacity service *s* needs in region *r* this way (notation simplified) [2]:

$c(s,r) = \Sigma_p \, a(s,p) \cdot \rho(p,r) + \delta(s,r)$

In words: for each product *p*, take the capacity of *s* attributed to *p* (a share of capacity, not the per-unit coefficient α). Multiply it by the share of *p*'s traffic that lands in region *r*. Sum over products, and add a leftover nobody can attribute. Flux's primary use isn't chargeback, though. It computes "an optimal joint capacity and traffic distribution plan," moving product traffic between regions so demand lands where supply exists; attribution is the input that makes the move computable [2].

Two cautions. First, attribution is an estimate. Traces are sampled (Dapper's first production version kept one in 1,024 [12]), and Flux joins them with fleet-wide profiling [2]. Small callers are estimated least accurately. Second, much of a stateful service's work (compaction, replication, re-indexing, backups) has no request at its root and must be apportioned by declared policy. Figure 5 follows the money in two hops, and the part no trace reaches.

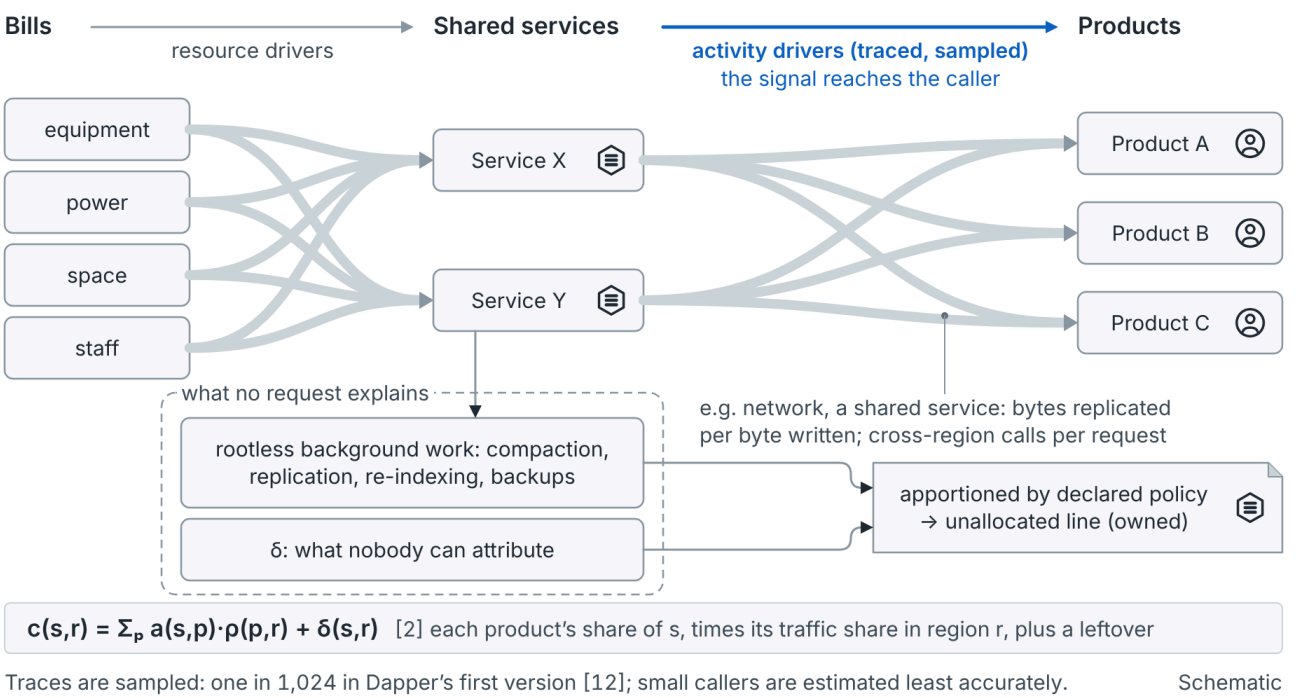


Figure 5. Bills land on shared services first, then follow traced calls to the products that caused them. What no request explains is apportioned by declared policy and kept on an owned line. Schematic.

3. What is published, and what it leaves open

In short: the formula is published. What this paper adds, as far as the sources I could read show, is a rule tying each supply order to how firm a launch is. It also adds the governance that keeps the numbers true.

Before redesigning an intake, look at what's public: three families of work, which differ mainly in when they decide.

Family	Representative public work	When it decides	Question it answers
Statistical driver regression	Uber: per-service 99th-percentile quantile regression on the most-correlated business metric [13]	After observing history	How much will this service need soon?
Coefficient graphs and attribution	Auxon [1]; Flux [2]; Twitter's chargeback on service identity [14]	Each planning cycle	Who causes the load, and what will it be?
Priced allocation	Futures market in computer time, 1968 [15]; quota auctions, 2009 [16]; priority-tiered quota [17]; Quota Marketplace, 2026 [18]	Before resources are assigned	Who gets scarce capacity?

The closest prior art, and the base for much of this paper, is Hixson and Guliani's 2015 article [6]. It treats capacity planning as one problem seen by engineering, product and finance, and asks for the fewest drivers that capture most demand. It suggests weighting actions with "costs" so they normalize into one driver, and builds blow-up factors from design choices. Both are early forms of this paper's coefficient. It asks product to project usage "in the growth dimensions used by the engineering model," and ties the planning horizon to resource order time. It also asks finance how confident the models are. Demand that hasn't resolved, it answers with fungibility. For example, it suggests "building out datacenter space based on past growth trends for the company, but not deciding which product you would fund until right before the machines landed" [6].

This paper works inside that frame and adds mostly mechanics. It works the fungibility answer out layer by layer (Sections 5.4 and 6.5), and adds the governance that keeps the numbers true (Section 9). Charging cost to the callers who cause it, and formal sign-off among the three perspectives, are outside the article's scope; they take most of this paper's words.

Others name pieces. Auxon's performance data state "for every unit of demand X in cluster Y, how many units of dependency Z are used?" [1]. ITIL, the FinOps Foundation, sales and operations planning and the standard capacity texts name the parties and stages [19–24]. The FinOps Foundation's data center guidance, added in 2025, also notes planning horizons "typically requiring months or years of lead time" [19–24].

The discipline is older than any of them. Its oldest root may be telephone engineering: Erlang's 1917 formula for loss and waiting time was soon used by telephone companies in many countries [A1]. Cockcroft and Walker's 2001 book is described, in its publisher's words, as treating capacity planning as "a well known discipline, particularly for sites that have a mainframe oriented background" [25]. Gunther made it tactical in 2007, forecasting from whatever performance data is at hand [26], and Allspaw brought it to web operations in 2008 [24]. At SREcon24, two Elastic engineers argued it should become a more prominent SRE skill [27].

The second edition of *Site Reliability Engineering* (September 2026) has a capacity planning chapter by Gordon, Kirby, Hixson and colleagues; Hixson co-wrote the 2015 article too. The chapter says it explains how Google tackles "operational and mid-term capacity planning" [7]. It splits inorganic demand into planned and unplanned, and asks for probabilistic forecasts and backtesting. It turns an unconstrained forecast into a "capacity plan of record (PoR)" under supply's real limits [7]. It also covers pooling across services, reclaiming stranded capacity and shaping demand to fit supply. Its two

case studies cover Google Meet during COVID-19 and scaling for AI [7]. Much of what this paper builds on is there first, and the sections below say where. When this paper calls a topic missing from the published record, that includes the chapter.

The form has converged; it hasn't been solved, and the authors say so. In 2022, tracing covered 52% of capacity usage in Flux, with over 90% expected with adoption. Flux's residual (Section 2's δ) is 3–5% for a stateless web product. For database infrastructure it fell only from 22% to 15%, "primarily due to improved attribution accuracy and coverage," and load-test models cover 46.4% of capacity [2]. Netflix reports that "Services can have multiple owners, cost heuristics are unique to each platform" [28]. Dapper warned that with coalesced work "a traced request can be blamed for a deceptively large unit of work" [12]. I found no published time series of coefficient drift.

How were the errors shrunk? Flux's authors "improved planning accuracy significantly, primarily by working with service owners to improve their attribution and capacity models" [2]. Borg's users "overbuy" quota as insurance [17]. Flux's first author recalls that before it, each ordering cycle was a negotiation between planners and service owners, "leaving nobody happy in the process" [29]. These are facts about people, reported in engineering papers. Microsoft's researchers describe the supply side the same way. They note that "Cloud supply chain operations were largely executed manually due to their complexity and dynamic nature" [A2].

Step-change demand has a smaller public record, mostly about who supplies the number. Hixson and Guliani define organic growth as coming from "the natural adoption and usage of your product by customers" and inorganic as "step changes, likely the result of a feature launch, a marketing campaign, or business-driven change to how your product is being used," and advise keeping step changes itemized [6]. Google SREs sketched the split in 2020: add planned new consumers to the organic forecast, and make the service behave reasonably under unexpected demand [30]. The 2026 SRE chapter, with Hixson among its authors, splits inorganic demand in two [7]. *Planned* demand is launches, promotions and known events such as Black Friday, accounted for "during launch planning" [7]. *Unplanned* demand is viral trends and news, met with buffers, autoscaling and demand shaping [7]. Hixson and Guliani warn of "a very natural tendency to overstate the potential upside of a launch," and suggest planning for only some of several launches to succeed [6]. Operators add detail:

- **A Google data science team** describes customer forecasts as capturing "future inorganic growth events or trends that are difficult to extract as features for a predictive model," reconciled monthly between customers and finance into base and high scenarios "so that all downstream teams could confidently execute according to the high scenario forecast," with misses traced to "the consensus forecast of record at lead time" [8].
- **Meta's planners** give the forecast to products ("Products forecast capacity: They tell us what they want, how much of it, and where") and the promise to infrastructure [31]. They frame it as an agreement: "Capacity contracts are the commitments between Meta Capacity Planning, Platforms, and Products" [31]. This paper extends the idea from the amount of capacity to the coefficient and rate behind its price.
- **Netflix** predicts new titles from "various internal and external forecasts" and will "use human judgement to pin certain upcoming titles high in the popularity ranking to ensure that we have adequate capacity to serve them" [32].
- **Alibaba**, planning a one-day shopping event, concluded that "Capacity can be predicted, but cannot be planned for, and we can only limit it," and set hard limits on transaction throughput [33].
- **A practitioner account** replaces machine counts with "expected traffic growth, data ingestion, retention, replication" and deducts a team's idle capacity from its future forecast [34].

Outside computing, Kahneman and Lovallo trace optimistic forecasts to an "inside view" that "anchors predictions on plans and scenarios" and neglects "the statistics of the past" [9]. Flyvbjerg's reference class forecasting bases a project's

forecast on the outcomes of comparable past projects [10].

So the sources agree: step changes are declared, declarations run optimistic, and someone must hold them to account. What I haven’t found, in the sources I could read, is a rule tying each supply layer’s commitment to the stage a declaration has reached by that layer’s commit point. That’s the last date its order can be placed. Sections 5.4, 6.5 and 8 propose one. The closest public pieces are in the 2026 chapter. It asks for percentile forecasts and backtesting to choose a “target confidence level,” and for supply’s limits “at different time horizons” [7]. So the chapter has the ingredients. As far as I can read, it doesn’t combine them into one rule per supply layer, tied to how firm a launch is when that layer must be ordered. If it does, I’d welcome the correction. Figure 6 places the published families by when they decide, on Hixson and Guliani’s frame.



Figure 6. The formula is published [1, 2], and much of the frame is Hixson and Guliani's [6]. What this paper adds sits on top: a rule tying supply commitments to how firm a launch is, and the governance that keeps the numbers true. The 2026

SRE chapter [7] publishes much of the middle layer and has the ingredients; as far as I can find, it doesn't write them as one rule per supply layer.

4. Three prices, three questions

In short: "what does capacity cost?" is three questions (who pays, should we build, who gets it now), and each needs its own price.

Ask what new capacity costs and you can get three numbers, each right for a different question. A hotel shows why (Figure 7). The year's bill over the room-nights it can sell gives each guest's fair share. Building a new wing takes another number: its cost per room added. Neither says who gets the last room on a sold-out night, worth what the turned-away guest would have paid. On a half-empty night, one more guest costs little more than clean sheets. Capacity works the same way:

Price	What it is	Question it answers	Misleads when used for
Average loaded rate	Fully loaded pool cost ÷ practical capacity; charged on coincident-peak contribution (Section 5.2) where capacity is peak-sized	Who pays?	Marginal decisions: too high in slack, too low under scarcity
Long-run incremental cost	Cost of the next increment of supply per unit of driver	Should this growth be built for?	Pricing capacity that already exists
Shadow price	What one more unit of the binding resource is worth; zero for slack	Who gets scarce capacity now?	Recovering fixed cost; budgeting

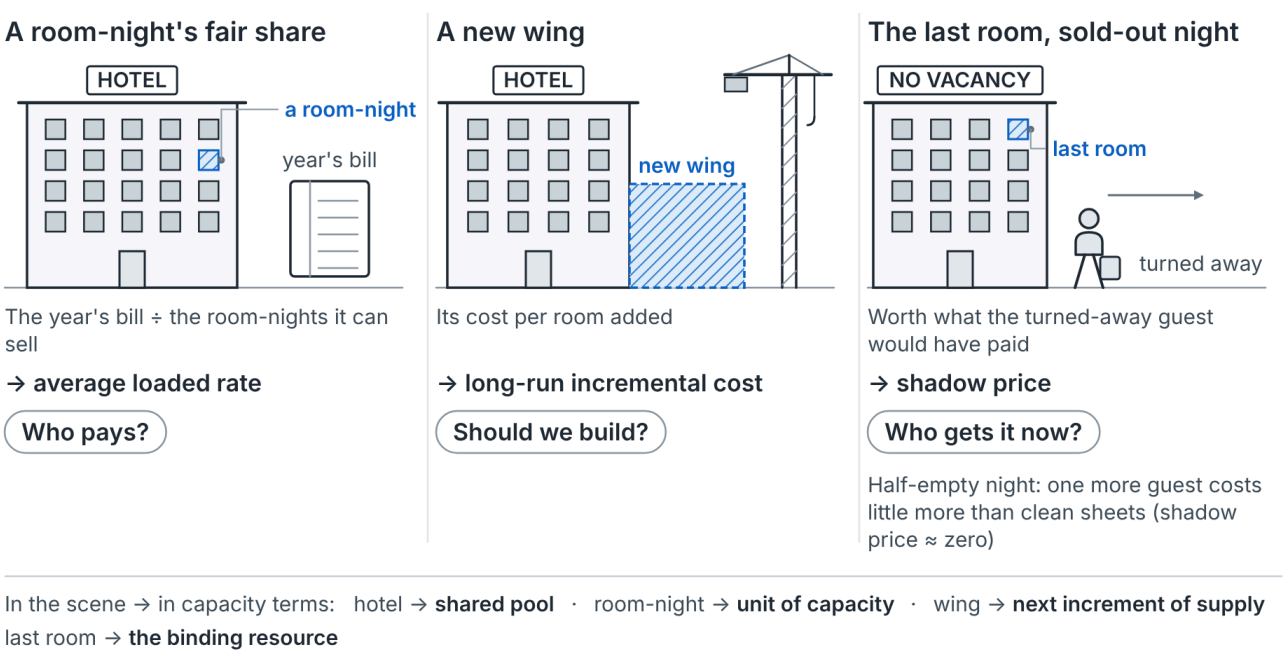


Figure 7. One hotel, three prices: a fair share of what already runs, the cost of building more, and the worth of the last room when it's scarce. Asking one number to do all three misleads on at least one.

Machines are re-bought on a refresh cycle and facilities extended in blocks. So, with stable prices and technology, an attributed average rate approximates the cost of the next envelope (the capacity budget for a planning horizon). It also signals whoever causes demand. It's a poor guide to running one more job on capacity already paid for. In my judgment, that job costs little until capacity binds: mainly energy, plus pulling the next purchase a little earlier. Capacity binds when one resource runs out first (planners call it the binding constraint).

Then one more unit of that resource has a value of its own, like the hotel's last room: the **shadow price**. It's zero for a resource that isn't scarce, and possibly well above its cost share for the one that runs out first. In Section 7, Product A needs 0.8 TB more memory than its CPU-sized machines bring. With only standard 200 GB machines, that's four extra machines, about five times what Section 7's unit weights say that memory is worth (Section 6.1 does the arithmetic). Textbooks call it the dual value of the capacity constraint [35]. Machines come whole, so any computed value is an estimate that jumps at machine boundaries. Internal markets try to discover it, following Hirshleifer's rule of pricing internal transfers at marginal cost [36].

Isn't any split arbitrary? An economist will say dividing joint cost is. Thomas called such allocations "incorrigible," neither verifiable nor refutable [37], and the Technology Business Management (TBM) Council's cost allocation guidance concedes that "choosing appropriate weighting factors can be subjective" [38]. The practical answer is the table: stop asking one price to do three jobs. Two bodies of work narrow what's left. Axiomatic cost allocation (the Shapley value and its descendants) chooses a split by stated properties [39–42]. Peak-load pricing assigns capacity cost to the demand present at the peak the capacity was built for [43, 44]. Neither removes judgment; both make it declared and defensible.

"Fully loaded" must be declared too: depreciation over a stated useful life, contracted power, space, network, staff and ideally a capital charge. Useful life alone is a large lever that moves both ways. Meta extended the useful life of non-AI servers from four years to five during 2022, in two steps [45]. Amazon shortened a subset of servers and networking equipment from six years to five from January 2025, citing the pace of AI and machine-learning technology [46].

Divide by what you can use, not what got used. Illustratively, a $9.0M pool with 65M core-hours *used* costs about $0.138 per core-hour. If one consumer leaves and usage falls to 45M, the rate becomes $0.20. Every remaining bill rises about 44% with no change in behavior, which gives the next consumer a reason to leave. Utility economists call the loop a death spiral [47]. Time-driven activity-based costing breaks it. Set the rate on *practical capacity* (usable after redundancy, maintenance reserve and queueing headroom; Section 6.3), and report the unused remainder as its own owned line, the *unused line* [48, 49]. Illustratively, 75M core-hours of practical capacity gives $9.0M ÷ 75M = $0.12, the rate Section 7 uses. After the consumer leaves, it's still $0.12. The 30M idle core-hours (10M already idle plus the 20M it stopped using), $3.6M, sit on the unused line, not on everyone else's bill. Figure 8 draws both rates.

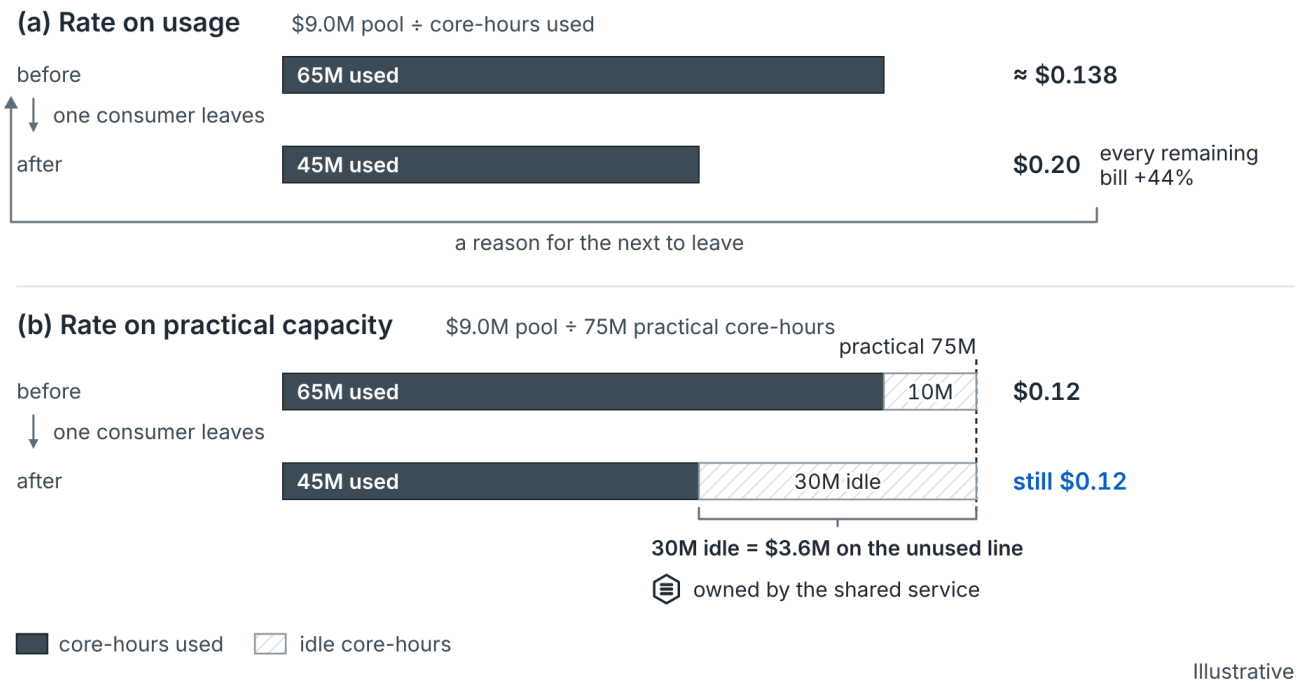


Figure 8. Set the rate on usage and one departure raises every other bill, inviting the next. Set it on practical capacity and the idle remainder becomes its own owned line. Illustrative.

The honest design **attributes** what can be measured and **apportions** what can't (headroom, idle capacity, platform overhead, rootless background work) under a declared policy. Cloud tools encode versions of it. Google Kubernetes Engine (GKE) cost allocation charges on "resource requests, not resources consumed" and reports capacity neither requested nor reserved for system overhead as `kube:unallocated` [50]. Kubecost charges the greater of request and usage, and lets idle cost be hidden, shown as its own line or shared [51].

5. Coefficients are agreements

In short: one coefficient forecasts the service's load and prices the caller's growth, so it needs an owner, versions, and restating when the hardware or the meter changes. Demand with no history enters as a declaration that firms up in stages.

5.1 The grain problem: who caused the load

Put the usage export (jobs by cluster) next to the budget review (products and teams), and neither maps cleanly onto the other. Consumption is measured where it happens: binary, job, resource dimension, location. Decisions are made at another grain: product, team, budget line. Someone has to own the roll-up, and the data a decision needs often sits one level below it.

Twitter (now X) described a chargeback design that began with "a canonical way to identify a service across infrastructures" [14]. But a tag says who owns a resource, not who caused the load on a shared service.

5.2 One coefficient, two readings

In one meeting, a service's planner sizes next year's fleet; in another, a product team defends its bill. Both lean on one number. A consumer's bill is approximately:

Cost = V × α × r

In words: how much the consumer does, times the resource each unit uses, times what that resource costs. *V* moves with the consumer's product; α with code, cache behavior, request mix, data size and hardware; *r* with the service's cost structure. Hixson and Guliani's blow-up model is an early α: a product of design factors, such as 3.12 bytes stored for each byte a user uploads [6].

Take Figure 9's Product A: 0.4 calls to Service X per request at 2.5 ms of CPU per call, so α is 1.0 ms per request. A busy core supplies 1,000 ms of CPU a second, so every 1,000 requests per second keeps one core busy. The planner reads that forward: 100,000 more requests per second means 100 more busy cores. The product team reads it backward: each extra 1,000 requests per second is one busy core-year on its bill, about $1,051 at Section 7's illustrative rate. Figure 9 draws both readings. If α moves, the fleet plan and the bill move together.

So if α or r is opaque and moving, a consumer can know its demand exactly and still not price it. A drifting rate is an unrecorded change to every consumer's plan. The service owes its consumers a legible, stable rate; the planner owes the service a forecast of the load they'll cause.

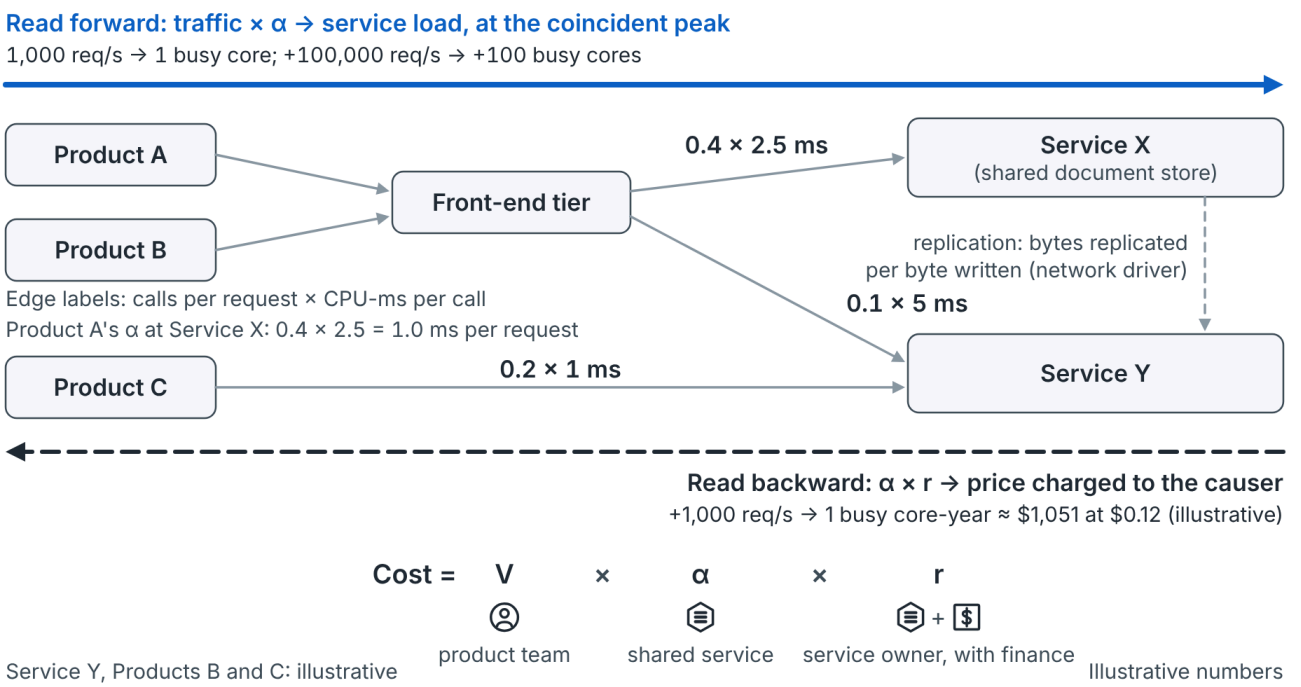


Figure 9. The same coefficient forecasts a shared service's load and prices a consumer's growth, so both sides need it stable and readable.

Decompose the variance. When a bill misses, split the miss by term so each piece goes to whoever moved it:

$$\Delta Cost = \Delta V \cdot \alpha_0 \cdot r_0 + V_1 \cdot \Delta\alpha \cdot r_0 + V_1 \cdot \alpha_1 \cdot \Delta r$$

In words (0 is plan, 1 is actual), swap in one actual at a time: volume first, then the coefficient, then the rate. The pieces add up exactly to the miss, the logic accountants use for standard-cost variances; Section 7 runs the numbers. One choice hides in it: the order. Picture a rectangle growing wider and taller. Each side strip belongs to one move, but the corner grew from both, and someone must say whose it is. Here the overlap lands on the later term, a policy choice; Section 7 shows

how much moves when the order changes. A symmetric method such as the logarithmic mean Divisia index avoids choosing, by treating no term as first [52]. Nor is the coefficient term, often read as “efficiency,” one owner’s: α moves with the service’s code and with the consumer’s mix and data, so record the cause of each restatement.

Version the rate and the coefficient like a price list, with effective dates and a change log. Govern each coefficient as an approximation valid within a range, with an owner, an age and an error threshold that triggers a re-fit. *One way to think about it:* set that threshold in money, not percent, as drift times volume times rate. The SRE chapter’s test (Section 12) applies here. Illustratively, 5% drift on Section 7’s $525,600 bill is about $26,000 a year; on a $40,000 bill it’s $2,000. A threshold of $25,000 re-fits the first and lets the second wait for its age limit. Hixson and Guliani advise checking their blow-up factors regularly to see they “remain accurate” [6]. They also call for writing down the design assumptions a system depends on (a read-to-write ratio, a traffic spread). That’s because demand that breaks them needs engineering first or a consciously accepted risk [6]. Writing them down is the same idea as a valid range.

Measure the ratio, and keep measuring it. Who notices when a coefficient drifts? The 2026 SRE chapter goes further on monitoring, and this half of the loop builds on it [7]. It starts from a driver that makes sense for the product, which it calls “intrinsic product metrics” [7]. Arranged as steps, its advice runs like this (the arrangement is this paper’s, not the chapter’s):

1. **Pick a business driver,** such as daily active users, sales transactions or page views [7].
2. **Instrument the driver and the infrastructure together,** including allocations and utilization [7].
3. **Compute the ratio continuously.** The chapter asks teams to graph ratios such as “cores per 1,000 active users” and alert when they change significantly [7].
4. **Alert on two things:** the ratio shifting, which catches regressions and changes in usage, and, as the chapter suggests considering, consumption that “deviates from predictions” [7].
5. **Route deviations to planning.** If the service handles overload gracefully, a deviation can be “logged as tickets for the planning team” instead of paging on-call [7].
6. **Reuse the ratios before launch,** in “pre-launch performance testing and A/B testing” and in periodic benchmarks [7].
7. **Re-validate the model periodically,** and use the relationships to find “potential efficiency and optimization projects” [7].
8. **Feed it back into design.** Knowing what each unit of business value costs gives teams the data to “influence the product architecture” [7]. The chapter says measuring value against cost can steer future design, “helping to create a virtuous cycle” [7].

The chapter treats the mapping as “a living model that evolves alongside the system” [7]. What this paper adds is the other reading: the same ratio is a price, so it needs an owner and versions. When the measuring loop finds a shift, a change to the ratio is a change to every caller’s bill. So it gets an owner and a new version with an effective date, not a quiet edit.

One way to see it (I haven’t found it in the public record): the chapter’s two alerts are the variance decomposition above, run continuously. The ratio alert watches α, the service owner’s term. For that, it has to compare the ratio with its published version, not just its recent past: a coefficient that was wrong from the start never shifts. The forecast alert, read while the ratio holds at that published value, watches *V*, the product team’s term. So each alert already knows which owner it goes to. The owner still records the cause, since a consumer’s mix can move α too. Neither alert watches the third term, the rate; that’s the price list’s job.

One way to think about it: the re-baseline of Section 10.2 doesn’t have to be a one-off. Run it on a set cadence, or whenever the gap between plan and actual passes a threshold set in money. Then it becomes a *sharpen the estimates* step

in the loop. The gap at each re-baseline is the forecast error. The decomposition splits it by owner: volume to the product team, the coefficient to the service owner, the rate to the service owner with finance. Each owner's record then sets how far its next forecast is trusted, as stage records do for declarations (Section 5.4). It sits close to the chapter's own advice to backtest past predictions and periodically validate the planning model [7]. What's added here, as far as I can find, is scoring the gap by owner. Figure 10 draws the two loops: the chapter's measuring ring and this paper's agreement ring, meeting at the alerts.

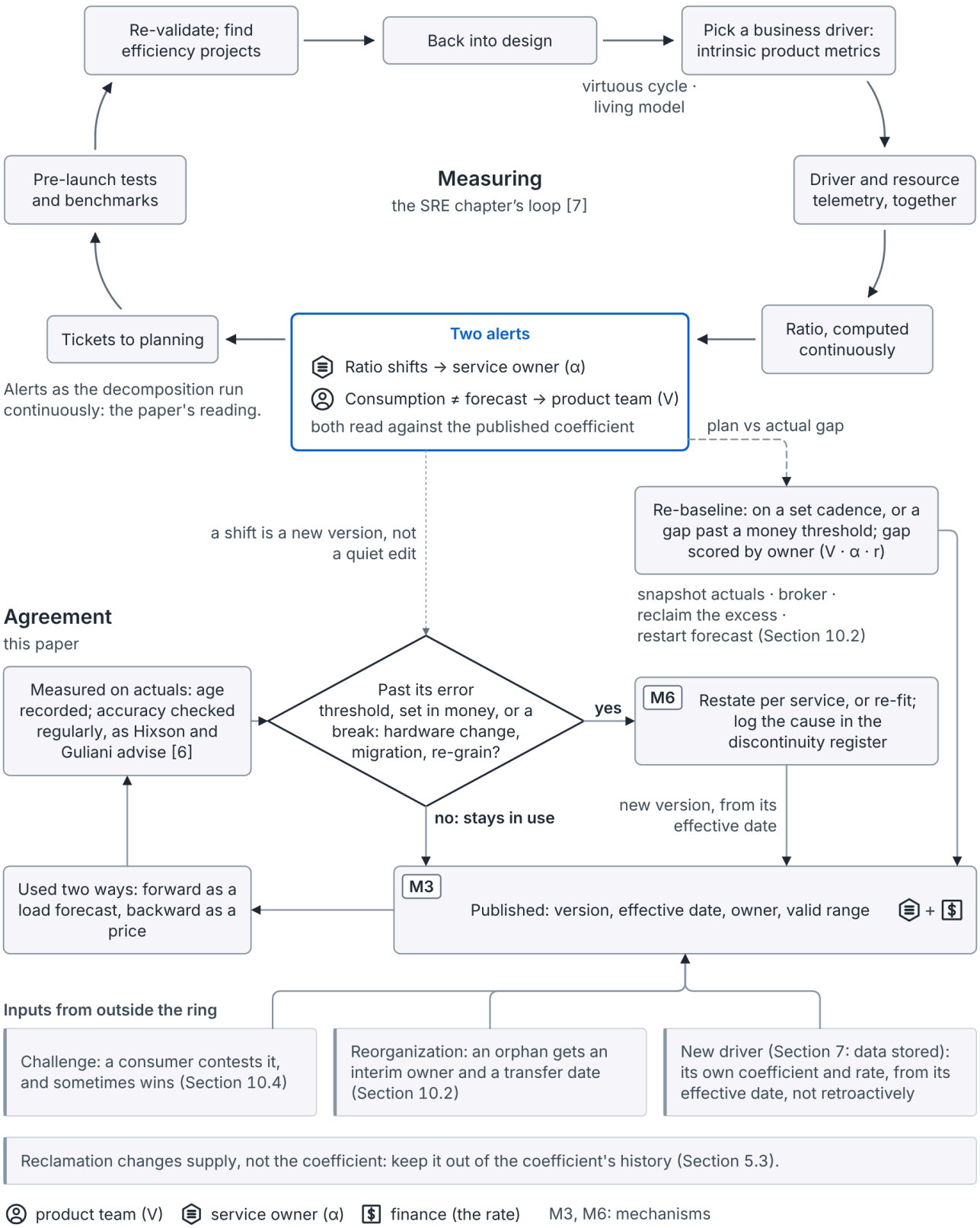


Figure 10. Two rings. The SRE chapter's measuring ring tracks the ratio and raises two alerts [7]; this paper's agreement ring publishes, versions and restates it. The alerts are where they meet, each going to the owner of the term it watches.

The linear form hides two things (Figure 11).

First, capacity is sized to the coincident peak. A road is built for rush hour, not midnight, so a consumer's share is its contribution at that peak. Illustratively, two consumers each peak at 100 cores, one at 2 p.m. (the service's peak), one at 3 a.m. If the night one sends 20 cores' worth at 2 p.m., it caused 20, not 100, and a linear average rate overcharges it. Measure contribution as one of the highest coincident peaks over the horizon (a high quantile; Section 6.3), failover load included. So moving flexible work off the peak, demand shaping, is a capacity lever. The SRE chapter's case for it: "it is cheaper to fix demand than it is to have more and more supply available" [7]. Google's carbon-aware computing does a version of it daily, capping hourly capacity for flexible work while keeping its daily total [53]. A consumer that accepts shaping should see that in its rate. Model providers already price it that way: one batch tier returns results within 24 hours, "at a 50% discount compared to our synchronous APIs" [A9].

Second, redundancy is a multiplier that moves in steps. Surviving the loss of k of n failure domains takes $n/(n-k)$ times peak, assuming load redistributes evenly. The survivors carry the whole peak, so three domains with one loss need 1.5 times peak. Layout sets the multiplier, but the consumer's availability tier sets k, so a higher tier is a driver and belongs in the rate. In practice, k is set per service and cell, often to cover one planned maintenance plus one unplanned loss. And failover load follows routing and latency, not an even split. Illustratively, with four domains, surviving one loss takes 4/3 ≈ 1.33 times peak; surviving two takes 4/2 = 2: half again as much capacity, from the tier alone. Hixson and Guliani note that reliability can be bought "with increasingly large piles of money as you request more 'nines'" [6].

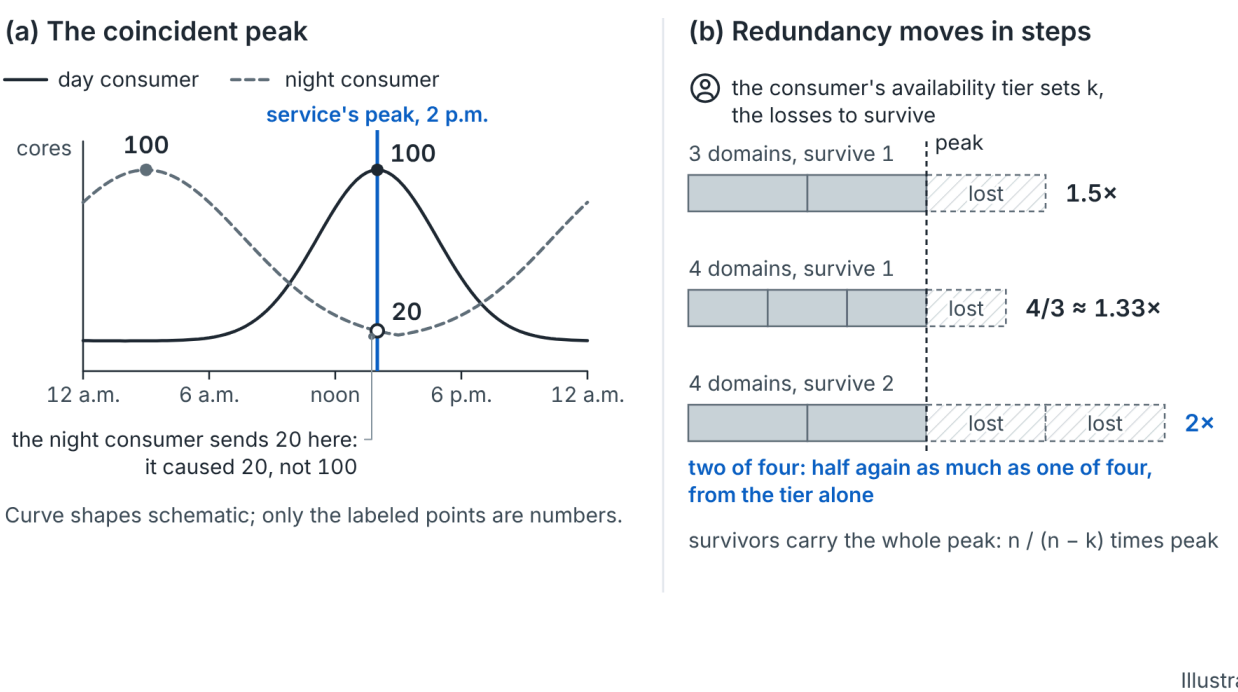


Figure 11. Two things Cost = V × α × r hides: a consumer's share is its load at the service's peak, and the availability tier multiplies capacity in steps. Illustrative.

5.3 Back-adjust through the breaks

Take a CPU-per-request chart with a cliff in it: the quarter new hardware landed. A capacity trend is a smoothed, lagging average that can't tell a one-time jump from a change in direction, so a trend fitted through the cliff misreads next year.

Financial data offers an analogy (Figure 12):

- **A split.** A hardware-generation change, migration or re-grain (a change in how the metric is measured) is a split: restate the coefficient series on the new basis. Unlike a stock split, the ratio isn't one number: a new generation speeds up compute-bound and memory-bound services by different factors, so restate per service, measured. RAS's relative resource units already "take into account a workload's relative performance" on each hardware configuration [54].
- **An acquisition.** A launch with no history adds a declared component to the demand series.
- **Reclamation** isn't a coefficient break at all: it changes the supply series, so keep it out of the coefficient's history.

Price series are adjusted for such events, not forecast through them. Where the ratio is known, restate the history; where it must be estimated, intervention analysis models the break explicitly [55].

In Figure 12's synthetic series, new hardware in Q5 cuts CPU per request by a quarter (×0.75) and a measurement change in Q8 raises it 15% (×1.15). Demand didn't change, only the ruler. A trend fitted through both forecasts too low; restated by the same factors, history is one smooth line.

Simple statistical methods are often "surprisingly effective" [56], and driver models aren't necessarily more accurate; their advantage is being attributable and robust to breaks. So the natural design is a hybrid: smooth the organic baseline statistically, keep a register of discontinuities, and declare launches, migrations and re-grains.

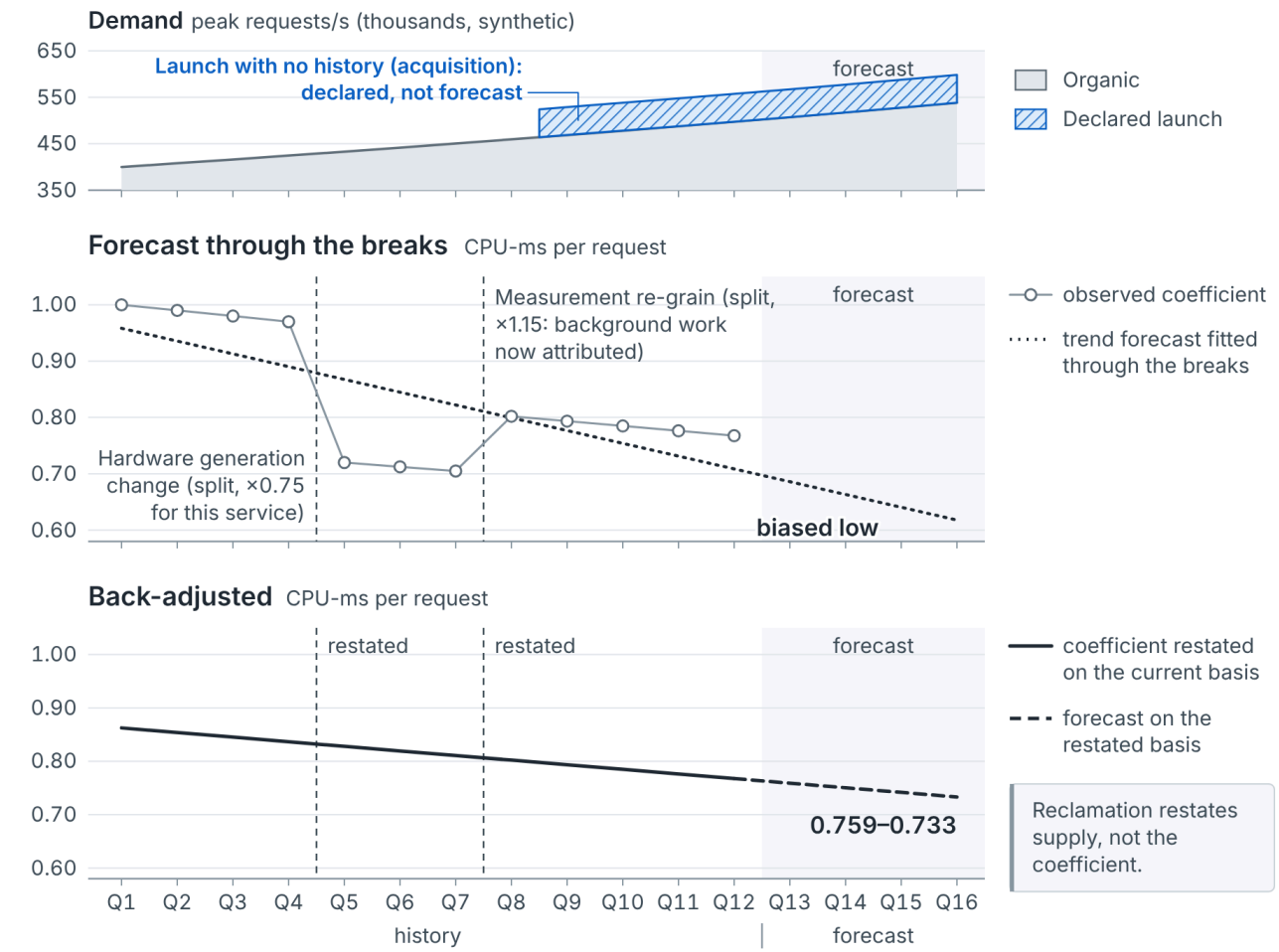


Figure 12. Restate history at known breaks instead of asking a trend to learn them. It's the capacity version of split-adjusting a price series.

5.4 Step changes are declarations that mature

A step change has no history for a trend to extend and no running system to measure. The usual substitute is a multiplier ("we'd double it") nobody can explain.

Specify instead. A launch with no history is usually not new at the resource level. So it enters the plan as a declaration in units the planner already prices, as Auxon and Hixson and Guliani anticipate [1, 6].

A declaration is still a forecast; what changes is who makes it. The team forecasts its product from the inside view, anchored on its plan [9]. The planner forecasts the team's error from the outside view: how comparable declarations turned out [10]. Olavson's team applies this frame to capacity: a customer forecast is checked against a statistical one. A large gap needs "a good story to explain the gap that everyone understands" before approval [8]. What follows extends it to declarations with no history for a statistical check. Where a team can't declare at all, the planner scales the nearest comparable step and names the differences: an analog, not a model.

Declarations firm up in stages. Commonly, a launch is a rumor at the long-range plan, a date a few reviews later, and a configuration at the last review before launch. Its uncertainty drops at decision events (a funding approval, a design review, a go/no-go), not smoothly. Five stages are enough to govern; a stage's *record* is how past declarations made at that stage turned out (scored below). Figure 13 draws the staircase.

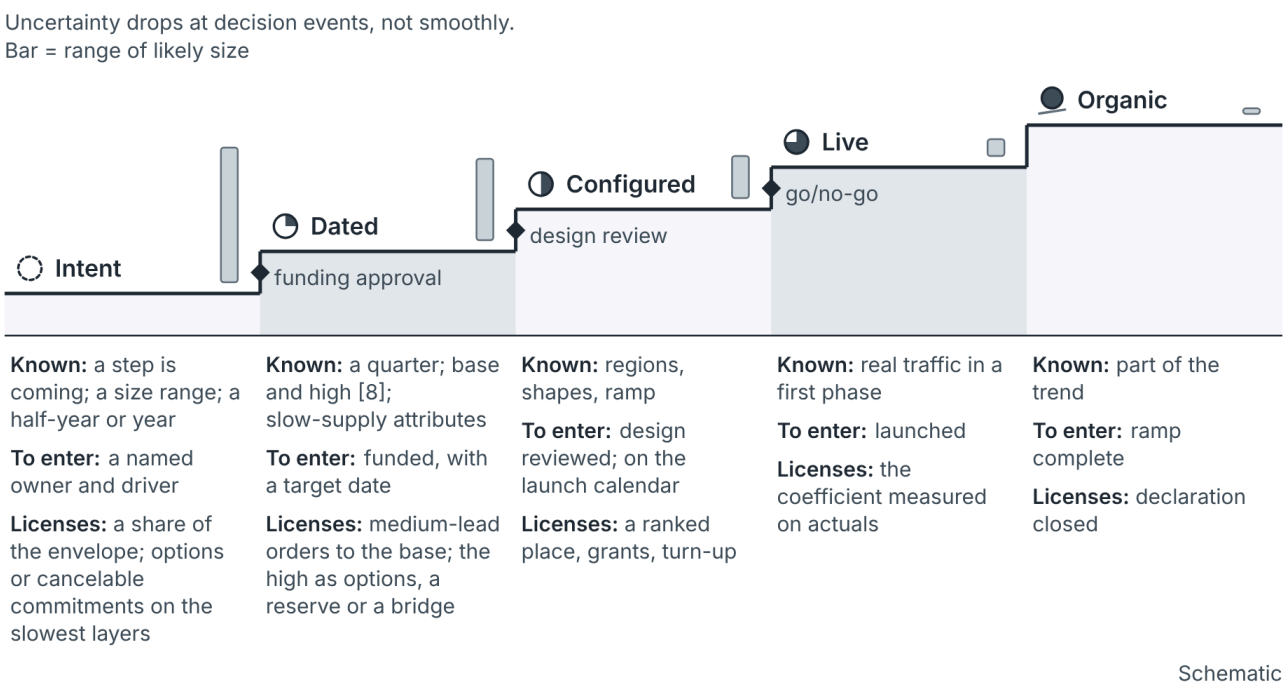


Figure 13. A declaration firms up in stages, at decision events. Each stage licenses only its own supply action, and promotion takes the evidence on its row.

Stage	What's known	Evidence to enter the stage	What it licenses on the supply side
Intent	That a step is coming; a size range; a half-year or year	A named owner and driver	A share of the envelope, sized from the stage's record; options or cancelable commitments on the slowest layers
Dated	A quarter; base and high scenarios [8]; the attributes that drive slow supply (region, redundancy tier, hardware class, data stored)	Funded, with a target date	Medium-lead orders against the base; the high held as options, a named launch reserve or a named bridge
Configured	Regions, shapes, ramp and peak shape	Design reviewed; on the launch calendar	A place on the ranked list; grants (approved capacity assigned to a team); turn-up
Live	Real traffic in a first phase	Launched	The coefficient measured on actuals; later phases sized on them
Organic	The step has become part of the trend	Ramp complete	Declaration closed; the series handed to the organic baseline

Read it down: each row knows more and may commit more (Sections 6.2–6.5 build the supply terms). The rule: a stage licenses only its listed supply action, and promotion needs the evidence in its row. It's a sales pipeline for capacity, with supply actions where a pipeline has win rates (Section 12).

The rollout is the measurement. Measuring can start earlier, too: a declaration at the Configured stage can carry a coefficient measured in pre-launch tests [7], rather than guessed. The last two stages turn a step change into organic demand. Phasing the first region small buys the real coefficient cheaply, and the rest of the rollout is sized on it. Hixson and Guliani note that once you observe actual behavior, “you will have a more accurate understanding” of the impact [6]. Hixson and Guliani ask “how quickly you start to treat past launches as part of the organic line” [6]. The Organic stage answers: the declaration closes when its demand can be extrapolated like any other.

Score declarations by stage. Hixson and Guliani keep step-change estimates itemized because that “gives you room to learn as you repeat the process” [6]; scoring writes the learning down. For step changes, prior actuals in the intake (Section 9.3) need three more fields. They are the stage at which the ask was made, declared size against actual, and declared date against actual.

A team whose intents land at half their declared size but whose configured asks land within 10% (illustrative) is calibrated late, not dishonest. Discount its intents and trust its configurations.

One team’s handful of launches is too few to estimate from. So the planner starts from the record of same-stage declarations by all teams (a reference class). It weights a team’s own record more as it builds up. That’s the way car insurance prices a new driver from everyone’s record, then from their own. Illustratively: intents across all teams land at 60% of declared size, and Team T’s two intents landed at 50%. Count the all-teams record as eight launches (an illustrative weight): $(8 \times 0.60 + 2 \times 0.50) \div 10 = 0.58$. So T’s next intent enters at 58% of its declared size, and each new launch shifts the blend toward T’s own record. Section 7’s planner takes only the first step, using the all-teams record alone. Figure 14 follows one declaration from intent to close, and its score into the next one.

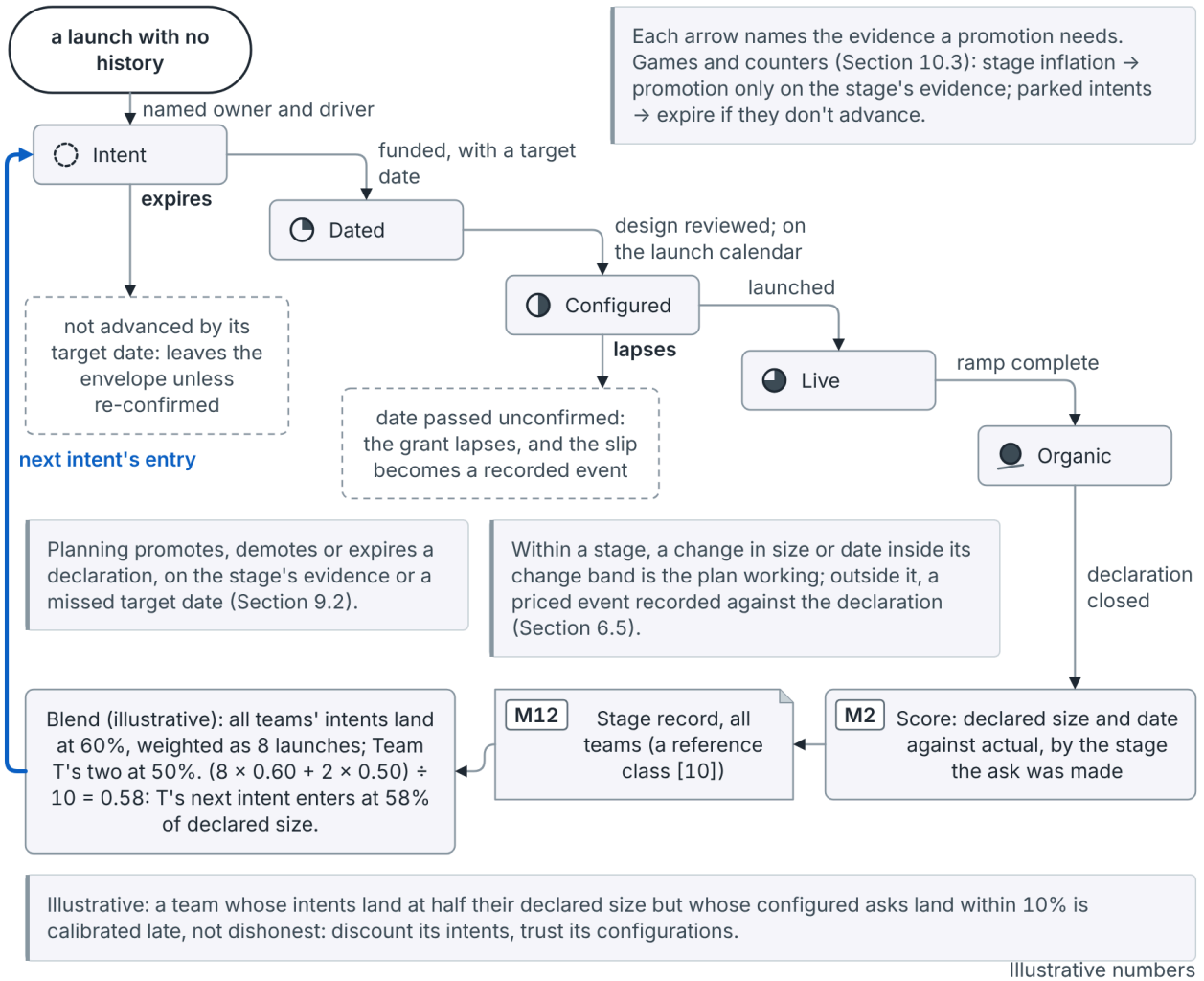


Figure 14. A declaration moves up only on evidence, expires if it stalls, and is scored against what landed. Its record, blended with every team's, sets where the next intent enters. Illustrative numbers.

Size and timing are scored separately because they cost different things (Section 6.6). Recurring events, such as an annual sale or a season opening, are the easy case: the reference class is the same event last time.

6. Approval is not allocation

In short: approving an ask in a common unit doesn't deliver machines of the right shape, in the right place, on time, and the gap needs an owner.

A declaration, however well scored, is only a claim on capacity. Say the planning forum (the standing group that approves and ranks asks) approves it, and months later the team still has no usable machines.

Between a declared forecast and productive work, demand passes through seven steps (Figure 15): declared, approved, allocated to a shape and a place, delivered, live, adopted and productive. Each is a separate act with its own owner, and each can leak; unlike in a sales funnel, every loss is a failure, except designed over-issue (Section 6.1). Hixson and Guliani's lead-time table runs a similar line, from generating planning numbers through approving and provisioning

resources to “Ready to serve” [6]. The funnel adds adoption and productive use after capacity is live. These steps are not Section 5.4’s stages: a stage says how firm a declaration is, a step how far approved demand has traveled. The slow steps must start before the declaration has firmed up, which Sections 6.5–6.7 take up.

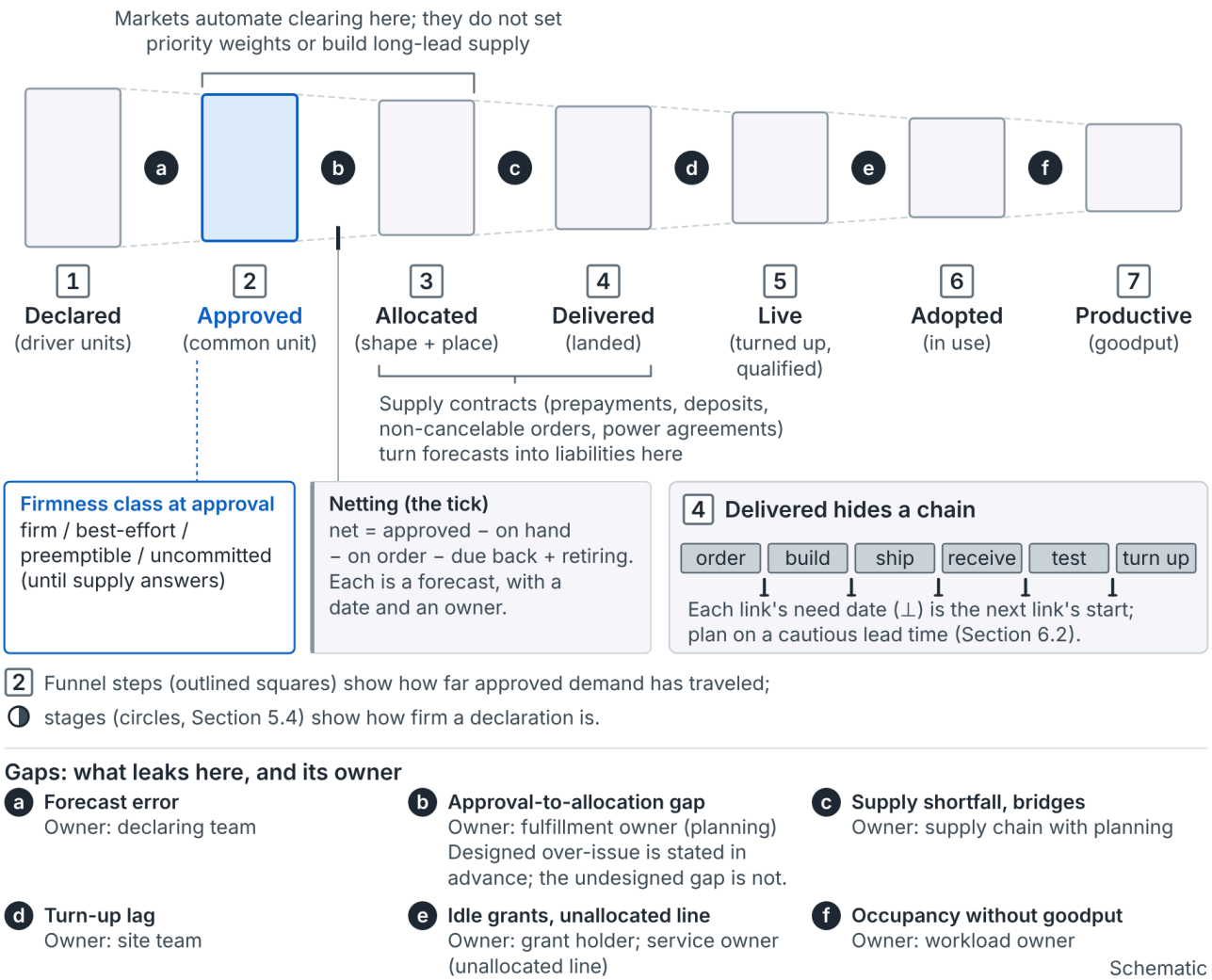


Figure 15. A common unit prices demand so it can be approved. The steps after that are where ownership most easily goes unassigned. Hixson and Guliani’s lead-time table runs a similar line [6]; the funnel adds adoption and productive use after capacity is live.

“Delivered” hides a chain of its own (order, build, ship, receive, test, turn up), and each link can leak: a vendor slip, a failed test, a queue at turn-up.

Netting: the step between approved and allocated. Every cycle, supply planners take approved demand, subtract what’s on hand, on the way, on order or due back from reclaim, and add back what’s scheduled to retire. The rest, the net need, is ordered part by part, each on its own lead time: the core of manufacturing’s material requirements planning. Here’s the catch: every number in that subtraction is a forecast too. A reclaim that doesn’t happen, or a decommission that slips, becomes a shortage that looks like a supply failure. In my experience, many undesigned gaps are born there, in the netting just after approval.

The rest of the section sets out the problem (Sections 6.1 and 6.2), then the tools: buffers, bridges and timing (Sections 6.3 to 6.5). Then come the ways a step can miss on either side (Sections 6.6 and 6.7).

6.1 A common unit prices; it doesn't clear

Planning functions normalize asks into a common unit (a normalized machine, a dollar, a megawatt) so executives can compare and approve unlike demands. RAS's relative resource units are a published example [54]. Section 7 invents one, the normalized machine unit (NMU). A standard machine of 50 cores and 200 GB is 1 NMU, and a high-memory machine of 50 cores and 600 GB is 1.4. Its forum approves an ask as 4 NMU; the cheapest machines that fit it come to 4.8. That gap starts here, because an approved 1,000 units must still become:

- a CPU-to-memory ratio;
- storage capacity *and* access rate (IOPS), which are separate resources;
- a placement within latency and redundancy limits (Auxon's example: within "30 milliseconds of network latency" of the caller [1]);
- and hardware that exists in time.

The unit's weights decide which asks look cheap. Section 7 weights by component cost: a core is 0.016 NMU, a GB 0.001. But under scarcity, the binding dimension can be worth more than its cost share when the machines on hand can't match the ask's shape. Say memory binds and only standard machines are on hand: every extra 200 GB brings 50 cores along, needed or not. Those cores are **stranded**: left unusable because another dimension ran out first. Counting them, each extra GB costs 1 NMU ÷ 200 = 0.005 NMU, five times its weight; that's memory's shadow price here. Swap in a high-memory machine and 400 more GB costs 1.4 − 1.0 = 0.4 NMU, or 0.001 per GB. Memory still binds, but nothing is stranded, so its shadow price falls back to the weight.

So a cost-weighted unit under-prices asks heavy on what's scarce, and approving in it predictably creates stranding for fulfillment to absorb. Re-weighting toward the binding dimension each cycle is a partial fix; a shadow price is the full one. Stranding occurs per machine, rack (power against slots), hall, failure domain and network port, mostly fixed when hardware is physically placed, months before any job runs. Microsoft's Pond paper measures the mirror case: "up to 25% of DRAM becomes stranded as more cores become allocated to VMs" [57].

Fulfillment is packing boxes into bins with both a weight and a volume limit, where some boxes fit only certain bins. Published optimizers treat it as just that: multi-resource (vector) bin packing under placement constraints. Auxon represents requirements as "a giant mixed-integer or linear program" [1]. RAS re-assigns servers to reservations with a mixed-integer solver "every few tens of minutes" [54]. Flux keeps each region within "its available supply" [2]. Their solvers' outputs are one way to estimate Section 4's shadow prices.

Much efficiency comes from **re-shaping** an ask rather than cutting it, such as a faster storage tier in place of a design with many more disks, when access speed binds. Before the order is placed, that's nearly free; after, it can mean change fees, rework, a new test cycle and a new lead time (Section 7). The SRE chapter's Meet case shows re-shaping at work: doubling CPU and memory per instance halved the instance count [7]. In my experience, re-shaping is a conversation more than a calculation. An ask often carries a design assumption the team didn't know it made, such as putting everything on spinning disk. Question the total, and you get a defensive answer. Question the assumption ("which part of this really needs fast access?"), and you often get a different mix, such as fewer spindles and a slice of flash. The team ends up with a better design, not a haircut, and that's why it agrees.

Not every gap is a failure. Borg limits production-priority quota "to the actual resources available in the cell" and deliberately over-sells lower priorities: "every user has infinite quota at priority zero" [17]. Quota Marketplace credits "are fully decoupled from physical resources" [18]. Google's pooled quota, as the SRE chapter describes it, "relies on *oversubscription*" [7]. A stockout "occurs when oversubscription fails," so pooled services need "stockout resiliency" [7]. Three cases follow:

- *Capped issue* limits approval to sourceable supply.
- *Designed over-issue* sells entitlements beyond supply under a stated firmness class (firm, best-effort, preemptible), so holders know their odds.
- The *undesigned gap* is an approval treated as firm that supply can't meet in the shape, place or time asked.

An approval becomes capped issue when supply answers it with a dated commit or a named gap (Section 10.1). Until then, label it *uncommitted*, a firmness class of its own, so its holder knows the odds.

The gap itself isn't new. Hixson and Guliani list approving and provisioning resources as separate steps [6], and Auxon's allocation plan reports requirements it could not satisfy [1]. The 2026 SRE chapter states the goal plainly: capacity has to reach "the right consumer, in the right location, at the right time" [7]. That's "not a standard multidimensional bin-packing problem," because you can reshape both the bin and what goes in it [7]. Its figure of stranded and overloaded cells by resource family and region is this section's problem in one picture [7]. Cloud providers say it in their documentation and guidance, too: "Quota and capacity are separate checks" [58], and "Capacity does not equal quota" [59]. What this paper adds is an owner and a ledger for the gap between approval and allocation (mechanism 5).

6.2 Supply is ordered long before it arrives, and bought through contracts

Picture trucks already on the highway: what arrives next quarter was ordered long ago, how long ago depending on the layer. Hixson and Guliani give the order time as an example: it "could be five years if you erect your own buildings," against "10 minutes if you run a small service in a cloud and have good automation and a credit card on file" [6]. The table gives public values where they exist; Appendix B has the sources.

Layer	Lead time	What the public record says
Land, shell, utility power, large electrical gear	Years	One operator: capacity "planned years in advance due to long lead times for land, utility infrastructure, and construction of physical buildings with cooling infrastructure" [8]. Grid connections and large transformers each take years [60–62] (Appendix B).
Long-haul network	Months to years	Meta's backbone planners describe amplifier-site space and power, and optical and IP gear, as "taking months or even years to deliver." They plan in two horizons: "the next 6 months to 2 years" and "2 to 5 years in the future" [63].
Machines and parts	Quarters	One operator: "over 6 months for some components" [8]. One accelerator vendor: "extended lead times of more than 12 months" in its own supply [64]. No published purchase-order-to-dock time found.
Turn-up	Weeks per job; months to staff	No published turn-up duration found.

Power needs care. A large new load waits in the utility's own queue, not the one for new power plants, and little is published on how long that takes [65] (Appendix B). Where nothing is published, the paper uses a plainly hypothetical

value. For example, if standard servers take one quarter and a new accelerator platform takes three, the two need different commit points even inside one launch. Figure 16 shows the layers and their lead times.

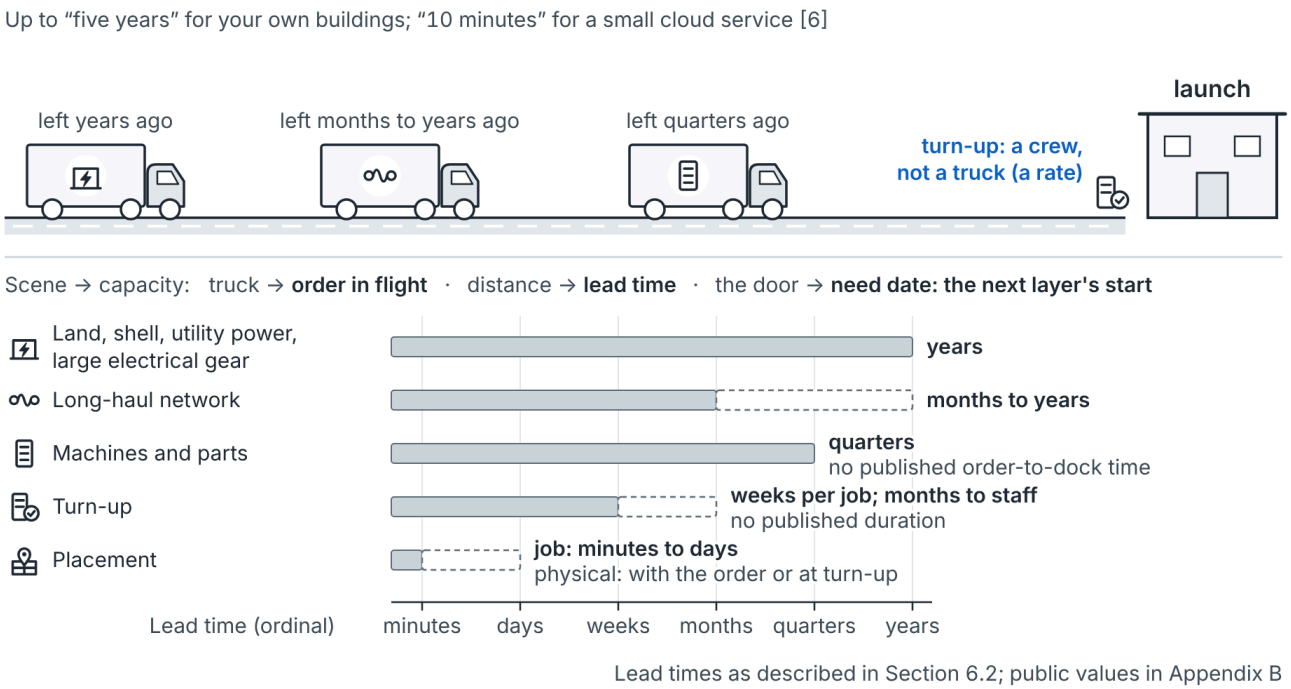


Figure 16. Supply is already on the highway. The slower the layer, the longer ago it had to leave, so each layer has its own commit point even inside one launch.

Placement is two decisions on different clocks. *Physical* placement picks the site, hall, power domain and network fabric. It’s fixed with the order or at turn-up, costly to undo, and decides which demand a machine can ever serve. *Job* placement puts work on existing machines in minutes to days; Flux re-cuts regional placement “in weeks-long cycles” [2].

Each lead time is a chain, a spread and partly a queue. The slowest link sets the pace (in my experience, not always the same link). In my experience, the spread is lopsided: late arrivals are common, early ones rare. For example, if machines usually take one quarter but one order in five takes two, a commit point one quarter out is a gamble in a bad year. So this paper plans on a *cautious* lead time, near the slow end of what’s been seen (Section 8). And since vendor allocation, test positions and turn-up crews are limited, lead times stretch when everyone orders at once.

Layers stack. The hall must be ready before the machines, the machines before turn-up, turn-up before the load tests that precede launch. So a layer’s *need date* is the next layer’s start, not launch day. Get the order wrong and the fast layer waits on the slow one. Microsoft’s chief executive described exactly that: “It’s not a supply issue of chips; it’s the fact that I don’t have warm shells to plug into” [66].

Turn-up is a rate, not an order. It’s set by people, dock space, test positions and network ports, staffed months ahead, so it works like a checkout line. For example, if a quarter’s deliveries fill four halls, turn-up takes two weeks per hall and a site works two halls at once, they take a month to go live. A slipped batch of four more halls landing alongside waits a month longer. So a supply slip often shows up twice. In my experience, per-site turn-up throughput, with retests and refresh work competing for the same crews, often binds, and delivered-but-not-live is where fulfillment backlogs quietly age. Figure 17 draws the three adjustments.

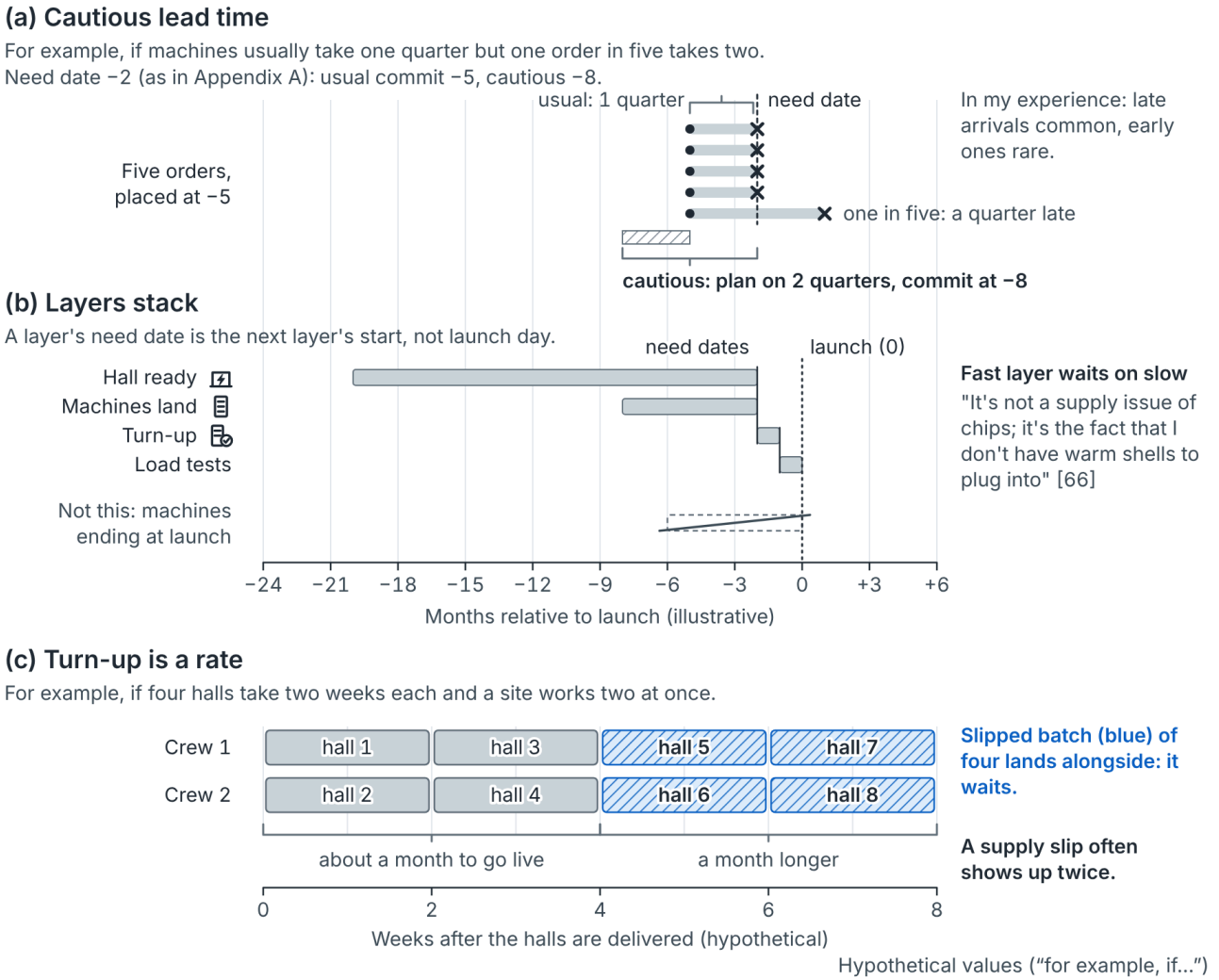


Figure 17. Three adjustments that move a commit point earlier than “launch minus lead time”: plan on the slow case, work back from the next layer’s start, and remember that turn-up is a queue. Hypothetical values.

Padding travels upstream. Padded asks, a rational response to expected rationing, get amplified into supplier orders: the “rationing game,” one of the four causes of the bullwhip effect Lee and colleagues list [67]. Illustratively, if three layers each add 10% to be safe, the order at the top is $1.1^3 \approx 1.33$ times real demand. When supply loosens and the padding comes off, orders drop as sharply.

Supply is bought through contracts as much as orders: prepayments and deposits, take-or-pay and capacity commitments, and non-cancelable orders. There are also power agreements and tariffs with minimum bills that run for years (Appendix B has examples). A signed contract turns its forecast into a liability, owed whether demand arrives or not once its cancel window (the last date you can still walk away cheaply) closes. So each contract needs signed demand behind it and a register entry with the liability, the cancel window and an owner. The risk operators name is a mismatch: long commitments behind shorter customer contracts [68, 69].

A cancel window is rarely one date. What you owe usually rises in steps toward delivery, and some orders are non-cancellable from the start. For example, if cancelling a year out costs a tenth of the price, a quarter out might cost half, and inside the vendor’s build time all of it. These are hypothetical numbers; I found no published terms of this kind. Many

agreements also let you push a delivery out, or pull it in, for less than cancelling. Since the most common timing miss is a slipped launch (Section 6.6), the right to push out is often worth more than the right to walk away. Figure 18 draws the steps.

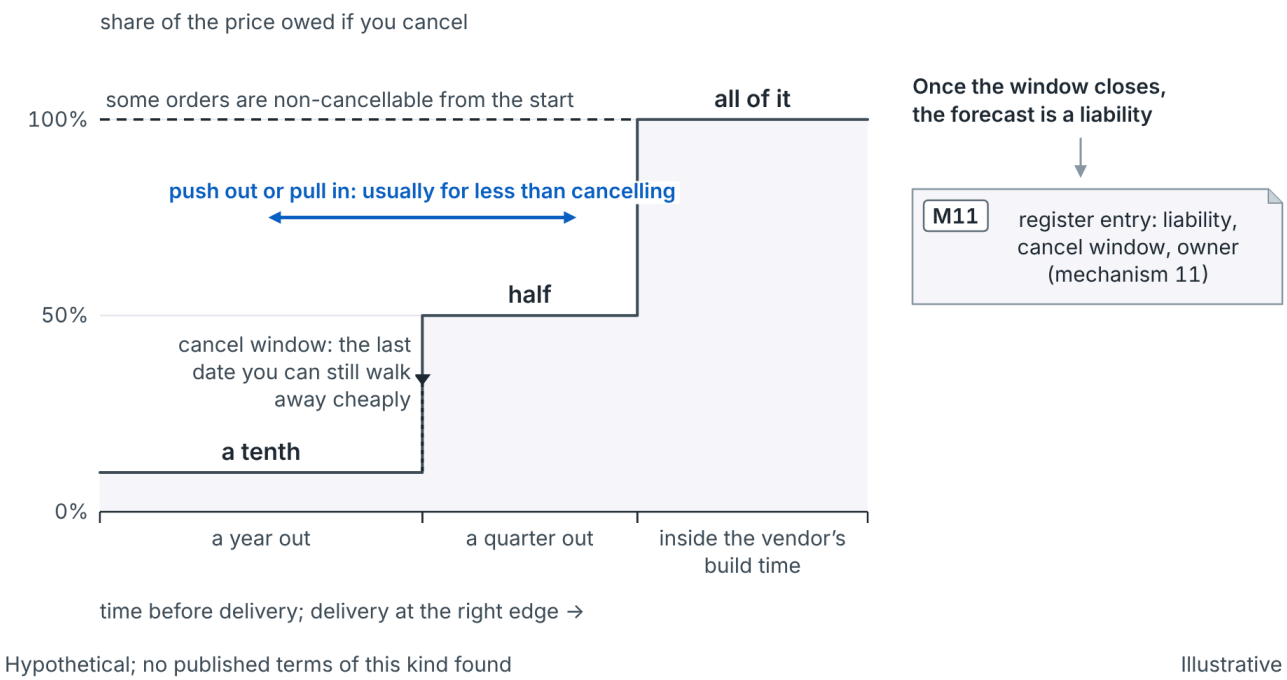


Figure 18. What you owe on cancelling rises in steps toward delivery, and some orders are non-cancellable from the start. Since slips are the most common timing miss, the right to push a delivery out is often the more valuable one. Hypothetical numbers; no published terms of this kind were found.

Quantity flexibility bounds how far an order may move. Tsay describes one such contract. It "couples the customer's commitment to purchase no less than a certain percentage below the forecast with the supplier's guarantee to deliver up to a certain percentage above" [70]. In its rolling form, each period's quantity may be revised only by a set percentage per planning round, so more change is tolerated far from delivery and less near it [71]. That's supply's own version of Section 5.4's stages, and Section 8 borrows it.

Suppliers ration, and they score you. In my experience, constrained parts are allocated as much as sold. Microsoft, short of capacity for many quarters, said: "We say no to some of the demand that may be something that we could serve, but it's not in our long-term interest" [72]. So a buyer of constrained memory, flash, accelerators or optics faces a second clearing problem (Section 6.4), run by a supplier deciding whose forecast to believe. It has the same reason to discount a buyer whose orders have run high as the planner has to discount a team's intents; that's my reading of the rationing game [67]. So send a vendor the base, hold the high as options, and keep internal padding from reaching it.

The rule follows: **commit early only to what is slow, defer what is fast, and make early commitments fungible** (able to serve whichever launch turns out to need them). Operators already do. Microsoft calls its data center leases "very long-lived assets," at "15 to 20 years," while for short-lived chips "the lifetimes of these and the lifetimes of the contracts are very similar" [72].

Every step down the chain spends some fungibility. Powered land serves almost any launch; a shell with a cooling design, fewer; a tested rack in one hall, only demand of its shape that can run there. Supply chains answer with *postponement*: keep supply generic until the decision that says which shape, as Feitzinger and Lee describe at Hewlett-Packard [73]. In capacity, commit to buildings, power and common parts on the slow clock, and to configured machines on the fast one. A shell stays fungible only if its specification (power per rack, cooling type) is written against the high scenario. In my judgment, a shell built for legacy air-cooled density may not host current accelerator racks. Figure 19 shows the chain.

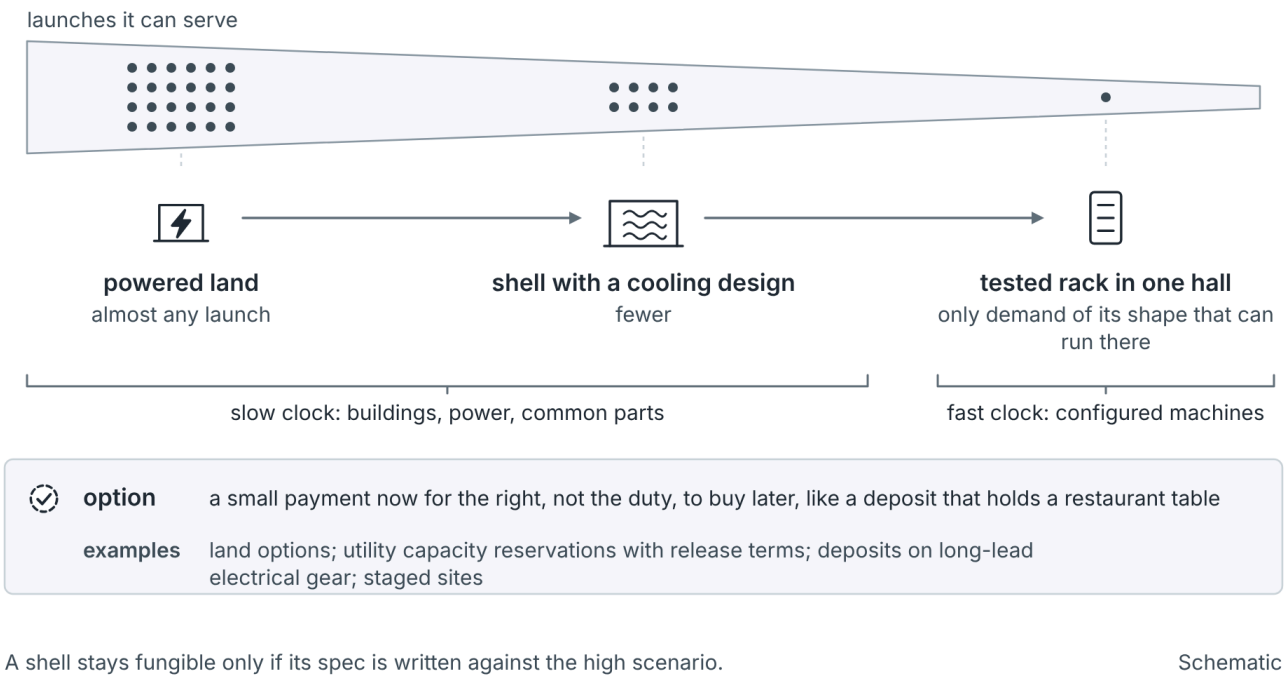


Figure 19. Postponement [73] in capacity: keep supply generic until the decision that says which shape, and hold the slow, generic layers open with options.

Options keep the slow layers open. An *option* is a small payment now for the right, but not the duty, to buy later, like a deposit that holds a restaurant table. For most hardware it's a cancelable or reschedulable order with a fee schedule. In practice:

- take options on the long-lead layer (land options, utility capacity reservations with release terms, deposits on long-lead electrical gear);
- stage sites ahead of need: Meta describes "staging data center sites so we can spring up capacity quickly in future years as we need it," and partnerships "that give us option value for future compute needs" [74]. For one campus, it leases on "a four-year initial term with options to extend," backed by a residual value guarantee for the first 16 years [75]. That's a short commitment plus options on a long-lived asset;
- buy the medium-lead layer against a base scenario;
- write down the trigger for converting options into orders.

The option value of waiting argues for buying late [76]; lead time and shortage cost argue for buying early. A written table of substitutions per workload class keeps commitments fungible, and needn't be complete. Jordan and Graves found that "limited flexibility (i.e., each plant builds only a few products), configured in the right way, yields most of the benefits of

total flexibility," with the most benefit when it chains "products and plants together" [77]. Here's the read-across to capacity (an analogy, not a result tested on fleets). If each workload can run on two machine shapes and the pairs link into a chain, you get most of the benefit of making everything run everywhere.

6.3 Buffers are a priced decision

In short: name each buffer and size it on which mistake costs more, running short or holding spare. Long lead times push buffers up; pooling pulls them down.

Consider a plan with one line called headroom, a percentage nobody can explain. Buffers come in at least six kinds, each with its own owner and sizing. They are redundancy (N+k), maintenance reserve, queueing headroom, forecast-error buffer, launch reserve (capacity held against a declared high scenario) and inventory buffer (parts and ready machines in stock). RAS publishes one operator's split: about 94% of servers as guaranteed capacity, 2% for random failures and 4% for correlated failures [54]. Hixson and Guliani's "blow-up" factors itemize storage the same way: 1.2 for RAID5, 1.3 for overhead and 2 for a second site come to 3.12 bytes per byte stored [6].

Not every buffer is installed. One operator describes its stock. It says "we would stock inventory of component parts based on forecasts so that we could quickly build and deliver machines," and "Component inventory is planned to high quantile forecasts" [8]. A stocked part can become many machine shapes, so its buffer pools across all of them (postponement, Section 6.2) and can be smaller than the installed buffers it protects. Someone funds and replenishes it, so a draw on it is a debt with a date (Section 7).

How much to hold. A newspaper seller stocks copies before knowing how many will sell; the right number depends on which mistake costs more. So compare the cost of running short with the cost of holding spare: this paper calls that comparison the *shortage-to-idle ratio* (textbooks call the whole problem the newsvendor problem). It sets how far up the range of likely demand you buy:

$$C_u/(C_u + C_o)$$

In words: the cost of being short, over the sum of the costs of being short and being over [78]. Picture ten equally likely outcomes for next year's demand, lowest to highest. If being short costs four times what a spare unit costs (a 4 : 1 ratio), the fraction is $4/(4 + 1) = 0.8$. So cover the eighth-lowest, and accept running short in the two above it. Hixson and Guliani frame the same choice without the formula. Within the planning horizon, "you must understand the risk of spending too much money, or of running out of capacity," and "That tolerance should define how aggressively you provision" [6]. The SRE chapter says capacity planning "heavily parallels the concept of error budgets", and "provisioning for every statistical edge case is the wrong target for capacity" [7]. Its ancestor is the first SRE book's "100% is the wrong reliability target for basically everything" [79]. Figure 20 pictures it.

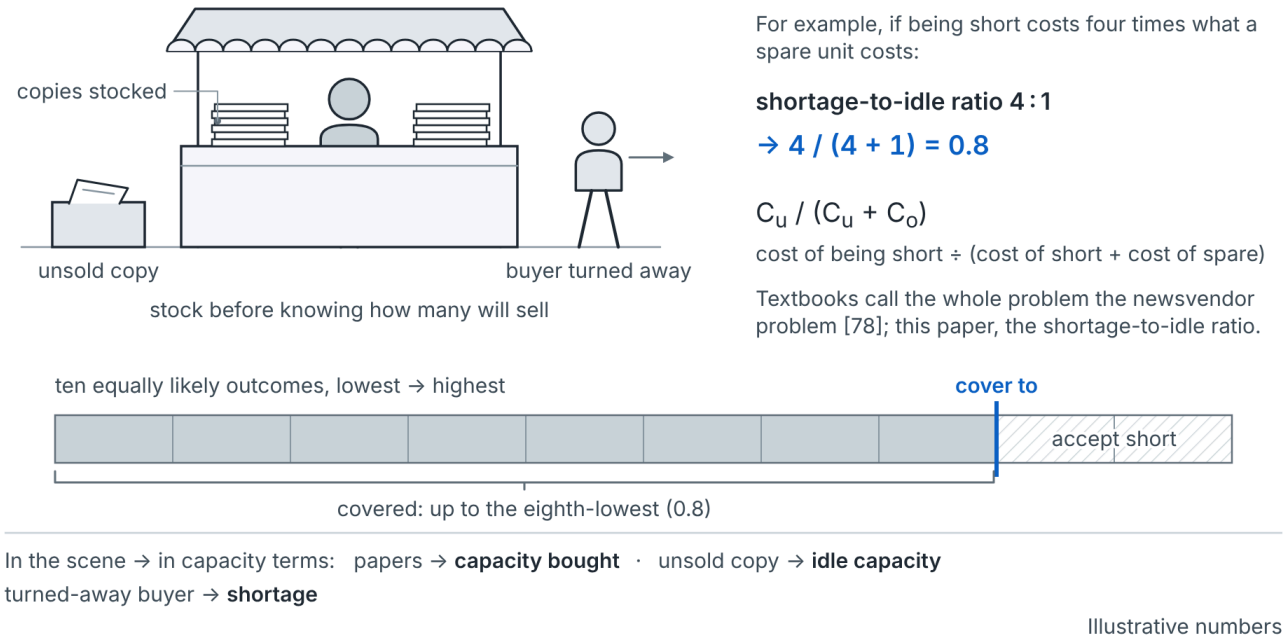


Figure 20. Buy up the range until the next unit is as likely to sit idle as to be needed, weighted by which mistake costs more. At 4 : 1, cover the eighth of ten outcomes. The rule is the textbook newsvendor formula [78].

For durable capacity under growing demand, the cost of being over is, in my reading of the capacity-expansion literature [80], mainly carrying a unit until demand grows into it. That means capital tied up, depreciation while it waits, power if it's energized, and the newer generation you didn't wait for. For machines the last two are material; for shells and power, small.

The cost of being short is harder to see, and it's usually argued, not measured. The SRE chapter points to the evidence: "Always measure when demand shaping occurs," because the record shows the value of the shed work [7]. It asks teams to choose intentionally between investing more in supply and accepting the cost of shedding [7]. So, *one way to think about it*, the shed and degradation log is product's half of the ratio, as carrying cost is supply's. For example, say a quarter's shed events cost about $200,000 in lost sales. The spare that would have prevented them would have cost about $50,000, about the size of the shortfall. Then the log points to roughly 4 : 1 for that layer (illustrative). It's a starting point for the signature, not a formula.

Lead time inflates buffers. Safety stock, the buffer against forecast error, must cover whatever demand does while the trucks are on the highway. If demand is independent from period to period, safety stock grows with the square root of lead time [78]. Illustratively, a ±10 monthly wobble becomes about ±20 over four months ($\sqrt{4} = 2$), not ±40, because ups and downs partly cancel.

In my experience, capacity forecasts rarely behave that way. Errors run in the same direction cycle after cycle (statisticians say positively autocorrelated), so they don't cancel; a growth-rate error keeps adding up, like a ship one degree off course. Illustratively, forecast 3% monthly growth and get 4%, a miss that could as easily have gone the other way. Then demand ends about 12% above plan after a year ($1.04^{12} \div 1.03^{12} \approx 1.12$), and about 26% above after two. So the error at the need date can grow linearly with lead time or faster. Measure it: score past forecasts by horizon and size the buffer from that curve. Persistent bias is different: correct it at intake (Section 9.3), don't buffer it.

The effect can be startling. One operator's first statistical forecast of component demand "required us to triple our safety stock inventory in order to meet our on-time delivery goals," and was never implemented; the team then set a separate forecast quantile per component type [8]. And since, as Section 6.2 notes, deliveries are rarely early and sometimes very late, the buffer must cover a late truck as well as a demand surprise. Section 8 does that with a cautious lead time.

Pooling is the main economy. Two regions whose bad days don't coincide need less than two full spares. For n identical, uncorrelated normal demands, pooled safety stock is $\sqrt{n}$ times one location's rather than n times [81]. Illustratively, two regions that each hold a spare of 10 need about 14 together, not 20; four need about 20, not 40. The 2026 SRE chapter makes the same point at the scale of a fleet. Provision for "the peak of the *summed* demands, rather than the sum of their *individual* peaks". In its example, pooled demand peaks at 48 units against 60 provisioned separately, and still needs a smaller buffer [7]. Google reports that pooling this way saves "approximately 5–20%" in its systems, and that easy access to capacity "reduces the incentive for teams to hoard" [7]. Correlation and heavy tails erode the gain [82]. Joe Weinman, whose book the chapter cites for this math, puts the condition plainly in a later article: "the assumption of workload independence is a key one" [83]. Queueing headroom pools the same way: at a fixed probability that a request must wait, a many-server pool's spare grows roughly with the square root of load [84]. Easy access to the pool reduces hoarding, the chapter notes [7]. I'd add that its buffer should be priced, so it isn't re-hoarded. Pooling has a counterweight, too: Meta describes moving AI training "from a model of scheduling globally fungible resources to pre-regionalized capacity contracts" [31]. Figure 21 puts the section's three ideas side by side.

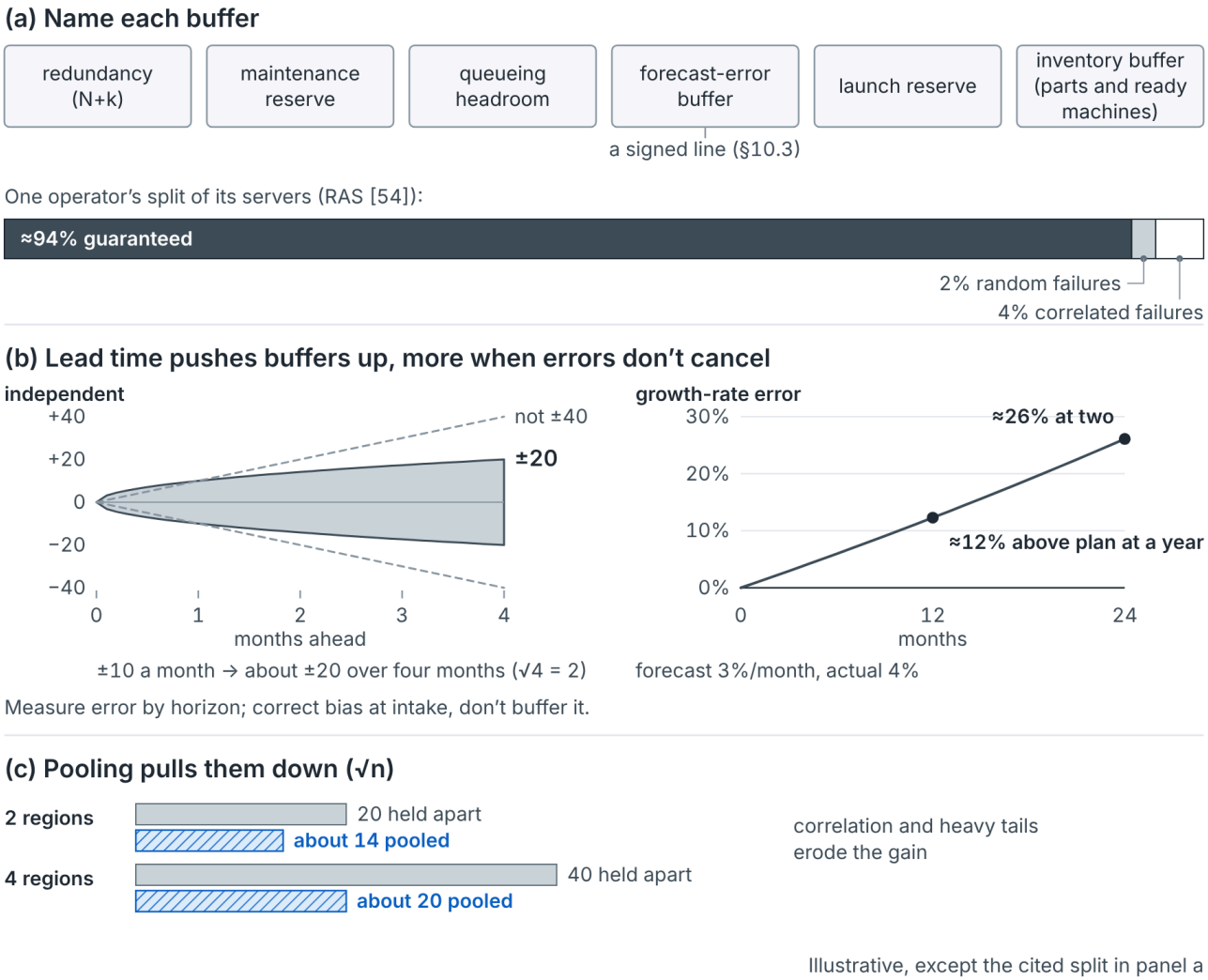


Figure 21. Name each buffer and size it on which mistake costs more. Lead time pushes buffers up, more than the textbook says when errors run the same way; pooling pulls them down. Illustrative, except the cited split [54].

Buffers need not sit idle. Borg reports about 20% of a median cell's workload running in reclaimed resources [17]. A backfill policy must say whether batch tenants pay for the buffer they borrow.

6.4 Bridges and clearing

Picture the 9 p.m. shortage email: approved ask, late machines, launch still on the calendar. When approved demand can't be sourced in time, you bridge. The levers differ in speed and premium (speeds are my judgment of orders of magnitude, not sourced figures):

Bridge	Time to capacity	Main cost
Reclaim idle or over-held capacity	Days to weeks	Effort; friction with the holder
Shift demand across regions or time	Hours to weeks for stateless serving; longer for stateful	Latency, data residency, migration
Reallocate between tenants	Weeks	Migration; churn

Bridge	Time to capacity	Main cost
Re-peg supply from a slipped launch to an on-time one	Days	Effort; the slipped launch re-confirms its date
Defer decommission of older hardware	Weeks, if not yet retired and if its space and power aren't needed by incoming hardware	Power, efficiency and reliability penalty
Third-party capacity	Months, with a standing agreement; longer without one	Price premium; capital becomes operating spend; usually a minimum term of its own
Facility redesign or retrofit	Quarters to years	Capital

Shifting demand is routine at scale: Flux reports global workload shifts every six weeks, each "reshuffling capacity for 100k-300k servers globally" [2]. In the SRE chapter's Meet case, capacity came partly from "borrowing machine resources from other teams across Google," a tenant-to-tenant bridge run under incident pressure [7]. Oversubscription is left out on purpose: it's better run as a standing policy, backed by peak prediction and power capping, than as an emergency lever [85–87]. The SRE chapter's pooled quota runs it that way [7]. Microsoft reports that predicting power peaks enabled "a 2× increase in oversubscription with minimum impact to critical workloads" [A3]. A later Borg study reports production compute usage at "only around 30% of the allocated level," with over-commitment as a standing bet [88].

A bridge has a lead time too. It works on your clock only if it was arranged before you needed it, so a standing third-party agreement is itself a slow-layer option. Power can be bridged too, with temporary on-site generation [60] (Appendix B).

Pegging makes the cheapest bridge possible. Supply planners link every order to the demand it was placed for; they call it *pegging*. It lets a late delivery be traced to the launches it hits, and a slipped launch to the deliveries it can hand to someone else. In my experience launches and deliveries both tend to run late, so the two can often cancel out, but only if the link exists.

Every bridge carries a premium, and the premium has a cause: a late ask, a forecast miss, an off-cycle launch. Charge the premium to whoever caused it. Say one team asks late, and the stopgap machines cost extra. If that extra is split across every team, the late team pays only a sliver of it. Everyone learns that asking late costs almost nothing, so more teams ask late. If the late team pays the full premium, its next ask tends to come on time. One case needs care: when a shared service caused it, say with a release that raised its own costs. That service recovers its costs through the rate every caller pays. So keep the premium out of the rate, as a cost the service carries itself. Otherwise every caller pays for one service's miss.

Clearing is the other half: one team holds capacity it doesn't need, another needs exactly that shape, there, now. Matching them is what Jevons called a "double coincidence" [89]: the baker who wants shoes must find a shoemaker who wants bread. Done by people **brokering** trades, it's a market with no price, and it scales poorly; the quota auctions and markets of Section 3 try to give it one [16, 18]. RAS automates part of it within one operator's region-wide server reservations [54]. The Quota Marketplace preprint does so for ML training accelerators, with teams bidding credits from an income that administrators "assign … to reflect their relative priority" [18]. The authors report average occupancy of 93% against 75% in a comparable conventional pool. That's the operator's observational comparison, not a controlled experiment, and occupancy is not goodput (useful progress; Section 11). Because the market can't see which accelerators share an interconnect domain, it produces preemptions "where quota exists but geometry does not" in about 1% of cases.

So markets automate clearing for fungible, short-horizon resources, and human work moves to weights, envelopes, shapes and long-lead supply, where the calendar, not just the shape, decides the outcome.

Human clearing runs on information. For example, if one team has spare cores but needs memory in three regions this quarter, someone has to find the other half. The trade clears only if someone already knows which team holds that memory there and needs the cores. Four things must match at once: the resource, the amount, the place and the time. That knowledge comes from a weekly habit of asking teams what's coming and what's idle, more than from any report. That's also why Section 10.4 says to log every trade: what the broker knows should outlive the broker.

6.5 Two clocks

In short: the slower a supply layer, the earlier you must order it and the less you know when you do. For persistent demand in a growing portfolio, this paper argues for aiming higher in the range.

Go back to the launch of Section 5.4: a rumor, then a date, then a configuration. The power for its building may have been ordered before anyone heard the rumor. Demand information and supply commitments run on different clocks. Alphabet's chief executive put the supply side in one line: "how we close the gap this year is a function of what we have done in the prior years" [90].

Each supply layer is committed at its own commit point: its need date minus its cautious lead time (Section 6.2). Hixson and Guliani advise planning at least as far ahead as the order time, and answer unresolved demand with fungibility [6]; this section works that out layer by layer. What matters for each layer is the **residual uncertainty at its commit point**. That's how much of the declaration is still open when the layer must be ordered, and how sure its arrival date is. Hixson and Guliani point at a closely related quantity. They tell finance to check "the sensitivity that those estimates have to their time horizon and how far into the future people are being asked to forecast" [6].

Figure 22 draws the two clocks as lanes. On top, the declaration's open range narrows at decision events (diamonds). Below, each supply layer's bar is its lead time, its dot the commit point, and the orange bracket above the dot the residual then. Land, power and shell commits before any declaration exists, with the whole range open. For simplicity every bar ends at launch; since each layer must be ready before the next starts, slow layers really commit earlier than drawn (Section 8 does it properly).

Put Section 7's launch on it. Machines and parts commit at −6 months, when the declaration is Dated: a base of 60,000 and a high of 90,000 requests per second. That leaves a 30,000 spread still open, and, as Section 7 shows, the launch could still land below the base. Network commits at −15, at Intent: 40,000 to 100,000, so 60,000 open. The further left the dot, the wider the bracket. The table generalizes Figure 22 and adds physical placement (commit points are illustrative and vary by operator and market):

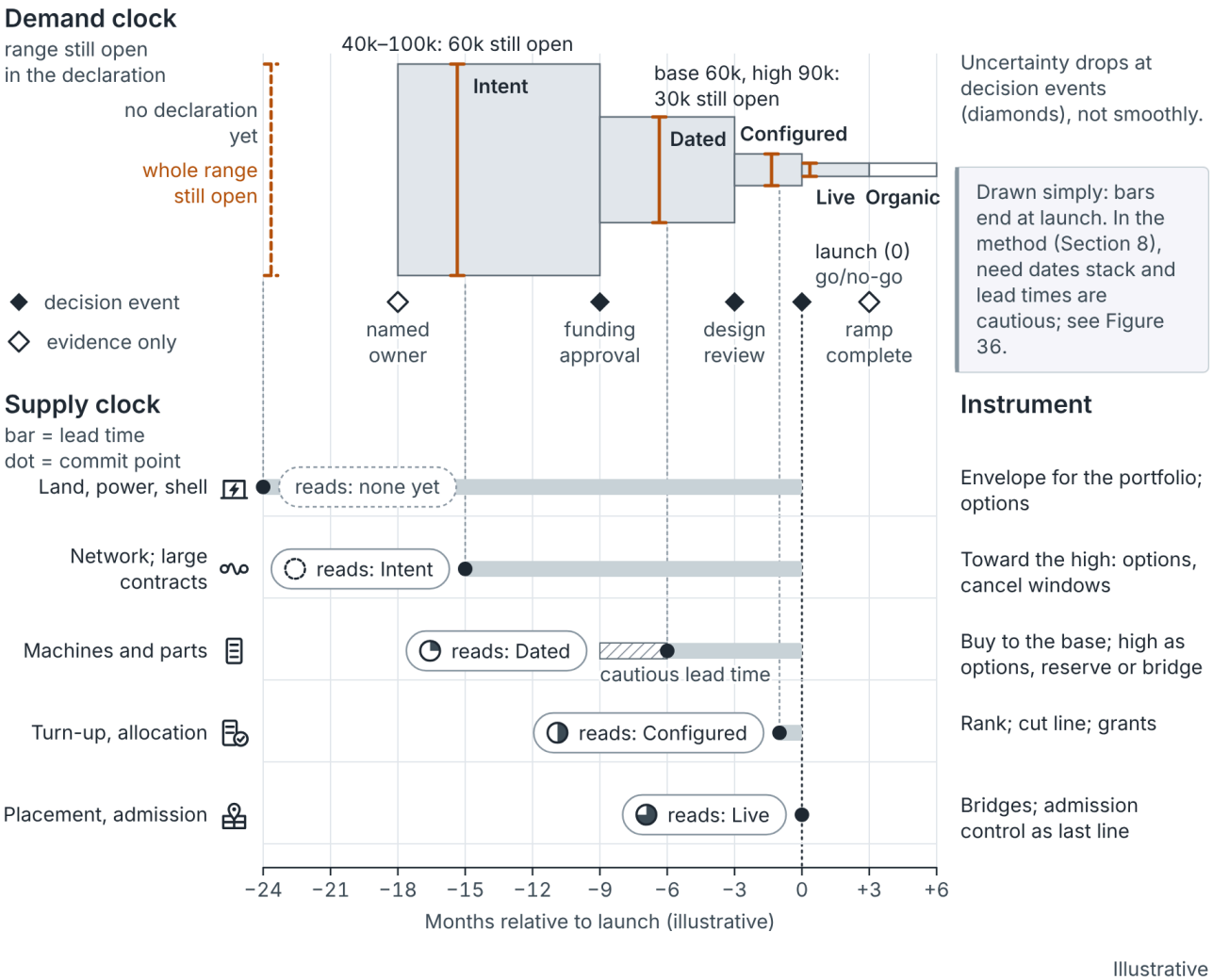


Figure 22. Demand firms up on one clock; supply commits on another. At each layer's commit point, the range still open in the declaration, not the point forecast, picks the instrument.

Supply layer	Typical commit point	Where the declaration usually is by then	Instrument
Land, power, building shell	One to several years ahead	Intent, or none	An envelope sized for the portfolio, not one launch; options; a fungible specification (Section 6.2)
Long-haul network; large supply contracts	Several quarters	Intent to dated	Buy toward the high, as options or with cancel windows
Machines and parts	One to three quarters	Dated	Buy to the base; hold the high as options, a launch reserve or a named bridge
Physical placement (site, hall, power domain)	With the machine order, or at turn-up	Dated to configured	A placement plan; spread across failure domains
Turn-up and allocation	Weeks per job; the rate is staffed months ahead	Configured	Rank against supply; move the cut line (the line on the ranked list below which asks go unfunded); issue grants; book turn-up slots

Supply layer	Typical commit point	Where the declaration usually is by then	Instrument
Job placement and admission	Minutes to days	Live	Bridges (shift, reclaim, defer decommission); admission control as the last line

Buy slow layers toward the high scenario. A shortage lasts as long as the short layer takes to fix or bridge: weeks for turn-up, quarters for machines, years for power when no third-party space is available. An excess costs the layer's carrying cost (Section 6.3) until growth absorbs it, which depends on the growth rate, not the lead time. And per unit of compute it enables, a slow layer is cheap to carry. Where power is paid for as drawn, an empty powered hall costs far less than a shortage of the machines it would hold. Take-or-pay power or a leased shell narrows that gap. So the cost of being short rises with lead time while, for shells and power, the cost of being over doesn't rise much. In this paper's argument (not a published result), the shortage-to-idle ratio therefore rises with lead time. Illustratively, for persistent demand in a growing portfolio:

Layer	Shortage : idle cost	Fraction	Buy to about
Turn-up	1 : 1	1/(1 + 1) = 0.50	The median
Machines	4 : 1	4/(4 + 1) = 0.80	The 80th percentile
Power and shell	19 : 1	19/(19 + 1) = 0.95	The 95th percentile

Turn-up is the exception: it's a rate you staff (Section 6.2), so its ratio sets how much rate to staff, not how much to buy. Figure 23 draws the table as three ranges.

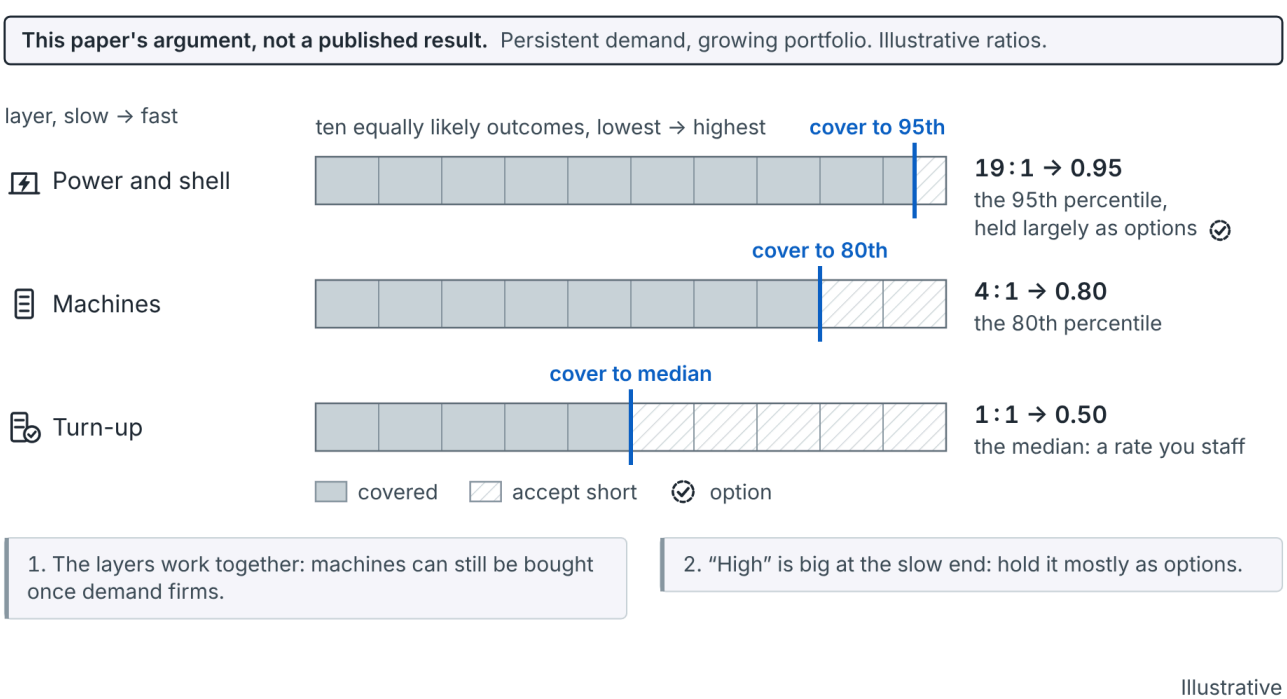


Figure 23. For lasting demand in a growing business, the paper argues that the cover point should rise with lead time: median for turn-up, about the 80th percentile for machines, about the 95th for power and shells. Illustrative, not a published result.

Two conditions keep that sound:

1. **The layers have to work together** (economists call them complements). A high quantile on power pays only if machines can still be bought once demand firms; that postponement lets the fast layers stay at the base. The "warm shells" remark of Section 6.2 is this condition failing the other way.
2. **"High" is a big amount at the slow end,** where the open range is widest. Hold it mostly as options (Section 6.2).

One operator publishes the slow end of this: it plans "to a high quantile demand forecast" because "the cost of empty data center space is much less than the shortage cost of not being able to deploy compute and storage when needed" [8]. Its per-component quantiles, chosen to meet on-time delivery targets [8], show the quantile already varying by resource. Meta's chief executive put the argument in operator terms: "aggressively front-load building capacity so that way we're prepared for the most optimistic cases"; if superintelligence takes longer to arrive, "we'll use the extra compute to accelerate our core business," and in the worst case "slow building new infrastructure for some period while we grow into what we build" [74]. That last clause is the carrying cost of excess, paid as a pause in building. Meta's finance chief said in July 2026: "we believe near term capacity is more valuable than long term capacity" [91].

The argument weakens if growth stalls, since excess then waits years, not quarters. It also weakens if leased or third-party space can bridge the slow layer, or if the launches behind the high scenario share a driver (Section 6.6). For a transient step, or a launch that may not happen, the cost of being over is the whole unit. There, the high belongs in options and bridges, not owned capacity.

Specify in lead-time order. The clocks can move toward each other: in Figure 22, supply can slide its dots right, and demand can narrow its range sooner. Supply does it with shorter lead times, which Hixson and Guliani count among the most valuable things a company can do to cut planning risk and cost [6]. The SRE chapter says the same: faster supply shrinks "the window of time for which you actually need to predict capacity" [7]. It also does it with contracts that allow cancelling or pushing a delivery out. Demand does it most cheaply through what the intake asks first. Size and date usually firm last. But what drives the slowest supply (region, redundancy tier, hardware class and power density, data stored) is, in my experience, often decided early and not asked. The chapter describes the supply-side twin: bringing spatial awareness of datacenter footprints into "the long-range planning process" early, so demand forecasts are physically achievable [7]. Ask for it at the dated stage, with size as a range. For the slowest layer even that is too late; only the portfolio envelope can serve it.

Place the gates between the events. A planning gate is most useful just after the demand event that resolves something and just before the supply commit point that needs it. In Figure 22, that's between a diamond and the next dot. When a commit point falls before its event, as Appendix A's high-memory order does, there are three choices:

1. Ask for the decision earlier.
2. Buy an option that carries the commitment past the event.
3. Commit to the stage's record and name the owner of the residual.

Write down which you chose: what was known, when, by whom, and what would have changed it. Figure 24 walks the choice for one layer, with Appendix A's case.

Bound how much the declaration may still move. Buying against the range open at a commit point works only if the declaration then stays inside it. Manufacturing's *time fences* handle this (the closer an order is to its date, the less it may change), as do quantity-flexibility contracts (Section 6.2). One way to apply it here is a *change band* per stage. For example, say an intent may move by half in either direction, a dated ask by a quarter and a configured ask by a tenth. Then

a change inside the band is the plan working. One outside it becomes a priced event recorded against the declaration, not a surprise (Section 8, step 7). Figure 25 shows where the gates go and how the bands narrow.

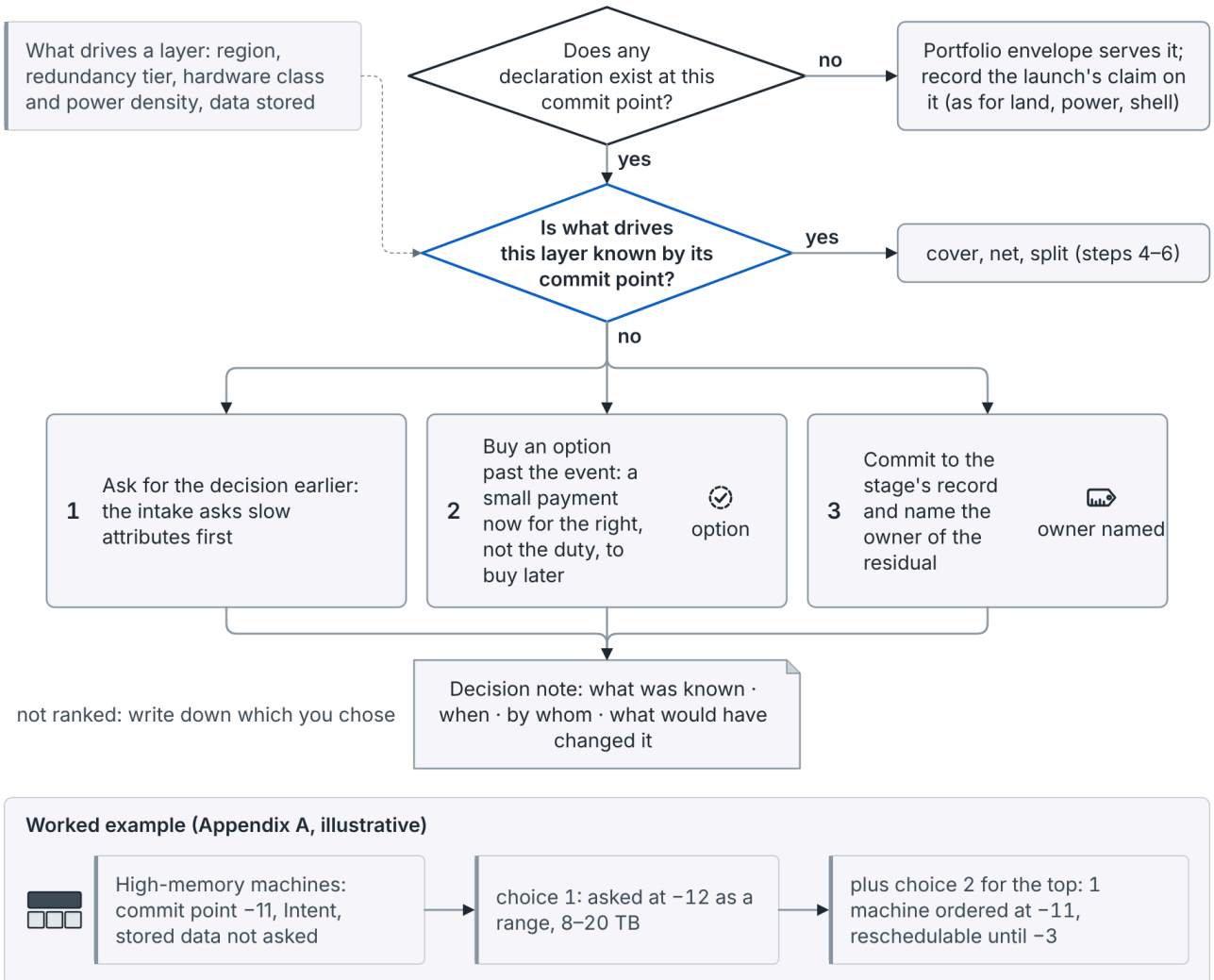


Figure 24. When an order is due before the event that would size it, ask earlier, buy an option past the event, or commit to the stage's record and name who owns the rest, and write down which. Before any declaration exists, only the portfolio envelope can serve the layer. Illustrative example from Appendix A.

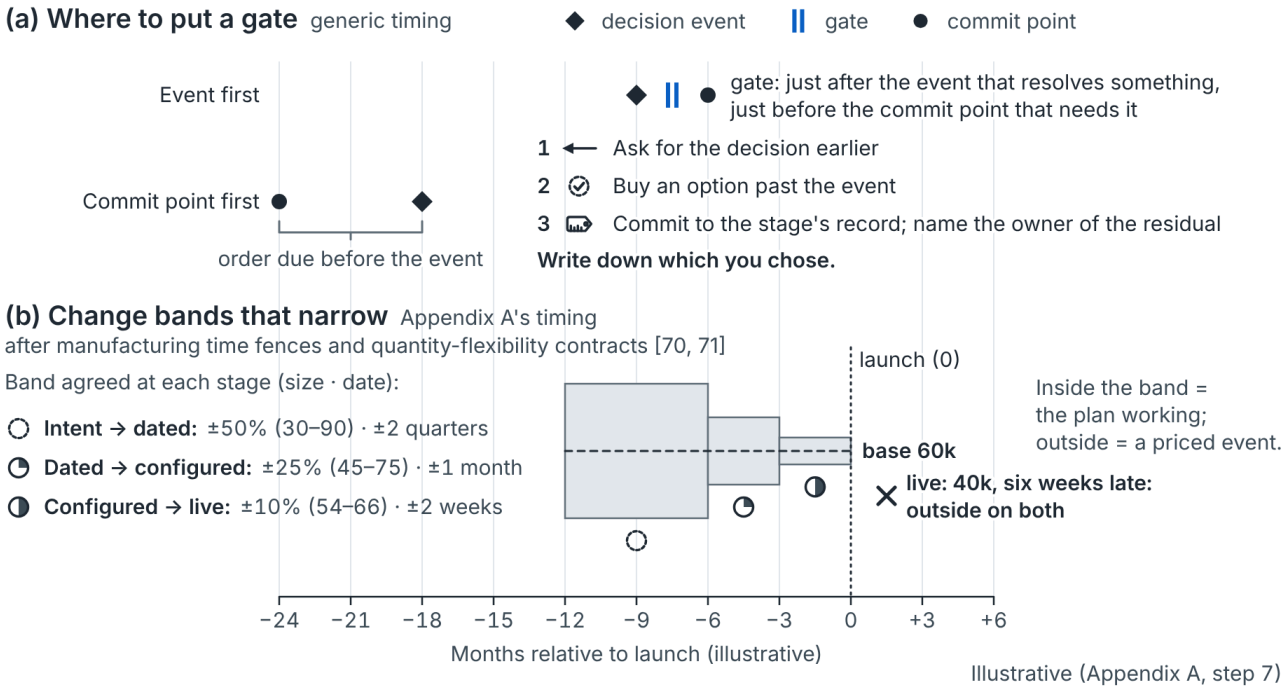


Figure 25. Put each gate between the event that answers a question and the order that needs the answer. Agree how far the declaration may still move at each stage. Illustrative bands.

6.6 Timing, size and persistence

In short: a launch can be wrong on size or on timing, can last days or years, and can share a driver with other launches. Each changes what to buy and who pays.

A size miss is paid for on what was bought for the size, mostly the slow and middle layers. Too high, and capacity waits for later demand; too low, and the gap needs bridges. A timing miss is, in my experience, most often a slip. It's paid for on everything committed for the launch, for the length of the slip, but shows mostly at the machine and turn-up layers. For example, if a building serves twenty launches, one arriving six weeks late hardly registers; the machines turned up for it do.

Section 5.2's variance decomposition shows neither error as what it is. A size miss shows as a smaller bill; a slip may not show at all, depending on how the consumer is billed:

	Billed on realized peak, held flat for the year (Section 7's rule)	**Billed on coincident-peak contribution, hour by hour**
Size miss	A smaller bill (volume term)	A smaller bill (volume term)
Slip	Invisible: the consumer pays for the slipped weeks as if it had launched on time	The bill drops for those weeks; their cost moves to the service's unused line

Capacity bought for demand that never arrives is billed to no one under either rule; it lands on the unused line. The service owner owns that line (mechanism 4), but the cost isn't booked to the declaration that caused it unless grants expire (Section 9.3). Each error needs its own owner.

Persistence is the second distinction. A *transient* step (a sale, a live broadcast, a launch-day spike) needs capacity for days; a *persistent* step (a new product, a new region), for years. Transient demand wants capacity that can be returned: shifted work, reclaimed and borrowed buffers, and third-party capacity. At the top of the peak, it wants admission control, the limit Alibaba accepts for an event it can predict but not plan for [33]. Persistent demand wants owned capacity. Many launches are both, a spike on a step. Size owned capacity on the spike and you buy permanent capacity for a temporary peak, so a declaration should carry the two shapes separately. Figure 26 shows what each kind of miss costs, and the two shapes.

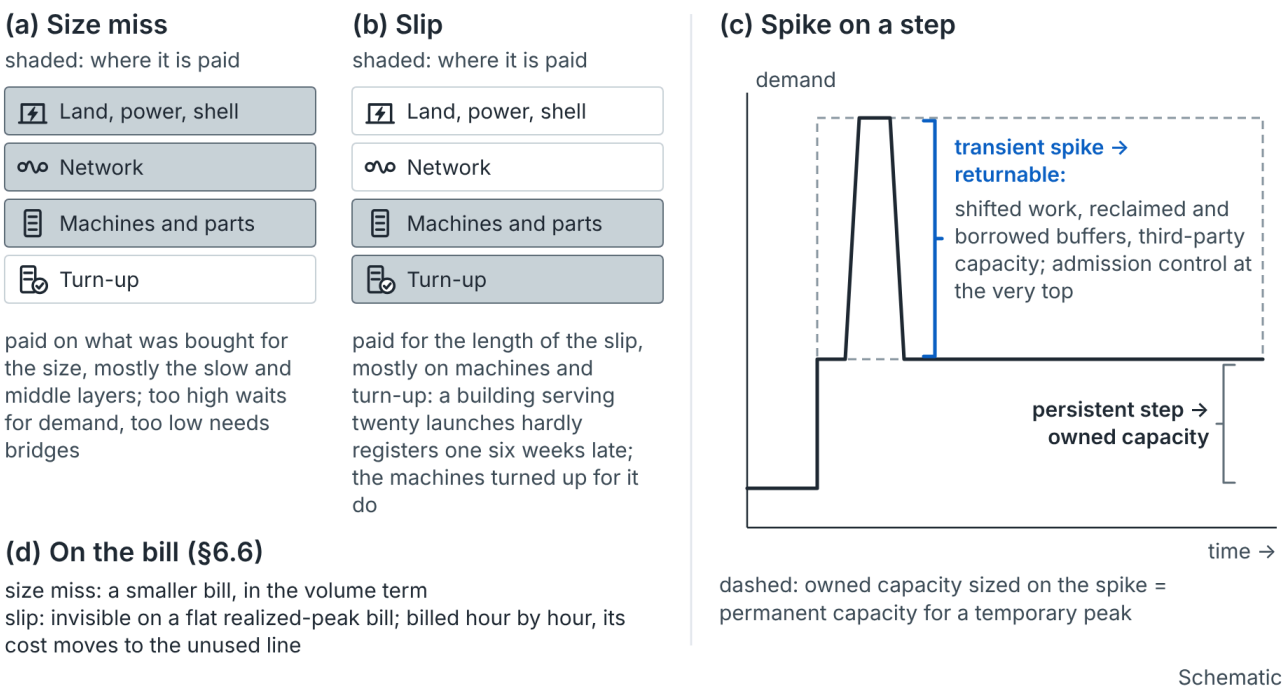


Figure 26. A size miss is paid on the slow and middle layers; a slip, on the machines and turn-up committed for the launch. A spike on a step needs two shapes: owned capacity for the step, returnable capacity for the spike. On a flat bill, a slip can be invisible. Schematic.

The third distinction is correlation. Hixson and Guliani suggest planning for only some of several launches to succeed, “a very specific way to trade increased risk for the organization against lowered cost,” and tell finance to check whether products “appear to be correlated with each other in terms of growth and cost” [6]. They also name what ties products together: technical coupling, and growth linked by brand, news or cross-promotion [6]. That check is what makes the trade safe, because planning for only some launches to land works like pooling (Section 6.3): independent bets average out. Illustratively, four independent launches each half likely to land will usually deliver about two; all four land only about one time in sixteen ($0.5^4 \approx 6\%$). Tie all four to one model release (or one strategy, partner or marketing date), and they land or slip together. Half the time all four land, and a plan for two is short by two. So name any shared driver in the register, and apply a portfolio haircut (planning for fewer successes than were declared) only within groups of bets that lack one. Name the driver; don’t infer independence from a low measured correlation. Risk managers warn that dependence goes “beyond simple linear correlation,” and the tails are where it shows [A4].

The same idea can size the launch reserve. Holding each launch’s high scenario on its own is, in the chapter’s terms, a sum of peaks [7]; the same pooling logic applies to launches. Hixson and Guliani already collapse several products’ upside into one plan, with someone in a position to resolve conflicts [6]. The chapter pairs centralized inventory with a correlation

check [7]. *One way to size it:* hold one reserve for a group of launches with no shared driver, sized to the group's high. Then give each launch a ranked claim on it (Section 8.3). Illustratively, four independent launches each have a 25,000 high slice, each half likely to be needed. Four separate slices make 100,000. At the 80th percentile, the group needs 75,000. Tie the four to one model release and they land together, so the group needs all 100,000. The chapter's own advice applies: first find which parts of a forecast "are correlated and which can be treated as independent" [7]. Figure 27 draws the two cases, and Figure 28 the reserve each one needs.

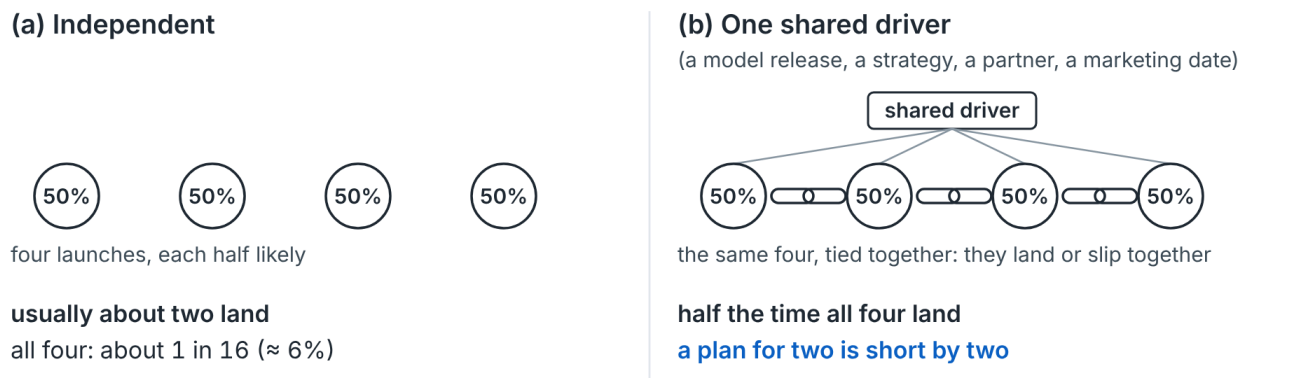


Figure 27. Independent bets average out; bets tied to one driver land or slip together. Hixson and Guliani's correlation check [6] is what makes planning for fewer successes safe. Illustrative.



Figure 28. One reserve for a group of independent launches, sized to the group's high, holds less than the sum of their high slices. Tie the launches to one driver and the saving disappears. Illustrative.

6.7 Supply has step changes too

The new site promised for one date now has another. Supply moves in steps too: a new site comes online, or a hardware generation changes shapes and coefficients (Section 5.3). A vendor rations a constrained part, a decommission retires a block of capacity, or a supply shock delays everything at once. No trend predicts these; someone must declare them. Auxon's authors list a slipped cluster delivery, alongside a service's efficiency regression, among the events that break a hand-built plan [1].

Supply declarations firm up in stages of their own: planned, contracted, under construction, delivered, live. Utilities and operators already report their power this way, with contracted load far ahead of what is under construction or active [68, 92] (Appendix B).

Supply can go live in phases, too. The SRE chapter describes phased delivery that brings basic compute, power and network online months early [7]. It replaces treating capacity as a binary "ready" state, and the early capacity carries a higher risk of stockouts [7]. Public examples exist elsewhere. One AI campus's first phase, "two buildings and 200+ megawatts," was "expected to be energized in the first half of 2025" [93]. Alberta's grid operator allocated an interim connection limit of 1,200 MW and moved other large-load requests to a second phase [94].

Here's where the two sides can meet; the parts are the sources', the field is my exploration. The chapter already sorts work this way, routing work "resilient to transient capacity shortfalls" to the early phases [7]. It calls the property *stockout resiliency*, and has clients tag their own traffic [7]. Hixson and Guliani ask each product what happens if growth exceeds planned capacity [6]. My exploration is to make it a number, with a duration, on every declaration. So it helps to ask each declaration one more question: how much shortfall, for how long, can it take? Call it *shortfall tolerance*. Demand that declares some can be pegged to a site that's only partly live, or hold an over-issued entitlement (Section 6.1). Demand that declares none waits for firm supply. And since a higher availability tier is a driver that raises the rate (Section 5.2), a declared tolerance should lower it. *My guess* is that this field, more than any forecast, decides how much early supply a fleet can use.

So score each supply declaration like a demand declaration, promised size and date against actual (Section 9.2, mechanism 11). A planner who discounts demand intents but takes supply promises at face value has half an outside view. The mirror holds too. A supply team that commits to a demand signal that changes every month inside its lead time has no plan to keep. And its own supplier scores it the same way (Section 6.2).

Slips travel. A supply slip opens the approval-allocation gap from the supply side: demand approved against a promised building or generation has nowhere to land. A late transformer delays the building, then every rack and turn-up already committed on the old date; the slipped batches then queue at turn-up together (Section 6.2). Nor does value arrive in proportion. A rack missing one cable isn't 99% of a rack. A training cluster with one block of its network missing may not run the job it was built for at all (Section 11). So fulfillment watches the scarcest small part as closely as the most expensive one. Figure 29 puts supply's stages beside demand's, and follows one slip.

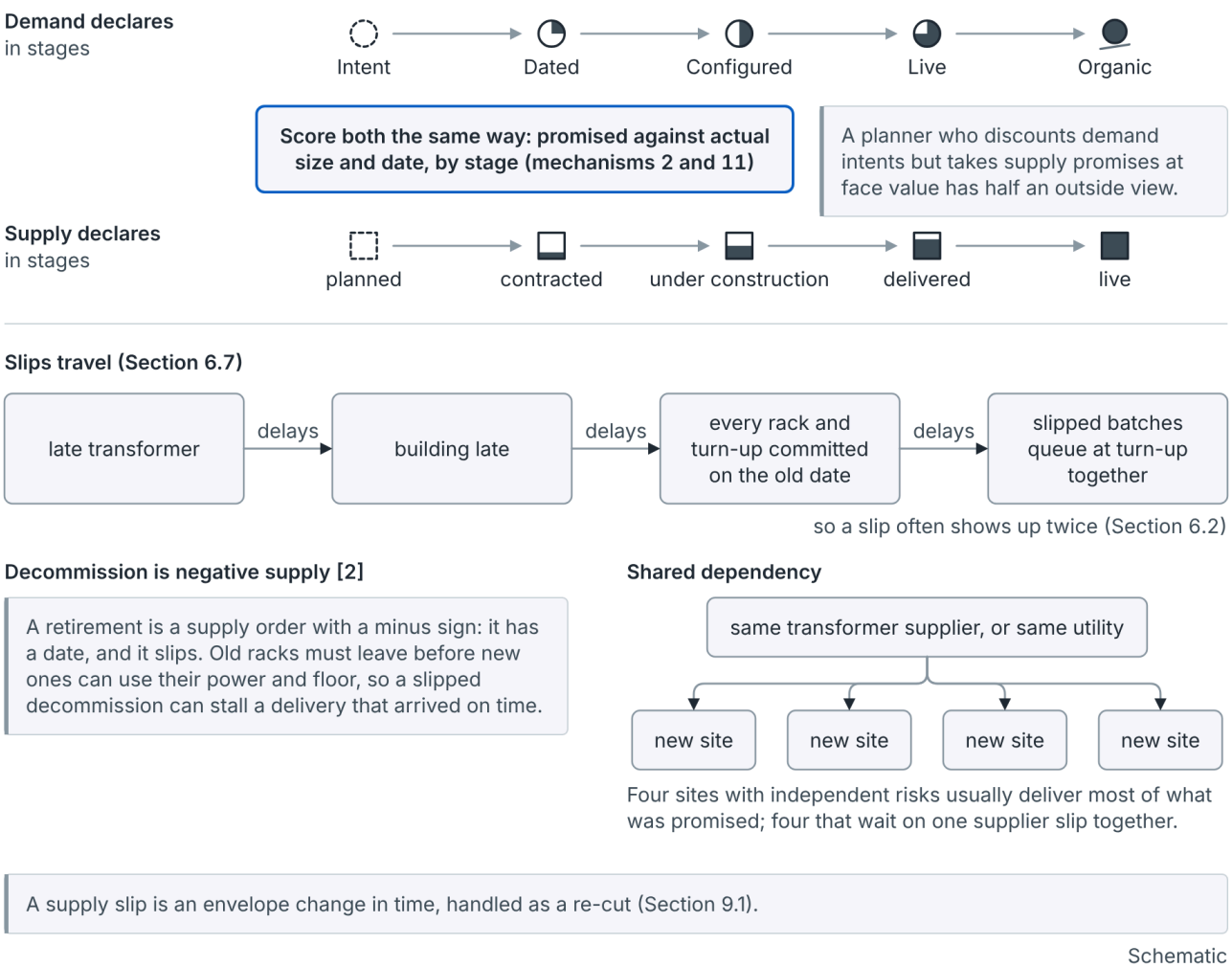


Figure 29. Supply declares in stages too, and is scored the same way. One late part travels down the chain and arrives at turn-up with everything else that slipped. Schematic.

Supply steps are usually lumpier than demand steps: one building's slip reaches every launch it serves. Tie a long-lead supply step to the portfolio, not one launch, and it stays fungible when any launch moves. A supply slip is an envelope change in time, handled as a re-cut (Section 9.1); a shock across many layers is the same re-cut, larger.

Decommission is negative supply. Flux records large-scale decommissioning of servers in three regions, "resulting in a reduction of supply in those regions," and manages "projected regional capacity deficits such as those caused by regional hardware refresh" [2]. A retirement is a supply order with a minus sign. It has a date, and it slips, usually because tenants can't migrate off in time. And old racks in an existing building usually have to leave before new ones can use their power and floor. So a slipped decommission can stall a delivery that arrived on time. In my experience, the fastest bridge for a late new site is often an older one. Refreshing it or deferring its decommission gives worse hardware, sooner, where it isn't in the new hardware's way.

Check supply for shared drivers. The correlation check of Section 6.6 works for supply too. Four new sites with independent risks will usually deliver most of what was promised; if all four wait on the same transformer supplier or the same utility, they slip together. So the supply register names shared dependencies, and the plan counts on supply averaging

out only where it really is independent. *My guess*, and it is only a guess, is that in a tight market lead times stretch together across vendors, because every queue fills at once. If so, supply errors average out least exactly when it matters most, and the cautious lead time of Section 8 should be more cautious then.

7. A worked example (all numbers illustrative)

In short: one service, one caller, invented numbers. The forecast lands within 4%, the bill lands 30% over, and each piece of the miss goes to whoever moved it.

Sections 5 and 6 took the pieces one at a time. Here they meet in one case, the year-end review of Section 1.

The setup. Service X is a shared document store whose calls run server-side queries; Product A is its largest caller. The numbers are invented, and machine counts and dollars are kept small so the arithmetic is easy to follow.

The common unit is Section 6.1's invented *normalized machine unit* (NMU), weighted by component cost, 80% for cores and 20% for memory. A standard machine (50 cores, 200 GB) is 1 NMU, so a high-memory machine with 600 GB is 0.8 + 0.2 × 3 = **1.4 NMU**. A core is 0.016 NMU (0.8 ÷ 50 cores) and a GB 0.001 (0.2 ÷ 200 GB).

The rate card. X prices only CPU: $0.12 per busy core-hour, the pool's cost over practical capacity, here half of installed cores.

Why installed is twice busy. Two buffers with different owners multiply. Surviving the loss of one of three failure domains takes 1.5 times peak, owned by reliability engineering. Holding latency means CPU at most 75% busy, a further 4/3 of queueing headroom, owned by X. If the latency target must hold even with a domain down, the two multiply: 1.5 × 4/3 = 2.0 installed cores for every busy one. The 4/3 is a flat rule, kept simple for the example; queueing headroom shrinks as a share of a larger pool (Section 6.3) [84]. Many teams accept degraded latency during a failure and run hotter, which needs less.

How A is billed. On its realized peak, held flat for the year: a simplification of charging its coincident-peak contribution hour by hour. X's release and the power re-pricing below take effect from the start of the year.

Declare. A declares next year's peak at 500,000 requests per second, up from 400,000. That's 40,000 of organic growth on A's trend, and 60,000 from a feature launch. The launch is a step change with no history, planned for the start of the second quarter.

The launch reached the forum in stages (Section 5.4; stage timing varies by launch, and Figure 22's is generic):

- **At the annual cycle**, as an intent of 40,000 to 100,000.
- **Dated, two quarters before launch**, with a base of 60,000 and a high of 90,000.
- **Configured, one quarter before launch**, with two regions and the data it would store.

The planner entered the intent at 60,000 (illustrative), where comparable intents had actually landed, and the dated base later confirmed it. So A's declared demand is the organic forecast plus 60,000.

X's coefficient is 0.4 calls per request × 2.5 ms per call = 1.0 ms per request, so busy cores rise from 400 to 500. A's plan is 500 × 8,760 h × $0.12 = **$525,600**, up from $420,480.

Approve. 100 busy cores × 2.0 = 200 installed cores. The intake asks only about CPU, so the forum approves four standard machines, **4 NMU**. The high scenario would add 30 busy cores, 60 installed; following Section 6.5, the forum

names a bridge for them rather than buying them.

Fulfill. Here's the catch: X's memory grows with stored data, not requests. A's data in X will grow by 16 TB, most of it from the launch. That figure is sized at configuration on the dated base volume, and re-sized once actual data is known. At an illustrative 100 GB of memory per TB stored (including failure reserve), 16 TB needs 1.6 TB of memory. Four standard machines supply 0.8 TB.

- **Naively**, the ask needs eight standard machines, **8 NMU**, with half their CPU stranded.
- **Re-shaped**, two standard and two high-memory machines supply exactly 200 cores and 1.6 TB for **4.8 NMU** (1 + 1 + 1.4 + 1.4). That's what the ask would have cost (200 × 0.016 + 1,600 × 0.001) had the intake asked about memory.

The fulfillment ledger (Section 9.2, mechanism 5) records 4.0 approved, 4.8 as the cheapest feasible allocation, and a **0.8 NMU gap**. The gap's named owner can fund it, phase the launch or ask X to shrink its working set. A higher memory weight, as when memory prices run high, would only widen the gap. Figure 30 counts the machines.

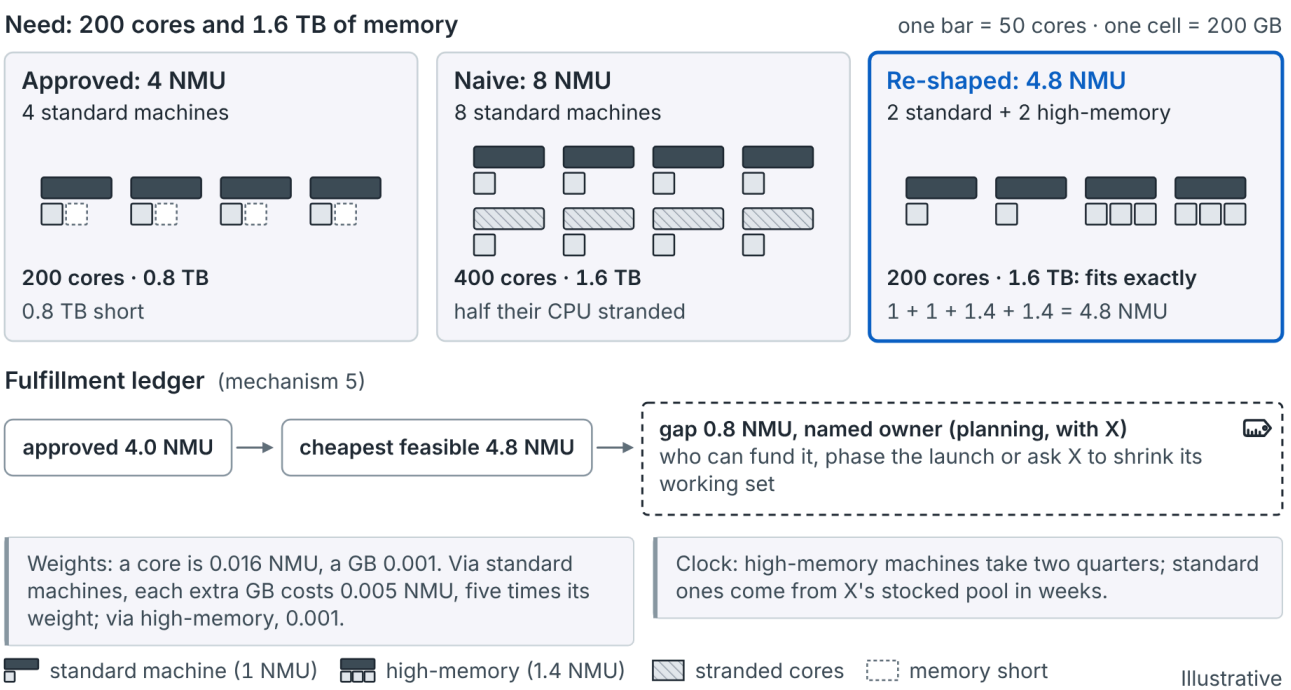


Figure 30. The forum approved 4 NMU because the intake asked only about CPU. The data A stores needs memory, and the cheapest fit is 4.8 NMU. The 0.8 gap needs a named owner. Illustrative.

Then the clocks bite. In this example, high-memory machines take two quarters to arrive; standard machines come from X's stocked pool in weeks. In months from launch:

- **Month −6:** the high-memory commit point. The launch was only dated, and the intake asked nothing about stored data.
- **Month −3:** the memory need surfaced at configuration, and the machines were ordered.
- **Month 0:** launch.
- **Month +3:** the machines arrive, a quarter after launch.

In Figure 22's terms, the memory question arrived after the machines' dot: the two-clock failure of Section 6.5.

Until the machines land, the ledger carries a dated bridge. It can be four more standard machines from the stocked pool, run for their memory with their CPU stranded (the naive fulfillment, as a bridge). Or it can be older high-memory machines kept past decommission, at a power and efficiency premium. A pool draw also leaves a replenishment owed, with a date and an owner, or the next team needing a standard machine finds four fewer.

The rate card sees none of this: A's data drives the memory, and X charges only for CPU. Without a second driver, the 0.8 NMU reappears next cycle as an unexplained rise in the CPU rate, paid by every caller for growth one caller caused.

Actuals. A's volume lands at 480,000 requests per second, within 4%. Two things A didn't control moved:

- A release of X raised CPU per call to 3.125 ms (α = 1.25 ms per request).
- X's rate rose to $0.13 after a sharp re-pricing of its power contract.

A's cost is 600 busy cores × 8,760 × $0.13 = **$683,280**, 30% over plan. Split by term:

Term	Calculation	Effect	Owner
Volume	(480k − 500k) × 1.0 ms = −20 cores × 8,760 × $0.12	−$21,024	Product A
Coefficient	480k × 0.25 ms = +120 cores × 8,760 × $0.12	+$126,144	Service X (its release)
Rate	600 cores × 8,760 × $0.01	+$52,560	Service X, with finance
Total		**+$157,680**	

Figure 31 draws the same split as a waterfall.

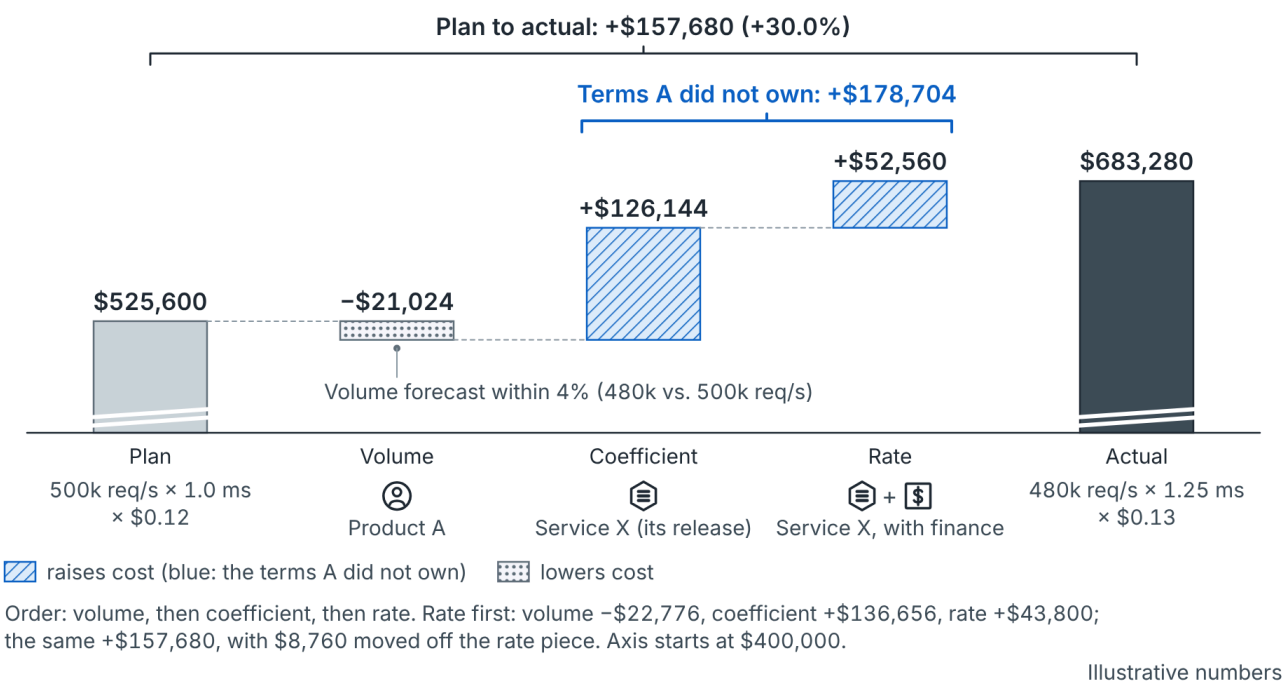


Figure 31. A forecast within 4% and a bill 30% over plan. Split the variance by term and each piece gets an owner; the two terms A did not own add up to more than the whole overrun.

Read the table as three swaps, one term at a time: plan $525,600; actual volume, $504,576; actual coefficient, $630,720; actual rate, $683,280. The order matters a little (Section 5.2). Put the rate first and its piece is 500 × 8,760 × $0.01 = $43,800, volume −$22,776 and coefficient +$136,656: the same +$157,680, with $8,760 moved off the rate piece.

Supply missed too: 600 busy cores need 1,200 installed against 1,000 planned. The extra 200 installed cores (four standard machines, 4 NMU) came from bridges. Until they landed, X ran inside its own buffers, below its planned redundancy and latency targets: a dated ledger item owned by X.

Size and timing. The 20,000 shortfall was all the launch's; organic growth landed on trend. The launch delivered 40,000, two-thirds of its base, six weeks late. As Section 6.6's table predicts for this billing rule, the size miss shows only as A's smaller bill in the volume term, and the slip shows in no term.

This year the slip cost A nothing it could see, and even eased X's shortage. At the drifted coefficient, organic demand alone (440,000 requests per second × 1.25 ms = 550 busy cores, 1,100 installed) needed more than the 1,000 planned. Two errors that offset each other are luck, not a plan working.

Now suppose the coefficient had held, so the launch's 60 busy cores, 120 installed, were really there, waiting. An installed core costs the pool $0.06 an hour (the $0.12 busy-core rate spread over twice as many installed cores). Six weeks is 1,008 hours, so about $7,258 of launch capacity sits idle, split two ways:

Launch capacity	Installed cores	Idle cost	Who sees it
Used, but six weeks late	80 (two-thirds)	About $4,838 for the six weeks	No one: A pays it inside its flat bill, as above
Never used	40 (one-third)	About $2,419 for the six weeks, and it runs all year	X's unused line
Total, first six weeks	120	About $7,258	

The never-used third doesn't stop at six weeks. The 40 installed cores built for the 20,000 that never arrived are billed to no one. As 20 busy-core equivalents of practical capacity, they put 20 × 8,760 h × $0.12 = $21,024 on X's unused line. That's A's favorable volume term, seen from X's side. X owns the line, but nothing books it to A's launch, which caused it (Section 6.6).

The review. A's leader says, reasonably, that the team hit its number and is billed for someone else's miss. The decomposition makes that conversation short and leaves four questions.

Should X have versioned the change when it shipped? Yes. Illustratively, with both of Section 5.2's alerts live, the coefficient miss would have reached X as a ticket long before the bill.

Does A pay the coefficient term? On this year's bill, yes: A is billed its busy cores at X's rate, so the $126,144 reaches A unless a rule moves it. The decomposition assigns accountability, not payment. In my judgment, this case needs a rule agreed with finance before any miss. Under it, the forum scores the term against X's efficiency commitment (Section 9.2, mechanism 7). Then it decides whether a central reserve holds A's budget harmless for the year. Without one, the pieces have owners in the meeting but not on the bill.

Who carries the bridge premiums? X's release caused the CPU bridges, so X does. Since X recovers its costs through its rate, the forum holds the premium outside the rate (Section 6.4). It sits as an unrecovered variance on X's own line, reported against its efficiency commitment. The memory bridge and this year's 0.8 NMU trace to a gap in the intake

design, owned by planning with X. No published driver attributes them to a consumer, so they go to the unallocated line, with both named as owners.

Who pays for the memory itself? The hardest one. A's data caused it, so by this paper's principle A should pay. But no published driver told A so, and the forum rules that a new driver applies from its effective date, not retroactively.

Without the decomposition, the meeting is an argument about fault, and A pads next year's declaration. Figure 32 draws the review as decisions.

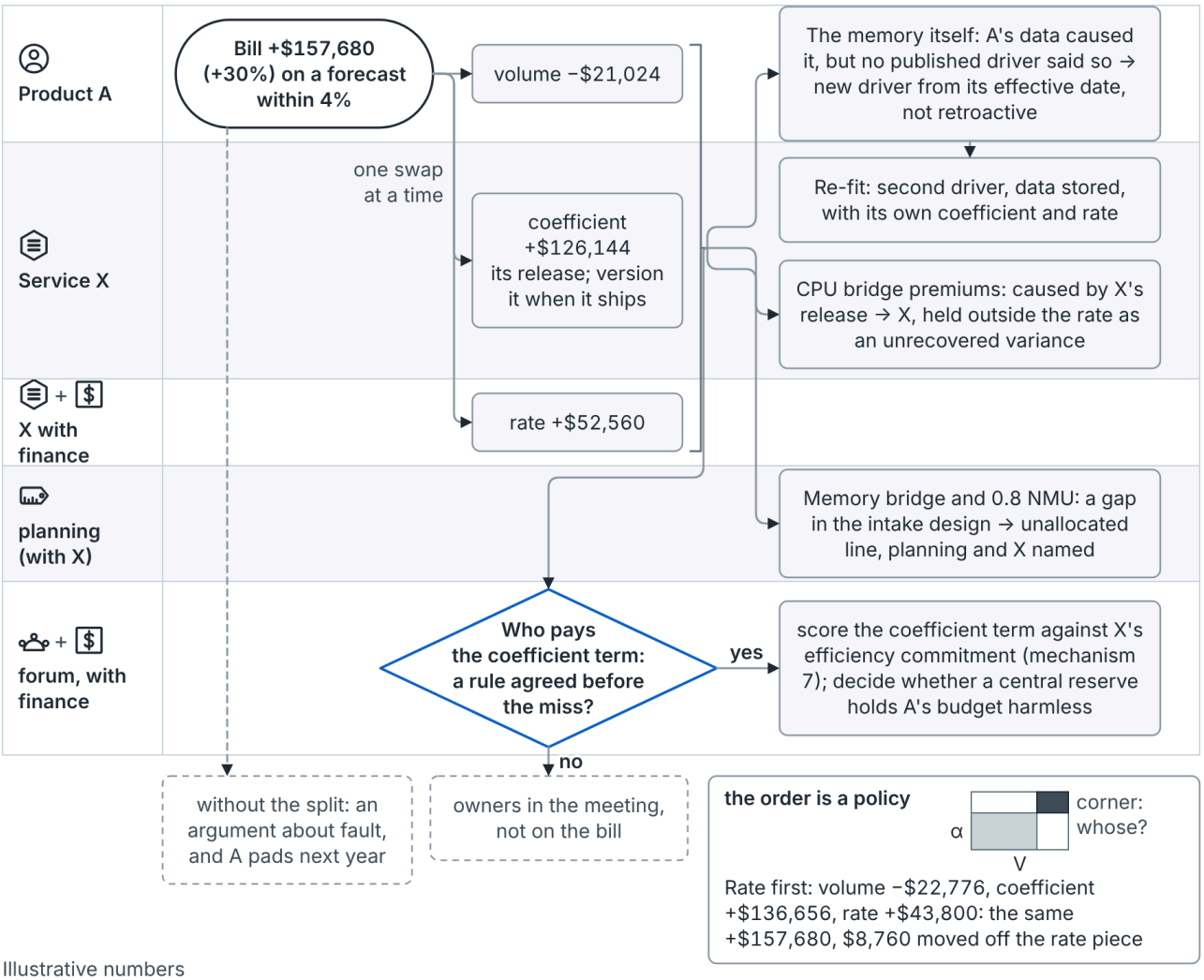


Figure 32. The split gives each piece an owner, and the forum rules on the bridges, the memory and the re-fit either way. Who pays the coefficient term needs a rule agreed in advance; without one, that piece has an owner in the meeting but not on the bill. Illustrative numbers.

Re-fit. X adds a second driver, data stored, with its own coefficient (memory per GB stored) and rate, logged with an effective date. Its intake now asks for stored data as a range at intent and a figure once dated. That's because memory is X's slowest dimension to source, and the dated stage falls right at its commit point. It also shows A's 500k ask beside its 480k actual.

A's launch joins the record by stage: intent 40,000–100,000, dated base 60,000 and high 90,000, actual 40,000, six weeks late. It landed below the 60,000 where comparable intents had, and will move the next estimate.

The three prices now give three answers:

- The **average rates** for CPU and memory set A's budget.
- The **long-run incremental cost** of A's growth is memory-driven: 4.8 NMU rather than 4.0, a 20% premium that belongs on the memory driver.
- The **shadow price** falls on whichever dimension binds. Under the naive eight-machine fulfillment, that's memory, which is why re-shaping saves 3.2 NMU. Once the coefficient drifted, 600 busy cores needed 1,200 installed against 1,000 planned, so CPU ran out first, and that's what the bridges were buying.

Every piece now sits on an artifact with an owner and a date. What the example doesn't show is how to catch the memory question in time. Section 8 does.

8. The commit-point method

In short: one launch, every supply layer, eight steps on one page. For each layer, ask when it must be ordered in a slow case, and how firm the launch will be by then. Then ask what to commit, in what form, with whom owning the rest.

Section 7 shows what goes wrong, not the walk that would have prevented it. A planner who agrees with Sections 5 and 6 still faces a blank page on Monday: which layer first, which number where, who signs what? This section offers that walk. It puts the earlier pieces in order: stages (Section 5.4), supply layers and contracts (Section 6.2), and bridges and pegging (Section 6.4). Then come commit points and change bands (Section 6.5), and the ways a step can miss on either side (Sections 6.6 and 6.7). It's a way to think a launch through, not a model; Appendix A has the one-page worksheet and the full walk-through.

8.1 The idea in one paragraph

Every supply layer must be ordered by some date, and by then the launch has only firmed up so far. So for each layer ask: when must this be ordered, if the order is slow? How firm will the launch be by then? Given that, what do we commit, in what form, and who owns the rest? Then net against what you have, agree how much the launch may still change at each stage, and write it down where supply can answer it. Figure 33 shows the walk and what it writes down.

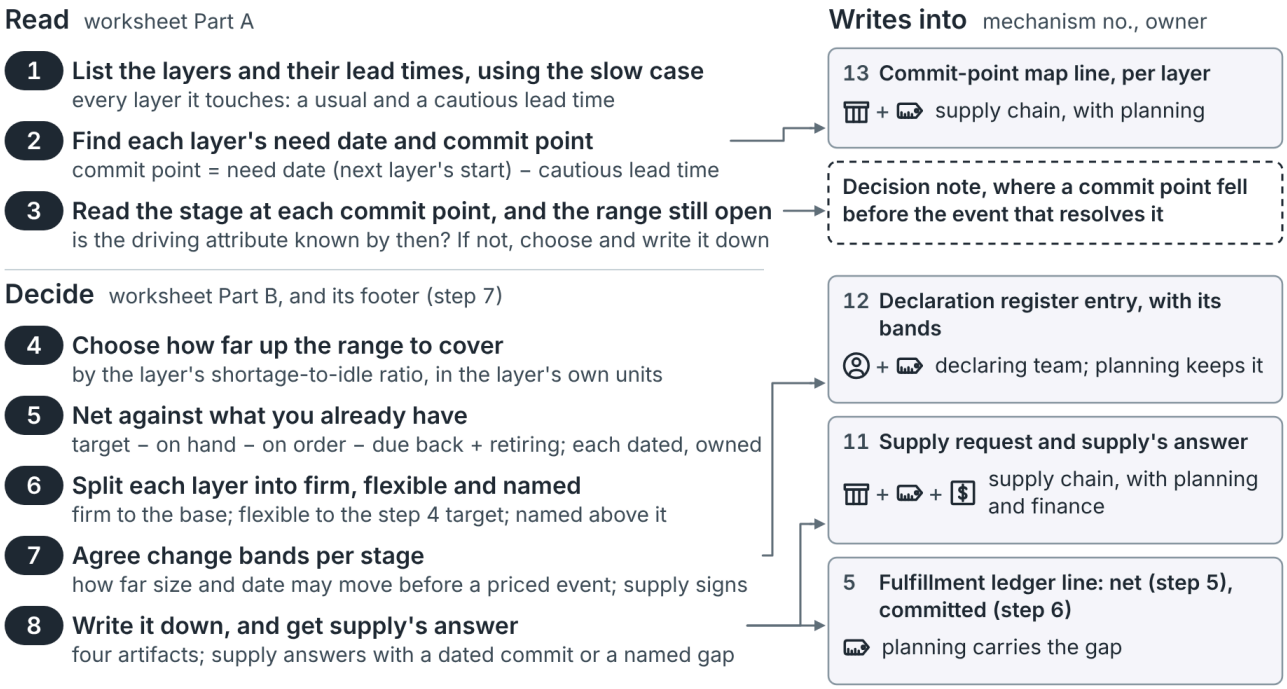


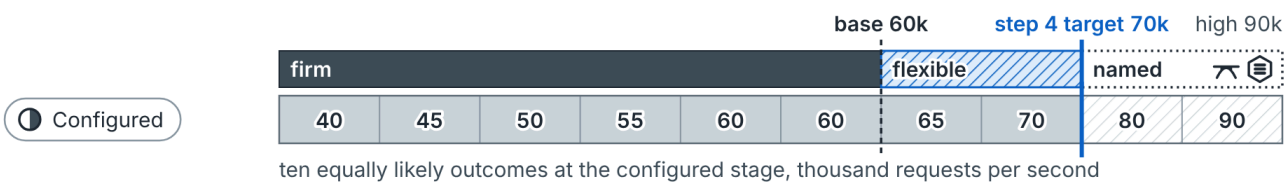
Figure 33. The walk ends in artifacts, not a number. Each step writes into a register, a map, a ledger or supply's answer, so the decision can be checked and defended later.

8.2 The eight steps

1. **List the layers and their lead times, using the slow case.** Take every layer the declaration touches: power and space, network, each machine shape, parts, turn-up, placement. Write each one's usual lead time and a cautious one, near the slow end of what you've seen (Section 6.2). A public value (Appendix B) is a sanity check, not your number. *Why:* in my experience (Section 6.2), deliveries are rarely early and sometimes very late.
2. **Find each layer's need date and commit point.** Work back from launch. The ready-for-service date sits ahead of launch by the testing the launch needs. Each layer's need date is the start of the layer after it; its commit point is that need date minus the cautious lead time. *Why:* layers stack, so a slow layer's real deadline is earlier than launch day suggests.
3. **Read the stage at each commit point, and the range still open.** Lay the declaration's stages (Section 5.4) on the same timeline. At each commit point, note the stage reached and the range still open, from the stage's record, not only the team's word. Then ask: is the attribute that drives this layer known by then? Memory drives a high-memory order; region drives power and network. If not, pick one of Section 6.5's three choices and write it down. Where the commit point comes before any declaration exists, as power and shell often do, the portfolio envelope serves that layer; record the launch's claim on it. *Why:* this question would have caught Section 7's memory problem in time.
4. **Choose how far up the range to cover.** Use the shortage-to-idle ratio agreed for this layer and kind of demand (Section 10.2), with supply's idle cost and liability terms. That ratio is the cost of running short against the cost of holding spare (Section 6.3). Failing that, start from Section 6.5's illustrative table (for turn-up, the ratio sets how much rate to staff). Convert the point into the layer's units: positions, machines of each shape, turn-up slots. *Why:* shortage and spare cost differently on each layer, so one level for all is wrong somewhere.
5. **Net against what you already have.** The net need is the target, minus what's on hand, on order or due back from reclaim, plus capacity due to retire that current load depends on. Each of those is a forecast too, so give each a date

and an owner. A deferred retirement can bridge only if its hardware isn't in the space or power the new hardware needs, and a draw on a stocked pool leaves a replenishment owed. *Why:* each number subtracted is a forecast too, and a reclaim that doesn't happen looks like a supply failure (Section 6).

6. **Split each layer into firm, flexible and named.** *Firm* covers up to the base, as a plain order. *Flexible* covers from the base up to the step 4 target, as options, cancelable or reschedulable orders, or a named launch reserve. *Named* covers the rest, as a bridge with an owner and a ready-by date and, at the very top, admission control. Figure 34 draws the split. Since the most common timing miss is a slip (Section 6.6), prefer the right to push a delivery out over the right to cancel it. *Why:* each slice gets the cheapest form that covers it, and none is left without an owner.
7. **Agree change bands per stage.** With the declaring team, agree how far size and date may move at each stage before the move becomes a priced event (Section 6.5). Supply signs the same bands, because they're its buying assumptions. Bands tighter than the stage's record supports make every launch an exception. *Why:* a band says when the declaration has left the range supply bought against.
8. **Write it down, and get supply's answer.** Record the result in four artifacts. The first two are the declaration register entry, with its bands (mechanism 12), and one commit-point map line per layer (mechanism 13). The others are the supply request and supply's answer (mechanism 11), and the fulfillment ledger line, with net and committed columns (mechanism 5). Where a commit point fell before the event that resolves it, add a decision note: what was known, when, by whom, and what would have changed it. *Why:* an approval is firm only once supply has answered it, and a written decision can be defended later.



firm
up to the base (60k): plain order

flexible
base → step 4 target (70k): options, cancelable or reschedulable orders, a named launch reserve

named
above 70k up to 90k: a bridge with an owner and a ready-by date; admission control at the very top

Worked row (Appendix A, step 6): standard machines

2 from the pool at −3 (the base)	1 from the pool as a named launch reserve, pegged to A (base 60 → 70)	a bridge ready by launch, owned by X: 2 more standard run for their memory, or 1 older high-memory kept past decommission

Worked row (Appendix A, step 6): high-memory machines (sized at intent, step 4)

1 ordered at −11, plus the reclaim	1 ordered at −11, reschedulable until −3	—

> Prefer the right to push a delivery out over the right to cancel.

Illustrative (Appendix A)

Figure 34. Step 6 of the method: buy the firm part, hold the next slice as options or a reserve, and name a bridge and an owner for the rest. Illustrative numbers from Appendix A.

8.3 What each side owes the other

This is where "approval is not allocation," the second claim of Section 1, becomes an exchange with two halves.

Planning owes supply six things, per layer and before its commit point. The first three are the net requirement by shape, site and need date; the stage and open range, not just a point; and the signed change band. Next are the substitutes the team will accept (other shapes, other sites, a smaller first phase, a later date). Last come the ramp, not just the peak, and the launch's rank under shortage.

Supply answers each line with a *dated commit* (quantity, date, confidence, any substituted shape) or a *named gap* (how much, by when, why, and a proposed bridge with its owner). Until then, the approval is uncommitted (Section 6.1). Pegging each order to its declaration (Section 6.4) lets a late delivery be traced to the launches it hits, and a slipped launch hand its supply to someone else.

Both sides get scored: supply on promised against actual, demand on how much it changed inside supply's lead time. Figure 35 follows one ask across the four parties.

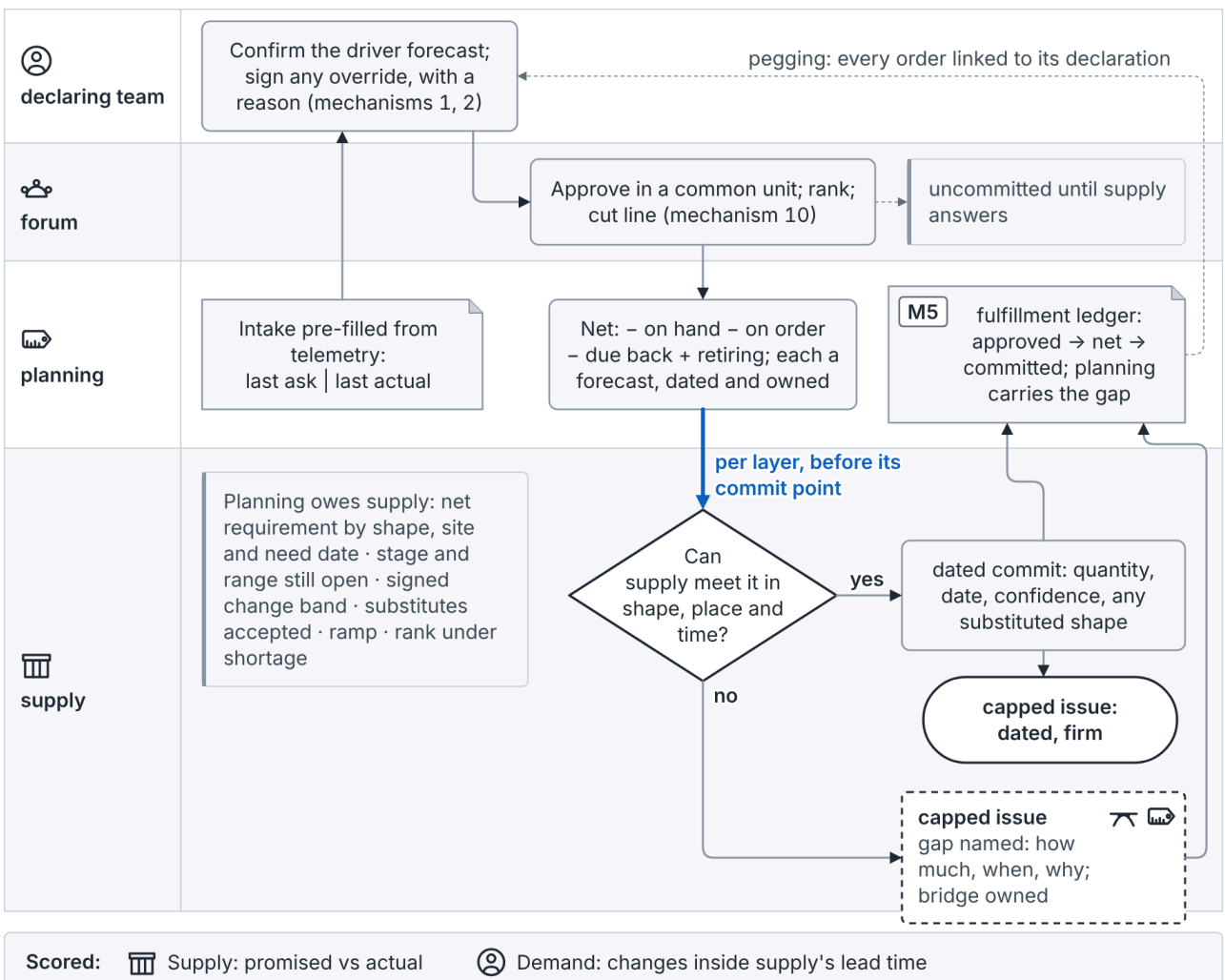


Figure 35. An approval prices an ask; supply's answer makes it firm. Planning sends a dated, ranged, signed request per layer before its commit point, and supply answers each line with a dated commit or a named gap; either answer makes it capped issue.

8.4 The same launch, walked through

Appendix A walks Section 7's launch through all eight steps, with one added assumption that makes memory scale with the launch's size. A cautious lead time and a real need date move the high-memory commit point to −11, while the launch is still an intent. That's where step 3 catches the missing stored-data question. Who pays for the landing still follows the rule agreed with finance. The table compares the two:

	Section 7 as written	With the method (illustrative)
High-memory commit point	−6, at dated	−11, at intent
Stored data asked	At −3 (configured)	At −12 (intent), as a range
High-memory machines arrive	+3, a quarter after launch	By −2
Bridge during launch	Four standard machines run for memory, or old machines past decommission	None needed at the base; named in advance above 70,000
Size miss	Lands on X's unused line, booked to no declaration	Same cost, recorded against A's declaration as a band breach
Slip	Invisible	Recorded; the launch reserve re-pegged to another tenant
Cost of the approach	—	About 1.0 NMU held above the base, and a nine-month view of a range still open

The method doesn't make the launch more accurate. It moves the hard question to the date it was needed, and it gives every surprise an owner. Its cost shows, too. At 40,000, A's growth needs 160 installed cores against 250 held (five machines, sized to the configured 80th percentile of 70,000). So 90 sit idle, about $47,304 a year. Section 7's plan would have left 40 idle ($21,024), so the method's extra machine accounts for about $26,280. That's the plan working inside its 4:1 ratio for persistent demand, and the band records which declaration the spare waited on. Figure 36 draws the two timelines.

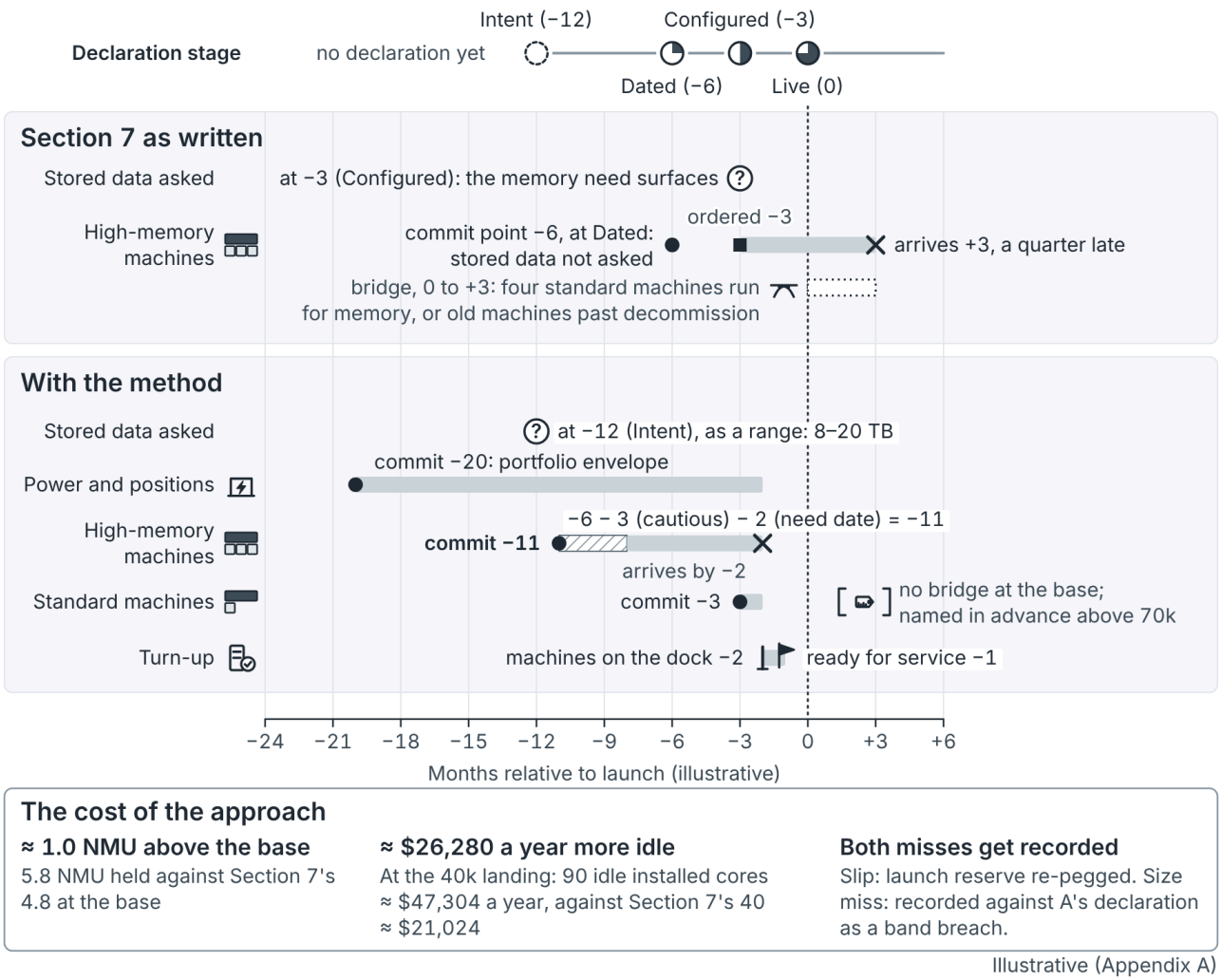


Figure 36. Section 7's launch with and without the method. A cautious lead time and a real need date move the high-memory commit point from −6 to −11, early enough to ask about stored data. The spare it buys has a price, and an owner. Illustrative.

8.5 Where it comes from, and what it adds

Almost nothing in the method is new on its own. Hixson and Guliani supply the frame: "You should plan at least as far ahead as the order time for your resources," use fungibility to delay deciding which product gets the capacity, and keep step changes itemized so the process can learn [6]. Their lead-time table, which ends at "Ready to serve," is step 2 in outline [6]. Olavson's team supplies the base-and-high consensus and execution to the high scenario [8]. Manufacturing supplies netting and time fences (standard practice) and postponement [73]. The operations literature supplies quantity flexibility [70, 71] and chained flexibility [77]. Textbooks supply the newsvendor formula behind the shortage-to-idle ratio [78], and Dixit and Pindyck the option value of waiting [76].

What the method adds, as far as the sources I could read show, is the walk. That's a fixed order of questions applied to one declaration across every layer, ending in signed artifacts. Its new parts would be commit points read against the stage, on a cautious lead time and a real need date, and a standing question about the driving attribute. The rest would be a firm, flexible and named split per layer, bands tied to the stages, and a supply answer per line. The 2026 SRE chapter supplies

pieces of the read half (Section 3) [7]. Its Meet case study ran a daily loop asking whether there were enough resources for the load expected a week out [7]. As far as the public record I could read shows, the per-layer walk tied to a launch's stage isn't there.

8.6 What it doesn't claim, and where it might go

It's **not a model**: the ten-outcome lists, ratios and bands write judgment down; they estimate nothing. It's **untested**: if it works, Section 9.2's tests should show fewer orders placed after their commit point, smaller bridge premiums, and band breaches growing rarer as records build up. It has **blind spots**: coefficient drift (mechanism 3 catches it), correlated launches (Section 6.6's shared-driver check does) and demand nobody can declare (Section 13). And it's **not optimal**. Buying to the 80th percentile on one layer and the median on the next is a rule of thumb. It's for layers that work together (Section 6.5, condition 1), not a joint optimum. A data scientist could do better.

Three explorations, labeled as such:

- **A fast lane.** Hixson and Guliani describe one: fungible pooled resources, provisioned "much more quickly than their full lead time" [6]. *One way to think about it:* Section 7's stocked pool is a fast lane, dearer than the slow lane but able to wait until the launch is configured. Its size is a decision. For example, if configured asks usually move by a tenth of their dated figure, and next quarter's dated asks total 50 machines, start from a pool of about 5 machines. Add cover for its own replenishment time. *My guess* is that with two lanes, the slow lane can be bought lower than Section 6.5's table suggests.
- **Re-pegging.** In the walk-through, the six-week slip freed a launch reserve for six weeks. *Speculation:* across enough launches, re-pegging may absorb a large share of timing misses at almost no cost. Nobody has measured it in public, as far as the sources show.
- **A simulation.** A spreadsheet can stress-test the hand calculation. Draw a few hundred random years from each declaration's stage ranges and each layer's lead-time spread. Then count which launches end up short, and because of which layer. It could show whether per-layer ratios come close to the best joint answer, and what bands and shared drivers change. That's a stress test, not a validation, and its answers would depend on the data used.

9. The operating discipline: keeping the agreement

In short: thirteen mechanisms, each an artifact with an owner, keep the agreement true from one cycle to the next.

Sections 7 and 8 built the artifacts for one launch. The harder problem is keeping them true, cycle after cycle, as people, budgets and coefficients move. Much of this is published. There's a monthly consensus forecast approved by customers, finance and operations, with accountability metrics [8], and capacity contracts with a plan of record approved by leadership [31]. There's also an intake that already knows each team's allocation, usage and idle capacity [34]. And there's the SRE chapter's plan of record (Section 3) [7]. What I haven't found, in the sources I could read, is one set of artifacts and owners that joins them with coefficient ownership, a fulfillment ledger and a declaration register. That joining, argued from practice, is what this paper offers.

9.1 A cadence that turns uncertainty into commitment

At the annual review, one slide sums every team's ask; the next shows finance's number. The cadence exists to close that gap. Each cycle commits only what its lead time forces. The cycles nest, slowest outside:

1. **Annual:** set the envelope and first cut line; pre-position the longest-lead resources; sign forecasts and efficiency commitments.
2. **Quarterly:** specify demand in driver units and shapes, and re-forecast against actuals. Hold a supply review (what supply can commit, where it can't, which risks moved), and place medium-lead orders on its answers.
3. **Monthly:** supply updates commits and named gaps; re-rank launches against supply; move the cut line.
4. **Continuous steering:** arbitrate, broker, run bridges, keep the fulfillment ledger.
5. **Post-launch:** actuals become the next coefficient; log discontinuities; decompose variance.

Each level serves one band of Figure 22's commit points, from intents (annual) to live launches (steering). It's sales and operations planning for internal infrastructure [23], nested by lead time as in hierarchical production planning [3]. Operators publish versions: a monthly rolling four-quarter consensus [8], quarterly plans built on the expected outcome of earlier quarters [1], and budgets whose quarterly or annual boundaries add to lead time [6]. In my experience, the step most often missing is the supply review. That's a fixed point between the demand review and the executive decision, where supply brings its own plan, constraints and risks. Figure 37 lays the cycles on a calendar, with the supply review highlighted.

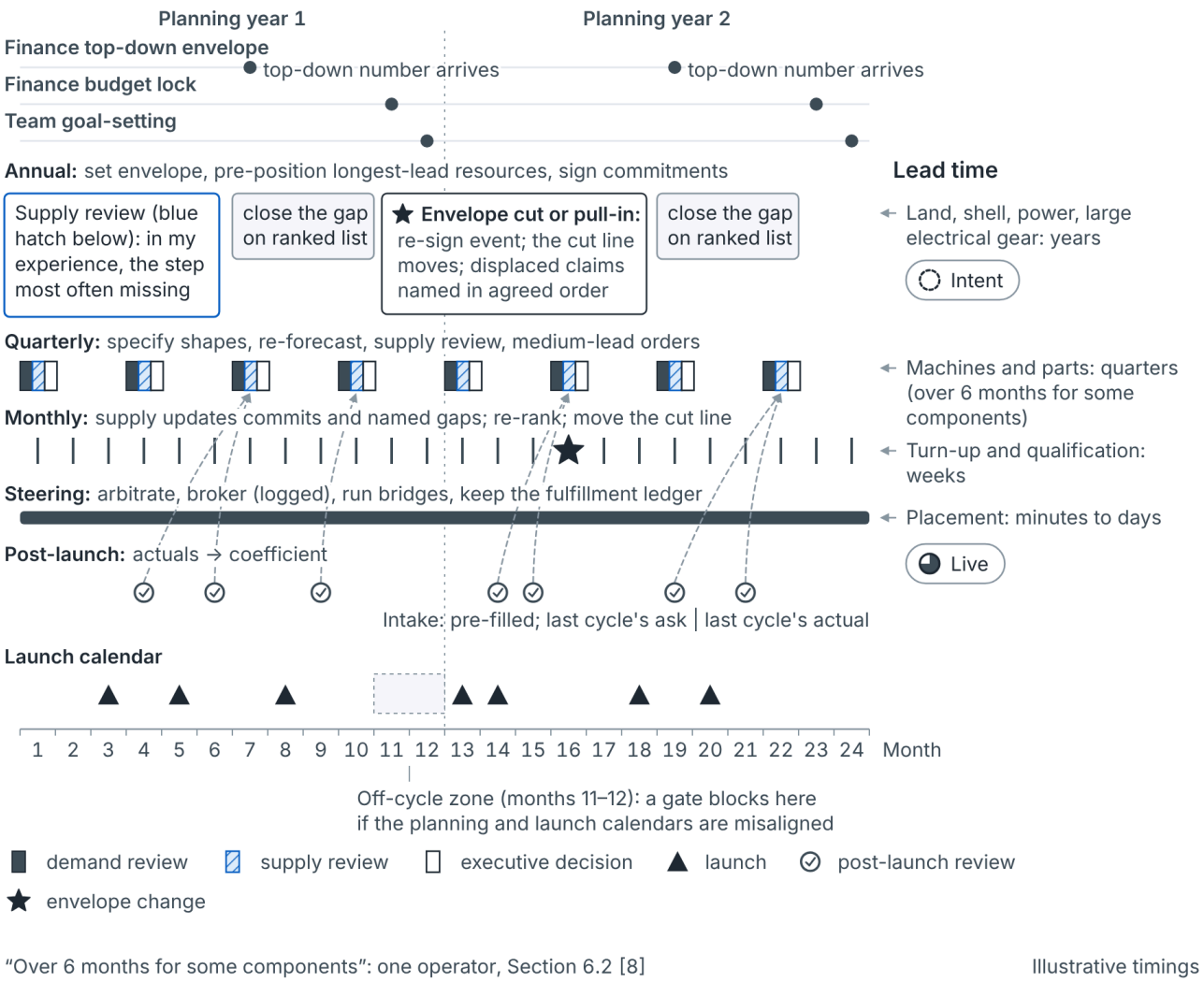


Figure 37 (illustrative timings). Each horizon commits only what its lead time demands. Actuals flow back into the next intake, and an envelope change re-cuts the ranked list instead of reopening every commitment. The supply review sits

between demand and decision each quarter.

Closing the gap with finance. In my experience, finance sets the envelope top-down, from cash, committed depreciation and, at a public company, capital guidance given to investors. Declared demand sums higher. As Hixson and Guliani put it, “it will almost certainly be impossible to fund the capacity requested of each product at its most optimistic growth rate without any improvements in efficiency” [6]. So the envelope draws a cut line through the ranked list, and everything below it is named, not silently trimmed pro rata.

Envelopes move after signing, too. A mid-cycle cut or pull-in is a re-sign event: the cut line moves, and displaced claims are named in an order agreed beforehand (the *re-cut order*). A plan that absorbs a 10% cut (illustrative) by moving one line survives; one renegotiated team by team does not. Figure 38 follows a cut, a slip or a shortage through the re-cut.

Fitting calendars you don’t control. Close the annual horizon a few weeks before finance’s budget lock, and write commitments into teams’ own goals. In my experience, a forum that meets only when decisions are due gets executives; one that meets more often gets delegates.

No surprises in the room. In my experience, a decision meeting goes well when nobody in it sees their number for the first time. So walk the numbers up in steps. First, take the baselines to the leaders who own the biggest launches, for their read on what matters. Then gather demand from the teams, and take it back to those leaders to rank and trim. Only then put the top-down view and the bottom-up one side by side for the executives. Finance, and whoever approves orders, come after that, and they’ll want the business cases for the largest asks already written. In the room, have answers ready. When a question has no answer yet, say so, take it back to the engineers, and bring the answer back by a date. That’s how a forum learns to trust the next answer.

Saying no. Planning must be able to refuse. Some refusals only reflect calendars out of step, and aligning the calendars removes them. For real scarcity, the refusal is a mechanism: written, at intake, citing the binding dimension and the cut line, and owned by the forum. It comes with a path forward: a re-shaped ask, a phased launch, placement elsewhere or a dated slot next cycle. A refusal with no path forward gets routed around, and capacity routed around is no longer planned.

Where does the right to refuse come from? In my experience, rarely from rank; planners usually have none. It comes from the number and the team’s own signature. A team that signed a forecast or utilization commitment, and is far outside it, can be asked to wait, and that’s defensible because the team wrote the terms. Keep the hold for serious misses, and say in advance what counts as one. A gate used for small misses feels like punishment, and gets routed around too. In the first year, enforcement rests on the planner’s persuasion, which doesn’t scale and doesn’t win friends. Mechanism 7 is the way out: write the commitments into the annual agreement, so after that the agreement enforces them, not a person. Once the planning calendar matches the launch calendar, teams know the dates and what they signed, and they unblock themselves by keeping their word.

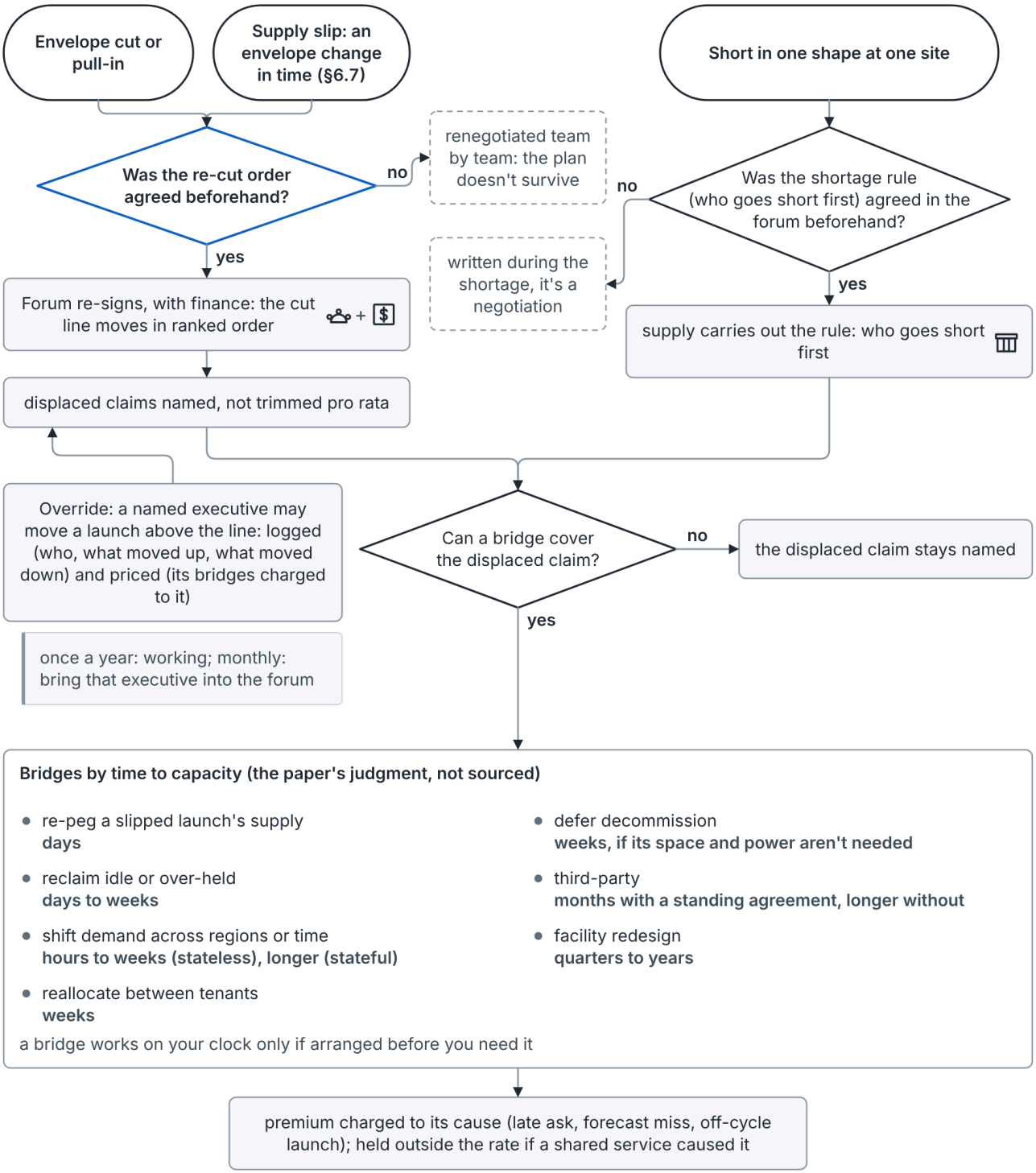


Figure 38. An envelope cut or a supply slip moves the cut line in an order agreed in advance; a shortage in one shape follows its own rule, agreed beforehand. Either way, name what's displaced, bridge what can be bridged and charge each premium to its cause. Bridge speeds are the paper's judgment.

9.2 Thirteen mechanisms

The whole discipline on one page. Read each row as a sentence: *this* artifact, owned by *this* role; without it, *this* goes wrong; *this* indicator shows it's working. The thirteen do four jobs. Mechanisms 1, 2 and 12 keep demand honest, and 3, 4 and 6 keep price honest. Then 5, 11 and 13 keep supply honest, and 7, 8, 9 and 10 make decisions stick. Two rows sit close to the claims, so here's the line between them. The first claim says a coefficient is an agreement; mechanism 3 is the register that keeps it one, version by version. The second says approval is not allocation; mechanism 5 is the ledger that tracks the gap. Figure 39 groups them.

#	Mechanism	Artifact	Owner	If missing	Indicator
1	Declared demand in priced units	Intake pre-filled from telemetry in driver units; team confirms driver forecast, signs any override	Consuming team's accountable lead	Planners guess; forecasts disowned	Capacity with a named declarer
2	Prior ask beside prior actual	Intake columns: last ask, last actual, variance; for step changes, the ask's stage, and size and date against actual	Consuming team; planning supplies data	"Just in case" inflation	Bias by owner at the commit point the ask fed; by stage for step changes
3	Coefficient register and versioned rate card	Each coefficient and rate, published with a version, an effective date, an owner and a valid range	Service owner, with finance	Opaque, drifting rate	Variance from unannounced changes
4	Owned unused and unallocated lines	Each its own line in every report	Service owner (not necessarily the cause); finance keeps score	Remainder folded into rates	Unused and unallocated shares and trends
5	Fulfillment ledger	Per ask: approved (gross) → net → committed → allocated → delivered → live → adopted; firmness class, including uncommitted	Planning carries the gap; each step's performer owns its entry (supply: committed, delivered; site team: live)	Approval treated as delivery; unowned netting assumptions; idle grants	Unfulfilled backlog and its age
6	Discontinuity register	Hardware changes, migrations, re-grains, reclamations, launches	Coefficient owner	Forecasting through breaks	Error after discontinuities
7	Commitments that become institutions	Signed forecast and efficiency commitment in the annual capacity agreement	Consuming team's leader, with finance	Enforcement depends on one person	Capacity under signed commitment
8	Efficiency landing rule	Written split for recovered capacity, agreed before any recovery	Planning, with finance	Incentive to recover dies	Recovery re-funded versus clawed back
9	Cadence aligned to lead time and launch calendar	Published planning calendar	Planning program lead	Hoarding; off-cycle exceptions	Launches needing exceptions

#	Mechanism	Artifact	Owner	If missing	Indicator
10	Standing arbitration forum	Ranked list, cut line, decision log with overrides and brokered trades	Named executive forum, run by planning	Escalation by volume; invisible overrides	Decisions reopened; overrides
11	Supply plan of record and commitment register	Per approved ask, a dated commit with confidence or a named gap; order-to-ask pegging; contracts with liability schedule (owed if cancelled, by date), reschedule terms, owner; decommissions as dated supply steps; substitutions; shared supply dependencies; supply step changes by stage	Supply chain, with planning and finance	Delivered treated as live; approvals treated as firm before supply answers; liabilities found at close	Promised versus actual dates and sizes, by stage, paired with changes to approved asks inside the lead time, by requester
12	Declaration register	Per step change: owner, stage, size and date ranges, change bands per stage, transient and persistent shapes, shortfall tolerance, shared drivers, record	Declaring team; planning keeps the register	Launches arrive as surprises or multipliers; slips hold capacity	Step-change capacity declared a stage before its commit point; size and slip error by stage; share of the high scenario held as options versus owned
13	Commit-point map	Per supply layer, and per declaration (Appendix A): usual and cautious lead time, need date, commit point, usual stage by then, instrument, cost to change shape after commit	Supply chain, with planning	Long-lead supply bought on rumors, or not at all; expedites	Orders placed after their commit point, by cause (late ask or late order); options exercised versus lapsed; bridges per layer against the count the signed ratio implies (Section 10.2)

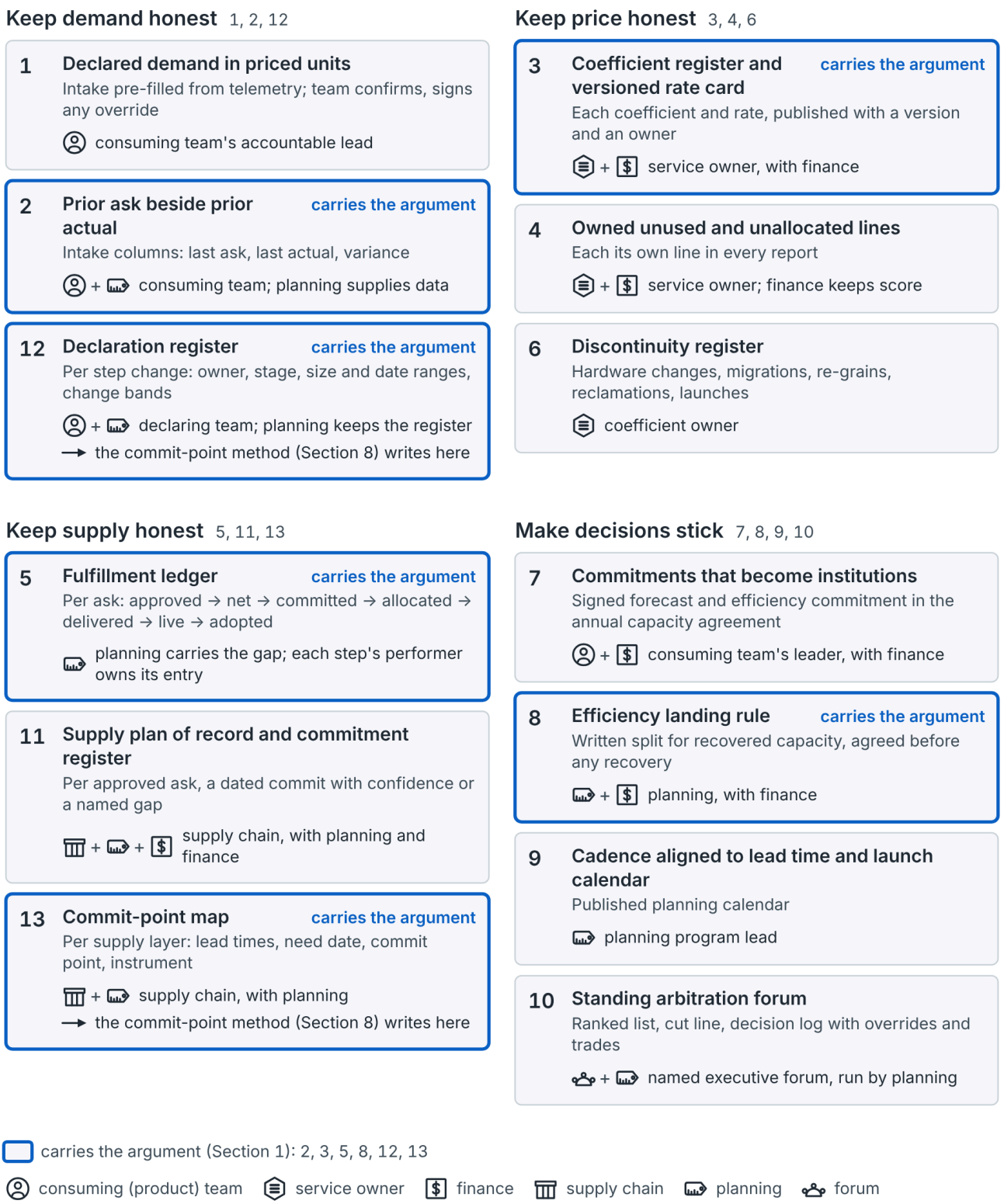


Figure 39. The operating discipline as thirteen artifacts, each with an owner, grouped by the job they do. Outlined: the six that carry the paper's argument.

You'll know it's working when… Seven of the mechanisms have a test that can fail:

- **2, prior ask beside prior actual:** forecast bias falls within a number of cycles stated in advance. If it doesn't, the column is decoration.
- **3, coefficient register and versioned rate card:** variance from unannounced changes shrinks.
- **5, fulfillment ledger:** fewer launches are delayed for capacity. If not, the gap lies in lead time or turn-up.
- **8, efficiency landing rule:** re-funded recovery lasts, and clawed-back recovery decays.
- **11, supply plan of record and commitment register:** the gap between promised and actual delivery dates narrows. And with a supply answer for every ask, fewer approvals turn out to be undesigned gaps; the paired indicators show whether late supply followed late asks.
- **12, declaration register:** scoring declarations by stage makes intents' size error stable enough to discount, if not smaller.
- **13, commit-point map:** fewer orders are placed after their commit point, and their bridge premiums shrink.

The other six (1, 4, 6, 7, 9 and 10) have no test of this kind yet. For them, watch the Indicator column.

Nine decision rights sit outside the Owner column:

- **Overriding the cut line:** a named executive, logged with the displaced claim.
- **Re-cutting the plan when the envelope changes:** the forum, in ranked order, with finance.
- **Promoting, demoting or expiring a declaration:** planning, on the stage's evidence or a missed target date (Section 9.3).
- **Re-signing after a reorganization:** the new accountable leader.
- **Interim owners for orphaned coefficients:** planning.
- **The shortage-to-idle ratios:** finance and product jointly; supply provides the cost of idle and the liability terms.
- **Change bands inside each lead time:** supply and planning, signed by the declaring team, with finance agreeing who pays for a change outside them.
- **Answering an ask with a commit or a gap, and accepting a substitute shape or site:** supply.
- **Reliability buffers:** reliability engineering.

Figure 40 sets the nine rights against the roles.

Decision right	product (declaring team)	finance	supply	planning	forum	named executive	reliability engineering
Override the cut line (logged with the displaced claim)						decides	
Re-cut the plan when the envelope changes, in ranked order		with			decides		
Promote, demote or expire a declaration (on the stage's evidence or a missed target date)				decides			
Re-sign after a reorganization (the new accountable leader)	decides						
Name interim owners for orphaned coefficients				decides			
Set the shortage-to-idle ratios (supply provides cost of idle, liability terms)	jointly	jointly	input				
Set change bands inside each lead time (finance: who pays outside them)	signs	who pays	jointly	jointly			
Answer an ask with a commit or a gap; accept a substitute shape or site			decides				
Set reliability buffers							decides

Also agreed before any recovery: the efficiency landing rule, who keeps capacity a team saves (mechanism 8; planning, with finance)

Figure 40. Nine decisions that sit outside the mechanisms' owner column, and who holds each. Supply holds the answer to every ask; finance co-signs every rule about who pays. The landing rule, who keeps capacity a team saves, is agreed before any recovery.

9.3 Notes on the mechanisms

Pre-fill, then confirm. A blank template returns last year's numbers plus a growth factor. So planning pre-fills the intake from telemetry and current coefficients, and the team adds two things: its driver forecast, and any override of the pre-filled shape, with a reason, signed. Effort goes where the team knows something the planner doesn't.

Talk before you send the form. A pre-filled intake still arrives as a form from a stranger. It comes back more credible when the people filling it in have already talked with the person collecting it, one to one, about their own service. So do that before the intake opens. Then gather teams in small groups, and let each one say what it believes its own biggest launches are. Hand out the pre-filled intake at the end of that session. *My guess* is that teams pad less in front of the peers they'll compete with for the same envelope. Either way, the planner leaves knowing who holds what and what's coming, which is most of what brokering needs later (Section 6.4).

Prior actuals in the intake. When the intake shows “you asked for X and used Y,” the padding talk happens before the ask. Bias (a governance problem) is tracked apart from dispersion (a buffer input), and step changes are scored by stage (Section 5.4).

Declarations expire. A placeholder can hold capacity for a year for a launch nobody mentions anymore. So set a clock: an intent that hasn’t advanced by its target date leaves the envelope unless re-confirmed, and a configured launch’s grant lapses if its date passes unconfirmed. The slip becomes a recorded event. Grid operators learned this the expensive way. Speculative requests helped back up U.S. interconnection queues, and FERC’s 2023 reform moved to “first-ready, first-served,” with readiness deposits and withdrawal penalties [A6]. One public analogue is automated: the SRE chapter describes AI sweeps that reclaim “inactive or expired ML quota credits” [7].

A sweep is only as fair as the rules it follows. The chapter notes that easy access to capacity reduces hoarding [7]; a harsh reclaim, I’d argue, brings it back. So an automated sweep should follow the landing rule (mechanism 8) and honor any right of recall (Section 10.4). It should also log each reclaim like an override, since the chapter warns that “agents can behave in surprising ways” [7]. *My guess* is that teams will judge a sweep less by how much it frees than by whether what they return comes back when needed.

Ask for slow attributes first (Section 6.5). And ask which substitutes the team would accept (other shapes, regions, a smaller first phase, a later date); each one is free supply flexibility.

Shortage rules come before the shortage. The re-cut order handles a smaller envelope. A delivery short in one shape at one site needs its own rule: who goes short first. Agree it in the forum beforehand; supply carries it out. Written during the shortage, it’s a negotiation. And don’t split a shortage pro rata. Supply-chain theory shows that allocation rules rewarding bigger orders make buyers “order more than they need to gain a more favorable allocation” [A5]. As I read that result, a ranked list removes the reward: a padded ask only buys capacity its team pays for.

Efficiency is a funding source. Under chargeback with no reinvestment rule, a team that cuts its footprint gets a smaller bill at best. At worst, it gets a higher rate once a usage-based denominator shrinks (Section 4). Recovered capacity has to land where the recoverer sees it, as gain-sharing plans recognize [4], while in a lean year finance will rationally want to book it as savings. So agree the split before any recovery is known: for example, shares booked as savings, returned to the recoverer and held centrally. A rule agreed in advance is a contract; one proposed after the savings appear is a request.

The landing rule matters most when the envelope freezes, because then the only capital is what can be freed. For example, if budgets freeze partway into a move from an old platform to a new one, the move has to be paid for out of the old platform. One path is to squeeze both ends at once. On the new side, negotiate requirements down to what the team can genuinely run on: every line removed is capacity nobody has to find. On the old side, work with the service owners to raise utilization, land real optimizations and size new launches tightly: every unit freed pays for the next step. Neither end alone frees enough. The first step is the hardest, because it’s paid for before anyone knows what the new platform really needs. So move in small slices. Each one buys headroom and a better read of the exchange rate between old and new (as in Section 5.4, the rollout is the measurement).

The unallocated line is a trust device. Most bills have a remainder nobody can attribute. TBM parks unallocated “fallout” in staging buckets with temporary pro-rata splits [38]. The FinOps Foundation describes an “informed ignore,” budgeting shared items centrally [21]. Neither names an owner accountable for shrinking it. Finance, not planning, should keep the score, or the indicators become self-assessment.

9.4 How to tell if this works

Little of this has been measured in public, as far as I can find. The nearest results come from two sources. Olavson's comparison of forecast methods puts component safety stock (the parts buffer of Section 6.3) at about 0.6 times its customer-driven level under triangulation [8]. Triangulation here means customer forecasts checked against statistical ones. Gupta reports that a driver-based intake cut a three-to-four-month planning exercise to about one month [34].

Each mechanism's own test sits with it, in Section 9.2, under *You'll know it's working when…*

10. People: the room around the math

In short: capacity is a handshake among capital, supply and demand, and the planner brokers it. Then come the seams, overrides, reorganizations and gaming.

Every capacity plan is a handshake among three parties. **Capital** is finance and the leaders who set the envelope: it has the money, and wants it to buy something. **Demand** is the teams that launch and grow products, and the shared services they call: they have plans, and want capacity on time. **Supply** is whoever turns money into running machines: the supply chain, the sites, the fleet. It has lead times and contracts, and wants a signal it can buy against.

None of the three can close the deal alone. Capital can approve, but it can't deliver. Demand can declare, but it can't fund. Supply can build, but only against a signal that's often late, padded or both. That's where the planner sits: in the middle, usually owning no capacity and no budget.

So what does a planner trade in? In my experience, four things:

- **Information:** what's coming, who holds what, and what supply can really do by when.
- **Timing:** when each question must be asked, so the answer arrives before the order has to be placed.
- **Trust:** numbers that hold up, from a planner who doesn't pad its own.
- **Credible commitments:** signatures that turn one party's promise into something the others can buy against.

The planner owes each corner something different. Capital gets numbers that hold, a cut line that names what falls below it, and early warning of finance decisions dressed as capacity ones. Supply gets signed demand before each commit point, with its stage and range, not churn. Demand gets the same picture as everyone else, a "no" that comes with a path, and capacity back when it gives some up.

The paper's two claims sit on the triangle's edges. A coefficient is the price demand pays capital for what supply provides, so it holds only while all three agree on it. And approval is capital's handshake with demand; allocation is supply's. The gap between them is the third seam (Section 10.1).

A shared service sits on two corners at once: it's supply to its callers and demand to the fleet. That's why the first seam below exists. Section 10.1's Figure 41 draws the triangle, with the seams on its edges.

10.1 Three seams

A platform lead gets blamed for a shortage in one meeting and for the bill in the next. It isn't personal. In my experience, the most persistent conflict in this field is structural, and it runs along three seams.

The first seam runs between product and platform. Product is rewarded for launches and growth; platform is measured on cost and reliability, and blamed for both. Section 5.2's identity sits on this seam: volume belongs to product, coefficient

and rate to platform. Neither can manage cost alone; the mechanisms keep the conflict productive, not personal.

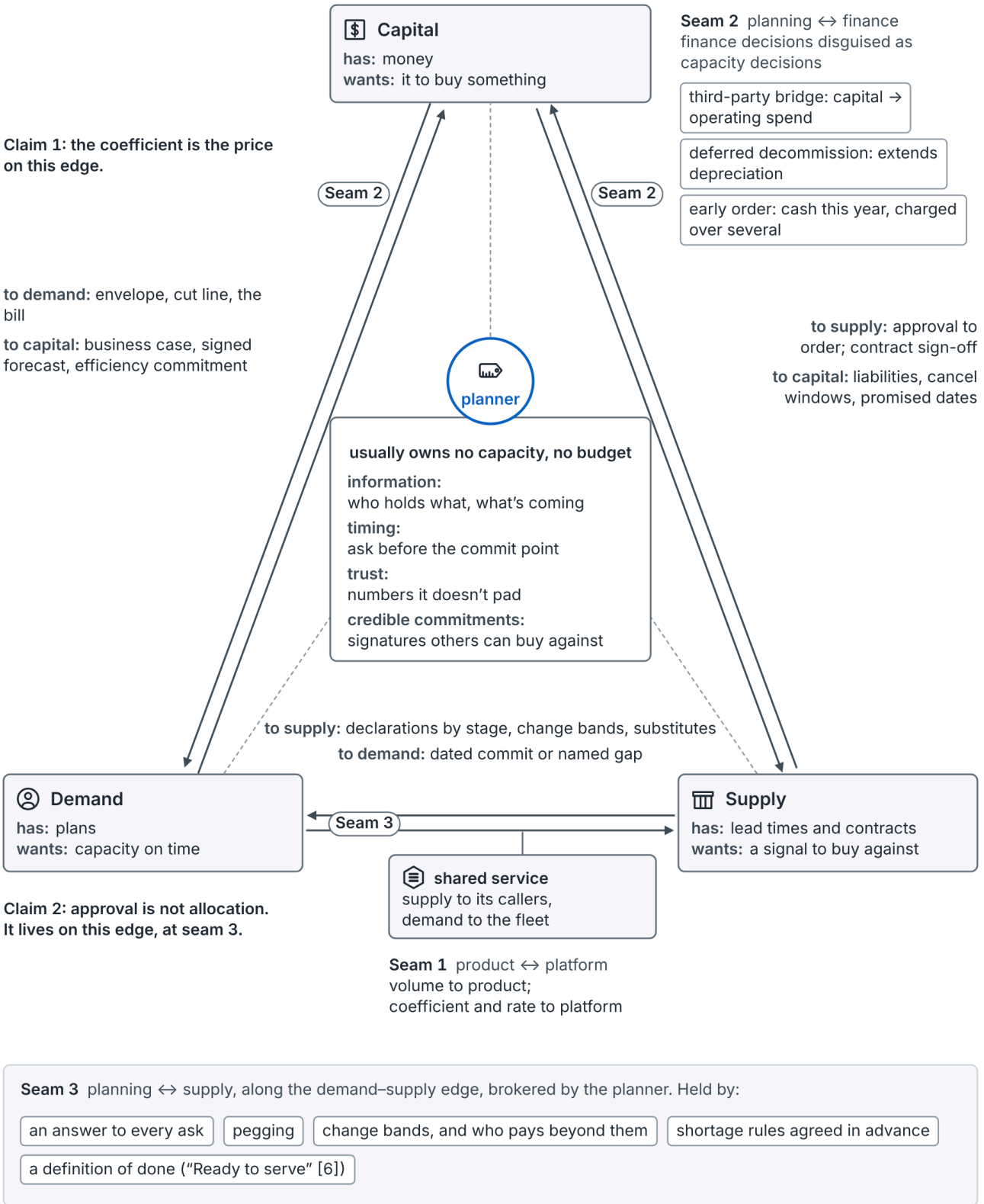
The second runs between planning and finance. Planning thinks in machines, megawatts and lead times; finance, in capital and operating spend, depreciation schedules, and a forecast already given to its own leadership. Some decisions that look purely technical cross this seam:

- A third-party bridge turns capital into operating spend.
- Deferring a decommission extends depreciation on an asset finance had written off its plan.
- An early order commits cash in one fiscal year for capacity charged back over several. So a plan reconciled only to the rate can approve spend the cash budget can't carry.

Each is a finance decision disguised as a capacity decision. Bring finance in only to set the rate, and you'll discover them at the quarterly close.

Finance often distrusts planning's numbers, and it often has reasons. Think of inflated asks, double counting, missing baselines and "just in case" capacity. Add savings promised and never seen, and teams that can't explain their own use of shared services. Trust comes back the slow way. Use one template and one format for every team, so asks can be compared. Put each team's baseline and last year's actual beside its ask (mechanism 2). Ask for a short business case tying the ask to growth, a launch, efficiency or risk. And show constrained supply against ranked demand on one page, with a way to drill down. None of it is clever. It's what makes an ask believable to someone who's been burned before.

The third seam runs between planning and supply, and it's where "approval is not allocation" actually lives. Planning approves in a common unit; supply buys parts, racks and buildings on dated contracts with penalties. Demand that changes inside supply's lead time makes every supplier look late; supply that commits without naming its risks makes every approval look firm. So the seam needs five artifacts both sides sign. Four are an answer to every approved ask (Section 8.3), pegging (Section 6.4), change bands and who pays beyond them (Section 6.5), and shortage rules agreed in advance (Section 9.3). The fifth is a definition of done for each handoff: what "delivered," "live" and "ready for service" mean and who accepts each, anchored on Hixson and Guliani's "Ready to serve" [6]. The SRE chapter warns against treating "ready" as binary, so a definition can have phases [7]. Supply gets matching decision rights (Section 9.2). An approval becomes firm when supply has answered it, not when the forum has voted. Figure 41 puts the three seams on the triangle's edges, with what has to cross each.



The seams of Section 10.1 run along these edges.

Figure 41. Capacity is a handshake among capital, supply and demand. The planner sits in the middle, usually with no capacity and no budget, and trades in information, timing, trust and credible commitments. The three seams run along the edges.

10.2 Overrides, reorganizations and misses

Overrides. Sometimes a senior leader will move a launch above the cut line for a reason the forum couldn't weigh, such as a contract or public commitment. Make overrides:

- **Allowed:** the executive who owns the trade-off may make it.
- **Explicit:** the log records who decided, what moved up and **what moved down**.
- **Priced:** any bridge the override needs is charged to it.

A rule of thumb from my experience: a forum overridden once a year is working. One overridden monthly is not the decision-maker, and the fix is to bring the overriding executive into it.

Reorganizations. A reorg can orphan a coefficient, unnoticed until it drifts. So attach every commitment, coefficient and unallocated line to a durable object (a product line, a cost center, a service identity), never a person or team name. Treat the reorg as a re-sign event: within one cycle, the new leader re-signs or formally amends the inherited commitment; until then it stands. An orphaned coefficient gets an interim owner and a transfer date.

A new owner needs more than a signature request, though. Engineers often know their own integration path well and the shared service's operating model barely. That means how a request is sized, what quota means across many sites, and why one launch waits on another. So onboard a new leader, or a new team, in one sitting. Show it what it asked for last year and what it used, its coefficients and who owns them, the commitments it inherits, and the dates that matter. Size its first ask with it, not for it. And tell it who to call when an alert fires or a launch is blocked. A team that understands its first ask usually writes a better second one.

Orphans don't only come from reorganizations. Some work falls between charters from the start. For example, if reliability engineering's charter covers serving and nothing else, the batch pipelines that feed serving may belong to no one. When they fail, the product degrades or loses features without a clean outage, so no one's alarms go off. That kind of work is often the work that matters. So once a year, lay the charters over the chain that delivers the product, end to end, and name an owner for every gap. Planning can hold a gap until someone takes it.

Planning's own misses. Finance remembers idle capacity; product remembers the launch that slipped. Left implicit, the shortage-to-idle ratios of Sections 6.3 and 6.5 get set by whichever critic spoke last. So finance and product sign them at the annual cycle, per supply layer and kind of demand, with supply supplying the cost of idle and the liability terms. Reliability engineering set the precedent for availability: "The business or the product must establish the system's availability target" [79]. The SRE chapter says it for capacity: base buffer sizes on "SLOs and business impact," with product, SRE and finance aligned on "risk tolerance, and cost-benefit thresholds" [7]. What this paper adds is a signed number per layer. Illustratively, that means Section 6.5's ratios for persistent demand, and near 1:1 for transient peaks. Surplus within them is the plan working, not a planning failure.

The SRE chapter ties capacity planning to error budgets (Section 6.3) [7]. A signed ratio makes that budget a number you can count. *One way to count it:* a 4 : 1 ratio covers the eighth of ten outcomes. So if each launch is bought at its own cover point, about one launch in five should need a bridge at that layer. Call it a *shortage budget*. For example, if 20 launches a year buy machines this way, expect about four bridges. Far fewer, two years running, and the ranges are padded. Far more, and the stage records run optimistic or the ratio is too low. It's a backtest of the ratio, the kind the chapter recommends for choosing a confidence level [7]. One caution: launches tied to one driver spend the budget together (Section 6.6). This is an analogy, not a result.

When a miss happens anyway, the funnel and the decomposition show where it arose, often shared. Resolve the launch before assigning cause.

Shocks. When a big shock hits (a new site slips by quarters, a key supplier stalls), the informal channels that planning runs on can stop overnight. The planner has to replace them on purpose, and fast. Give every team the same short briefing: the state of the fleet, what's known about the shock this week, where supply stands, and where finance stands. Then work through that team's own forecast and requirements with it. In my judgment, the same picture for everyone keeps rumors from turning into forecasts. It also makes the hard trades (tighter forecasts, reworked asks, moving tenants off constrained capacity, a bridge) feel shared rather than imposed. While supply is unknown, a simple holding policy helps. Everyone stays at current capacity plus approved growth and buffers, and everything else goes through a stricter review. The policy also says when it will be revisited.

Re-baseline. After a shock, or when plan and actual have drifted far apart, the old plan stops being a useful baseline. In my experience, patching it line by line rarely converges. It works better to pick a date, take a snapshot of actuals, and make that the new baseline. Then work through the teams one by one, brokering trades between those holding excess and those running short (Section 6.4). Hold the utilization floor agreed beforehand: capacity that keeps a team below it is excess, and goes back into the shared reserve, under the landing rule (mechanism 8). Then restart the forecast from the new baseline, with each team's declarations refreshed. Section 5.3 restates a coefficient's history at a break; this restates the plan's. Section 5.2 suggests making it routine rather than a one-off.

10.3 Gaming

Say an urgent email arrives the week after the cut line was drawn: an undeclared launch needs capacity now. If that ask costs its sender nothing extra, you've taught the whole organization when to file.

Capacity requests are budgets in hardware units, pathologies included. People "lie in the formulation of budgets" and "game the realisation" [95], and organizations get the behavior they pay for [96]. The common games, and their counters:

Game	What it looks like	Counter
The late ask	Demand arrives after the cut line as an emergency	Bridge premium charged to it; late asks reported by owner
Relabeling	Organic growth declared as a launch to jump the queue	Launch priority requires a dated declaration with a named owner
Stage inflation	A declaration presented as configured to win a grant or a place above the cut line	Promotion only on the stage's evidence (Section 5.4); size and slip scored by stage
Parked intents	Placeholder intents filed to reserve a share of the envelope	Intents enter at their stage's record and expire if they don't advance
Reliability as trump card	Buffer labeled "reliability" so no one questions it	Named buffers (Section 6.3), each sized by its owner
Holding for a slipped launch	Capacity reserved for a launch that quietly moved	Grants expire unless re-confirmed
Churn inside the lead time	An ask changed again and again after supply has ordered	Change bands; changes inside the lead time reported by requester
Planning's own padding	Unstated buffer added by planners	Forecast-error buffer as a signed line

The last row matters most: planning that pads its own numbers can't credibly ask anyone else not to. Figure 42 pins each game to the point in the cycle where it pays off.

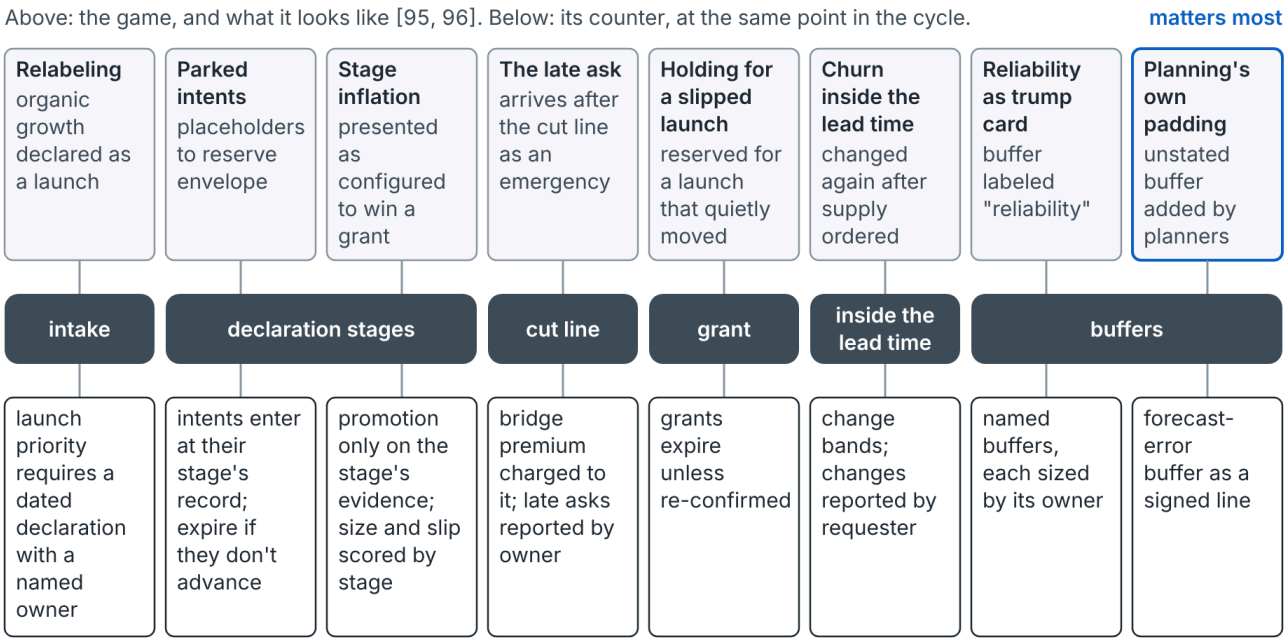


Figure 42. Each game pays off at one point in the cycle, and its counter sits at that point. The one that matters most is planning's own.

10.4 Legitimacy, change and staffing

Past a point, legitimacy matters more than precision in whether anyone acts on the number. Ostrom's design principles for durable commons map onto shared infrastructure [97], and one matters most: a channel to challenge coefficients. A consumer who can challenge a coefficient, and sometimes wins, trusts the rest of the bill. Figure 43 draws the channel with three parts the paper already has. The first two are the coefficient register, with each coefficient published and versioned (mechanism 3), and the forum, which rules and keeps the decision log (mechanism 10). The third is the discontinuity register (mechanism 6).

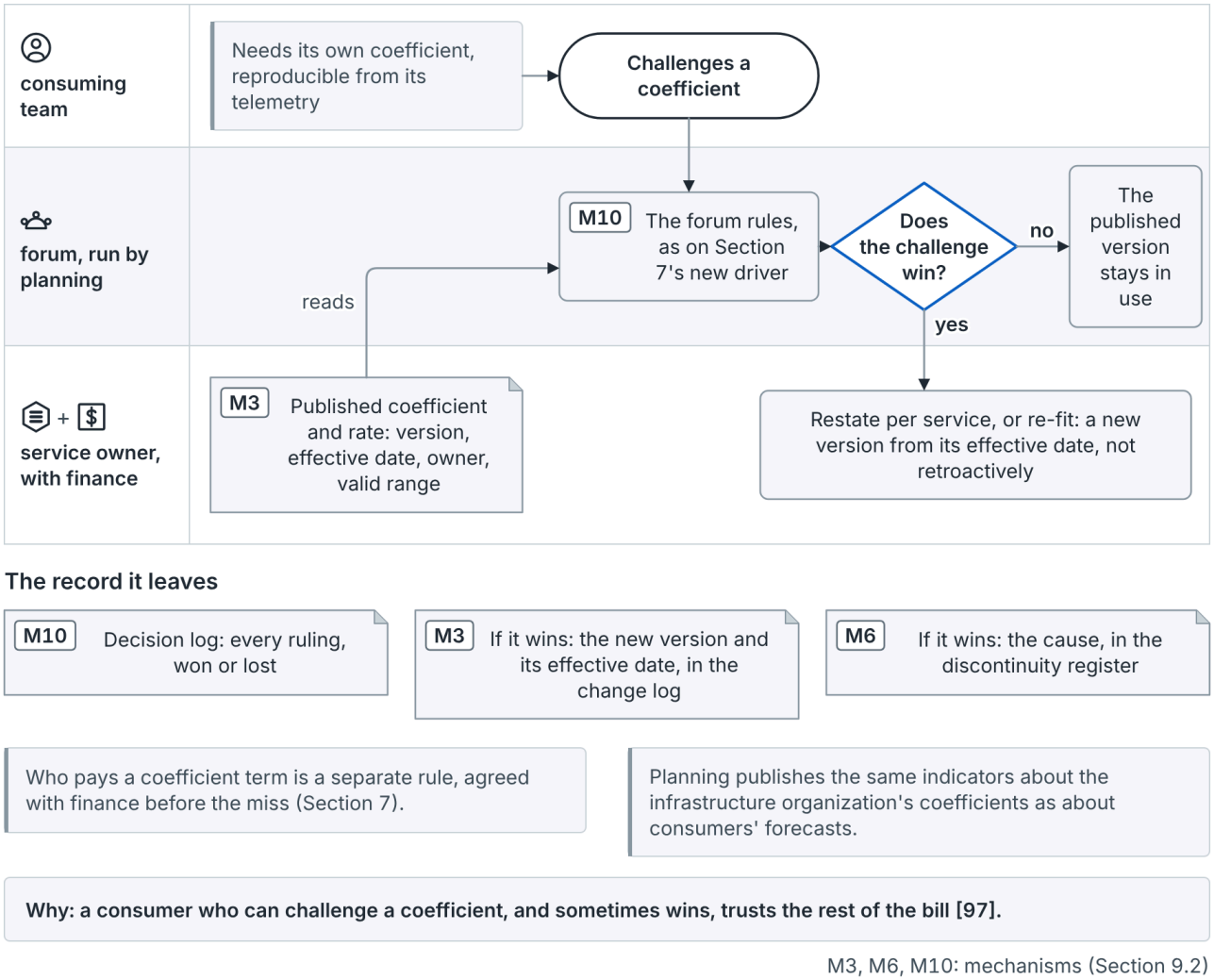


Figure 43. A channel to challenge coefficients: the consumer brings the challenge, the forum rules, and sometimes the consumer wins. Every ruling goes in the forum's decision log; a win also leaves a new version, from its effective date, with its cause recorded.

Escalate in steps: nudges before bills, bills before constraints. Spotify nudged only "teams that are growing faster than Spotify's business" [98]. Neither showback nor chargeback is more mature than the other [99], but chargeback without the right to change consumption is a tax.

Change in the right order. Moving from team-held reservations to shared pools asks a team to trade a certain asset for a promise. So earn it: prove fulfillment first, pool the buffer before the base, and give returners a right of recall. Move to attributed cost only after a full cycle of showback.

Give each audience what it needs. Executives need one page of decisions. It shows what is above and below the cut line, and what just below it could be funded and at what cost. It also shows what above it carries fulfillment risk and until when, and any decision needed today. Engineers need their own coefficient, reproducible from their telemetry, with forecast error treated as data, not fault.

In between, the planner translates. Product speaks in launches, users and requests per second. Supply speaks in shapes, sites and dates. Finance speaks in dollars, years and depreciation. In my experience, much of planning is saying one party's need in another party's words without losing what it meant. Reviews help when every line is one of three things: a **fact** (measured), an **assumption** (someone's judgment, with an owner) or an **ask** (a decision needed, by when). A review that mixes them becomes a status update; one that separates them produces decisions.

Where planning reports matters less than who owns the cut line. Inside infrastructure, planning knows supply but is suspected of favoring it; inside finance, it's trusted on numbers but not on shapes. Either works if the forum owns the cut line, and planning publishes the same indicators about the infrastructure organization's coefficients as about consumers' forecasts. And don't let brokering live in one head: log every trade and rotate the role, or unlogged trades become a second, invisible arbitration forum.

11. What changes in the AI era

In short: the structure holds for AI fleets. The units move toward megawatts and tokens, and a busy chip isn't necessarily a productive one.

For the largest operators' AI fleets, by their own public account, the binding constraint has moved toward power. Asked what kept him up at night, Alphabet's chief executive said "the top question is definitely around compute capacity, all the constraints, be it power, land, supply chain constraints". To another analyst, he said he expected to go through the year "in a supply-constrained way" [90]. Microsoft's chief executive described "maximizing tokens per dollar per watt" [72]. Meta reports having "recovered hundreds of megawatts of power" [100]. Google's SRE chapter states it as operating practice: "strict supply constraints define fleet planning," and "capital alone cannot solve the problem" [7].

When megawatts bind more than dollars, rank by value per unit of the binding resource, not by dollars (Section 4). Define the unit: critical IT megawatts. State the power usage effectiveness (PUE) used to convert to facility power, design or annualized. Illustratively, 10 MW of critical IT load at a PUE of 1.2 is 12 MW of facility power. Say which power: contracted, provisioned or drawn. Rack power density and cooling can also limit which accelerators a building hosts (Section 6.2).

Scarcity turns unused capacity into competing claims. Amazon's chief executive said "as fast as we install this capacity, this AI capacity, we are monetizing it" [101]. Where that holds, the unused line shrinks toward its buffers' floor (Section 6.3), and the cost that matters becomes the best claim displaced. Alphabet's chief financial officer said that for 2026 "just over half of our ML compute is expected to go towards the Cloud business" [90]. In July 2026, its chief executive said "we continue to be supply constrained" [102]. He described a priority order: what it takes "to continue AGI development at the frontier" first, then core products and Cloud [102]. Operators disclose such splits' outcomes, not their mechanisms.

Topology is a shape. Large training jobs need contiguous accelerators in one interconnect domain. So a fleet can hold enough chips and still be unable to place a job: the "geometry" failure of Section 6.4 [18]. A study of Alibaba's GPU clusters finds the same at a finer grain: sharing GPUs fragments a cluster until hundreds of GPUs can't be allocated [103]. Lumpy training demand suits declaration (Section 5.4); serving demand is driver-based.

Some AI demand is a policy, not a forecast. *One way to think about it:* an agent fleet's owner holds the dial. It sets budget caps, tokens per task and retry rules. A volume forecast for it is a forecast of the team's own choices. So let its declaration be the policy: the cap, an alert on how fast it burns, and the rule for raising it. Score the team on keeping the cap, not on forecast error. Demand that users drive, such as a feature that spreads, stays a forecast. *My guess* is that a padded cap is no better than a padded forecast, so caps need the same stage records (Section 5.4).

A training cluster is a step change on both sides. The run is declared, not extrapolated. Its block of accelerators needs power and cooling committed years ahead, while the run is still an intent. That commitment is made against a hardware generation that may not ship on time (Sections 6.5 and 6.7). A model launch is the opposite: a transient inference spike on a persistent step. Declare it as two shapes (Section 6.6). Figure 44 maps what changes, and shows a training cluster as a step on both sides.



Figure 44. For AI fleets the structure holds, but the units move: for the largest operators, by their own account, megawatts bind, an interconnect domain is a shape, a token has two phases, and a training cluster is a step change on both sides at once.

The token is a non-uniform driver. A request has two phases: reading the prompt (prefill) and writing the answer one token at a time (decode). DistServe serves prefill and decode on different GPUs under separate latency objectives [104]. Prefill cost grows faster than linearly with prompt length. Decode cost rises with context length, since each new token reads back over the key-value cache. That cache grows with context, consumes accelerator memory and so limits batch size (how many requests share a chip at once). Illustratively, 1,900 tokens in and 100 out is mostly prefill compute; 100 in and 1,900 out is 1,900 decode steps, each holding cache memory. One price per token misprices prefill-heavy, decode-heavy and long-context work. Batching also revives Dapper's coalescing problem, so declare the apportionment rule.

Utilization is the wrong productivity measure. ML Productivity Goodput multiplies scheduling, runtime and program goodput, rejecting the assumption that "Utilization == Productivity". Runtime goodput counts only progress saved in checkpoints, so work lost to failures and preemptions discounts supply [105]. Illustratively, of 100 accelerator-hours, a job is scheduled for 90 (scheduling goodput 0.9). A failure wipes out 9 hours since the last checkpoint (runtime goodput 0.9). The remaining 81 run at 70% of what the program could achieve (program goodput 0.7). So $0.9 \times 0.9 \times 0.7 \approx 0.57$: about 57 hours of real progress.

For an accelerator fleet, then, keep practical capacity as the denominator (Section 4) and split it four ways, not two:

- **Idle capacity** (the 10 unscheduled hours) goes on the unused line, owned by the service.
- **Work lost to failures and preemptions** (the 9 hours) is owned by its cause: infrastructure for hardware failures, scheduling policy for preemptions, the workload for its checkpoint interval.
- **Allocated time below the program's achievable efficiency** (about 24 hours) is owned by the workload (Figure 15, gap f).
- **Goodput** is the remainder (about 57 hours).

Fold the middle two into the unused line, and the provider is charged for losses the tenant caused. Figure 45 shows the split.

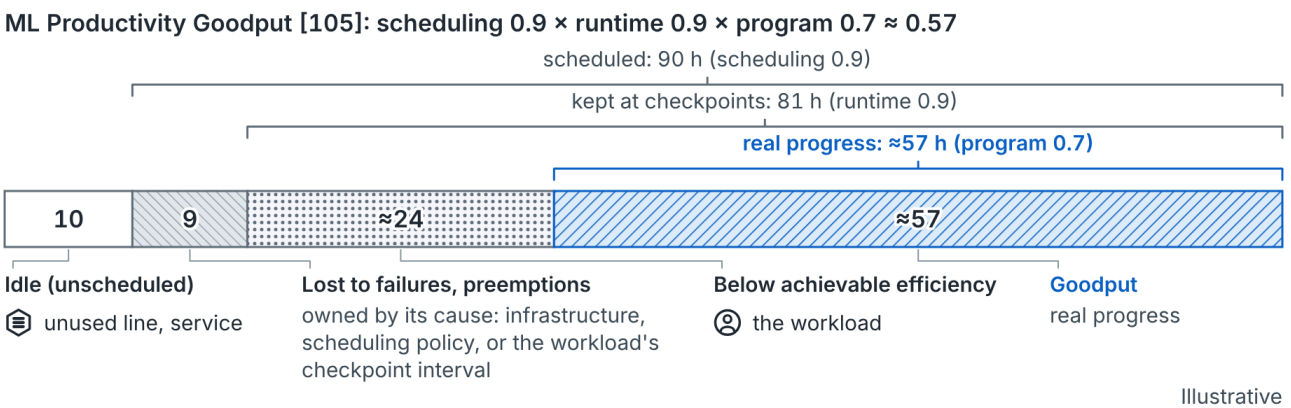


Figure 45. Of 100 illustrative accelerator-hours, about 57 are real progress. Each of the other slices has its own owner, and folding them together charges the provider for the tenant's losses.

Coefficients, lives and prices shift. The FinOps Foundation writes that "the unit economics of inference are non-stationary" [106]. Economic life can end before accounting life when a new generation is more efficient per watt, so a rate set on accounting life understates cost (Section 4). An old warning returns: Jevons argued in *The Coal Question* that economy of use can raise total consumption [107]. Today's version: "a token may get cheaper; tokens, in aggregate, are not" [106]. Price alone may not explain the growth. On one routing platform, "a 10% decrease in price corresponds to only about a 0.5–0.7% increase in usage" [A7]. There, agentic workloads passed humans' token use around February 2026, at "about 15x more per request" [A8].

12. For mid-sized companies

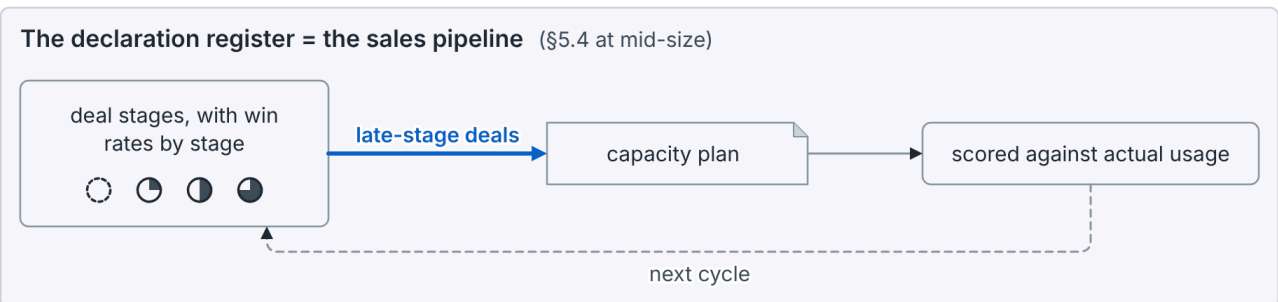
In short: at mid-size, your long-lead supply is the cloud commitment, and your list of launches and big customers may already exist as the sales pipeline.

A company of a few hundred to a few thousand people usually has no data centers and no planning organization. But it runs shared services (a database tier, a data platform, an observability stack, cloud commitments). And it has the same problem: the teams that cause cost aren't the teams that see the bill. The translation:

- **Long-lead supply:** multi-year cloud commitments, reserved capacity, colocation and license renewals. They're negotiated on the vendor's quarter-end calendar, so the renewal date is usually the real annual cycle.
- **The fulfillment gap:** commitments that don't match the shapes and regions used; licenses bought but not deployed. A discount isn't capacity, either: Azure says its Reserved Instances "provide a billing discount only and don't guarantee capacity" [58].
- **The coefficient:** cost per customer, transaction or pipeline run.
- **The unallocated line:** the share of the cloud bill no tag explains, plus software licenses and AI model usage, often on separate invoices or company cards. These can rival the infrastructure bill and have no owner at all.
- **The arbitration forum:** a regular CTO–CFO meeting, which owns the cut line.
- **The broker:** often nobody. Capital is the CFO, supply is the cloud account and its renewal date, and demand is the engineering leads. Someone has to carry information and timing between them; name that person, even part-time.
- **Chargeback:** showback may be the right end state. Below a certain scale, a monthly report each team can reproduce does most of what chargeback would.

Step changes follow the same logic. The demand step is a large customer onboarding, a campaign or a product launch; the supply step, a commitment renewal or a provider move. Many companies already keep a declaration register under another name: the sales pipeline, with stages and historical win rates by stage. Feed late-stage deals into the capacity plan, then score them against actual usage. That's Section 5.4 at mid-size. Figure 46 translates the paper's terms.

The paper's term		At mid-size (§12)
long-lead supply	→	multi-year cloud commitments, reserved capacity, colocation, license renewals: on the vendor's quarter-end calendar, so the renewal date is usually the real annual cycle
fulfillment gap	→	commitments that don't match the shapes and regions used; licenses bought but not deployed
coefficient (α)	→	cost per customer, transaction or pipeline run
unallocated line	→	the share of the cloud bill no tag explains, plus licenses and AI model usage on separate invoices or company cards
forum	→	a regular CTO–CFO meeting that owns the cut line
chargeback	→	showback may be the right end state



Don't:

- chargeback before a full cycle of showback
- a unit-cost gain booked as savings without asking whether volume grew
- a tool before drivers and owners
- a three-year commitment forecast as elastic spend

Figure 46. At a few hundred to a few thousand people, the long-lead supply is the cloud commitment and its renewal date, and the declaration register usually exists already as the sales pipeline.

Do you need this? The SRE chapter's first question is whether you need "the rigorous and expensive process" at all [7]. For small workloads, it says, "simple overprovisioning is often the best strategy," and precision is "only valuable relative to" your other problems [7]. Hixson and Guliani said the same in 2015: for small companies, the limit is "engineering time or management attention," not capacity [6]. *One way to answer it* is layer by layer. Compare each supply layer's lead time with how long a launch takes to firm up, from dated to live. Where supply is faster, react: autoscaling and shared buffers do the job. Where supply is slower, that layer needs a commit point. For most mid-size companies, *my guess* is that it's one layer: the commitment renewal. One path, in steps:

1. Overprovision and autoscale; watch the bill.
2. Show each team its cost, with an owner for the three largest coefficients.
3. Feed late-stage deals from the sales pipeline into the plan, and score them.
4. Walk the renewal through the worksheet (Appendix A).

Stop at the step where the next one costs more than it saves.

What not to do. Don't build chargeback before a full cycle of showback, or buy a tool before drivers and owners are agreed. Don't book a unit-cost improvement as savings without asking whether volume grew to meet it. Treat a three-year commitment as the long-lead decision it is: forecasting that applies "KPI growth on actual spend" [108] suits elastic spend, but committed capacity also needs lead time and shape.

Six questions a CFO or CTO can ask.

1. Who owns the cost of each of our three largest shared services, by name?
2. What is our cost per unit of the driver the business cares about, and did last quarter's change come from volume, efficiency or price?
3. What share of the bill does no one own?
4. When does our largest commitment renew, and who decides?
5. What did each team ask for last year, and what did it use?
6. Which launches or large customers are coming in the next two quarters, at what stage, and what have we committed for them?

13. Where this breaks, and what is unknown

In short: where this paper's approach breaks, and what hasn't been measured in public, as far as I can find.

Caching, batching, statefulness. Per-request coefficients fail when cost depends on cache state, batch composition or data size. Miss-rate coefficients, declared apportionment rules and load-test models help, but Flux's residuals are partly a modeling problem, not only an ownership one [2].

Elastic cloud. In cloud, fulfillment moves to the provider but comes back as regional stockouts and reservation planning. Even guaranteed cloud capacity has a lead time. AWS accepts a future-dated capacity reservation "between 5 and 120 days in advance," recommends asking "at least 56 days (8 weeks) in advance," and may decline it [109]. Below materiality, the mechanisms cost more than they return, as the SRE chapter also advises (Section 12) [7].

No reference class. Take a step that's new at the resource level, such as a first accelerator cluster or a first region on a new continent. The planner can only bound it, phase it and buy options.

Demand no one can declare. For a step no one sees coming, such as a product that spreads far faster than planned, what remains are buffers, bridges and, at the limit, admission control [33]. Hixson and Guliani's Plan B for a "success disaster" covers the same case: graceful failure modes, efficiency work with an estimated time to deploy, and alert thresholds [6]. The SRE chapter calls it unplanned inorganic demand, and its Meet case is the public example [7]. Google's public account reports "30x demand" in three months, met while working against "a previously unthinkable 50x growth forecast" [110]. Its planners "could no longer rely on the extrapolation of historical data" [110]. Under the same shock, Azure's allocator team reports that "we increased the utilization limits in each cluster by 1%," spending buffer on purpose [111].

A public case, read through this paper. Here's one way to read the SRE chapter's Meet case with this paper's tools. It's my reading; the chapter's lessons are its own. The first signal was pages for regional capacity issues [7]. Declarations still fed the loop, since its loop started from four reviews, one of them customer onboarding and promotion plans [7]. But as I read it, none could have been made in time for the slow layers. So most slow layers' commit points had already passed. The team's daily loop asked whether there were enough resources "for the load we expect one week from now" [7]. That's the commit-point question with a one-week horizon, set by what could still move. What moved was what could move inside weeks. Machines came through regular ordering and through borrowing from other teams, a tenant-to-tenant bridge

(Section 6.4) [7]. Bigger instances were a re-shape (Section 6.1), and efficiency work also moved capacity [7]. Then came "escape hatches" to degrade quality, built during the response before a crisis forced the team's hand [7]. That's the named slice of Section 8, step 6, built just ahead of need.

Two things follow. Where no one can declare in time, the named slice is what's left, so build it first. And the team had "deliberately avoided investing in deployment automation or capacity optimization" because the return wasn't there [7]. Skipping the discipline below materiality is a fair choice. It has a tail, and the chapter says so. Figure 47 puts the case on the two clocks of Figure 22.



Figure 47. The Meet case on the two clocks: most slow layers' commit points had already passed; machines came through regular orders and borrowing, and fast levers, some built just ahead of need, did the rest. Schematic; a reading of a public case.

Indicators become targets. By Goodhart's law, any indicator in Section 9.2 can be gamed once rewarded, so read them as a set.

What is unknown. This is argument from published sources and practice, not a measured study. In the literature I could read, I found no time series of coefficient drift, and no study of how often coefficients are recalibrated. Nor did I find published data on how well step-change declarations are calibrated by stage, or how often launches slip. One preprint describes arbitration among internal teams for ML training capacity [18]. Operators have stated splits and priority orders between internal and external demand [90, 102], but I found no published mechanism for arbitrating scarce accelerators between them. On how much uncertainty operators carry at a commit point, the published evidence I found is observational. One example is a "2x difference in the machine count forecasts during annual planning discussions" [8]. Attribution for stateful services and batched inference is open, and the tests of Section 9.2 have not been run.

On supply, I found no published order-to-dock time, turn-up duration or hardware contract terms. And Lawrence Berkeley National Laboratory finds little published on how long large new loads wait for power [65]. The commit-point method of

Section 8 is untested. The most useful next contribution would be multi-organization data on bias, unallocated share and fulfillment age, before and after these mechanisms are adopted. Figure 48 sorts the paper's main pieces by the evidence behind them.

Piece of the argument	Published	Argued here	Not found, or open
Cost-driver formula, computed at scale	published [1, 2]		
Attribution coverage and residuals 52% of usage traced (2022); residual 3–5% stateless, 22% → 15% databases	published [2]		open: stateful, batched
Lead times by layer	in part [6, 8, 60–64, 109]		order-to-dock; turn-up
Hardware contract terms			not found
How long large new loads wait for power little published	in part [65]		
Uncertainty at a commit point "2x difference in the machine count forecasts"	in part observational [8]		
Coefficient drift; how often recalibrated			not found
Declarations' calibration by stage; slip rate			not found
Nearest results on the mechanisms parts safety stock ≈ 0.6× [8]; planning 3–4 months → about 1 [34]	published [8, 34]		
The cover point rising with lead time	in part slow end [8]	not a published result	
The thirteen mechanisms, joined	in part pieces [8, 31, 34]	argued from practice	
Commit-point method; Section 9.2's tests		untested	tests not run
Mechanisms for arbitrating accelerators, internal vs external demand one preprint covers internal teams only [18]; operators state splits and priority orders, not mechanisms [90, 102]			not found

"Not found" means in the sources the author could read, including Site Reliability Engineering, 2nd ed., capacity chapter [7].

Figure 48. What rests on published evidence, what this paper argues, and what nobody has measured in public, as far as the sources show. The method itself is untested.

14. A minimum viable discipline

In short: six roles, and the promises each one keeps.

- **Platform leaders** publish each material coefficient and rate with an owner and a change log, and own the unused and unallocated lines.
- **Planning** keeps the fulfillment ledger, carries the gap, and walks each step change through the commit-point method (Section 8). It also brokers between capital, supply and demand: the same picture for everyone, numbers it doesn't pad, and a no that comes with a path.
- **Finance** declares what "fully loaded" contains. It agrees in advance the landing rule, the shortage-to-idle ratios and the re-cut order. It also agrees, in advance, who pays when a shared service's own coefficient regresses, and who

pays for a change outside the agreed change bands.

- **Product leaders** confirm the pre-filled intake (last year's ask beside its actual), sign what they change, and declare step changes in stages, slow attributes first.
- **A standing forum** owns the ranked list and the cut line.
- **Supply chain** answers every approved ask with a dated commit or a named gap, and sets change bands with planning. It declares its own step changes in stages (Section 6.7), and keeps the commit-point map with planning. It's scored on promised against actual, as demand is scored on how much it changed inside supply's lead time.

Figure 49 puts the six roles around the formula they keep true.

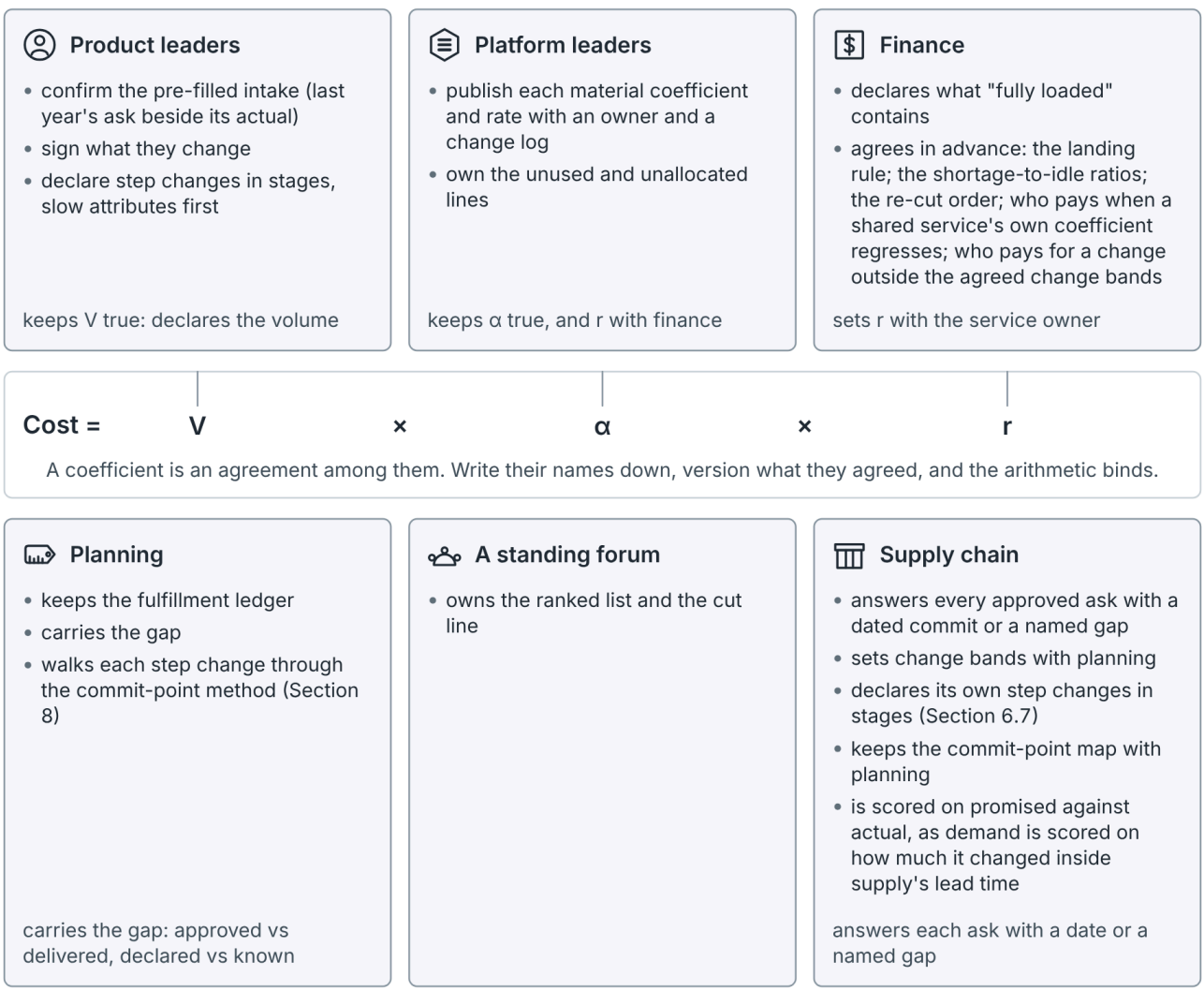


Figure 49. A minimum viable discipline: six roles, each keeping one part of the agreement true.

The formula for a cost driver fits on one line, and large operators have published it. Whether it binds depends on who keeps each term true. That's the team that declares the volume, the service that owns the coefficient, and the service owner who sets the rate with finance. It's also the supply team that answers each ask with a date or a named gap. And it's the planner who carries the gap between approved and delivered, and between declared and known.

A coefficient is an agreement among them. Capital, supply and demand each hold half of someone else's promise, and the planner's job is to make those halves meet. Write their names down, version what they agreed, and the arithmetic binds.

Terms used in this paper

Terms defined where they first appear are not repeated here.

- **Fulfillment**: carrying approved demand through allocation, delivery and turn-up to use (Figure 15, steps 3–6).
- **Apportionment**: spreading cost; where a cited source means this, it's written *cost allocation*.
- **Commit, commit point**: a supply order or promise, and the last date it can be placed; not the capacity reservation, temporary or static, that the SRE chapter calls a *commitment* [7].
- **Unallocated line**: unattributable cost, apportioned by declared policy (the cost-tool sense, not the physical one). Like the unused line, it is owned (mechanism 4).
- **Cut line**: divides a ranked list of asks into funded and unfunded.
- **Forum**: the standing executive body that owns the ranked list and the cut line (mechanism 10).

Appendix A. A worksheet for one declaration

A.0 How to use this appendix

What this is. A worksheet for walking one launch through the commit-point method, and the same launch walked through it with invented numbers.

Who it's for. The planner who owns the declaration, with its supply and finance counterparts and the declaring team's accountable lead.

How to use it. Copy the two-page form (Figure A.2) once per declaration. Fill Part A to read the situation, layer by layer, then Part B to record the decisions and supply's answer, then the footer's change bands. One row per supply layer. The walk-through (A.2–A.5) shows one filled in. Public lead times to sanity-check step 1 are in Appendix B; use them to test your own numbers, not to replace them.

What it isn't. A way to think a launch through, not a model; its numbers are illustrative and the method is untested. The paper's Section 8 gives the reasoning behind each step.

Figure A.1 keys the symbols on the form and in this appendix's figures.

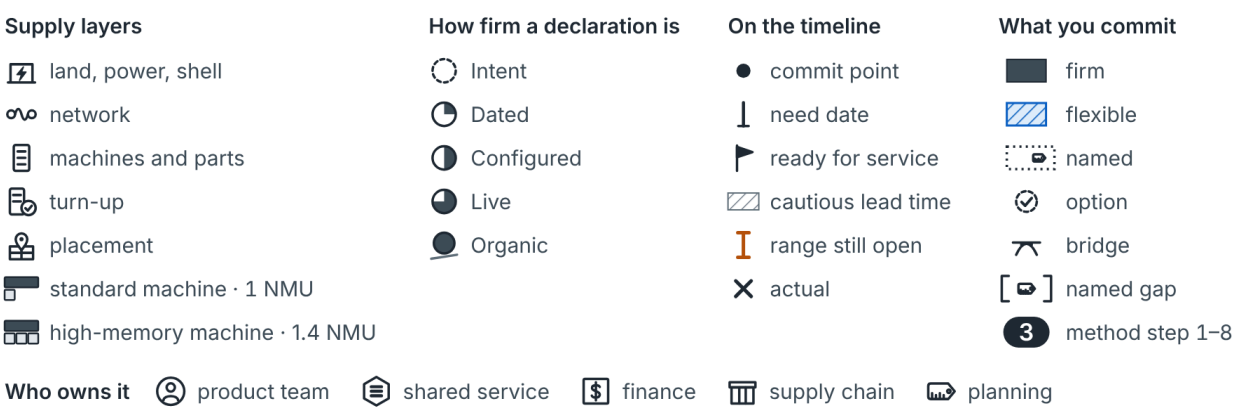


Figure A.1. The symbols used in this appendix.

Terms. One line each, in the paper's words.

- **Declaration:** a signed statement of expected demand for a launch with no history, in units the planner already prices. It firms up in stages.
- **Stage:** how firm a declaration is. *Intent:* a step is coming, with a size range and a half-year or year. *Dated:* a quarter, base and high scenarios, and the attributes that drive slow supply. *Configured:* regions, shapes, ramp and peak shape. *Live:* real traffic in a first phase. *Organic:* the step has become part of the trend, and the declaration closes.
- **A stage's record:** how past declarations made at that stage turned out. The ranges below come from it, not only from the team's word.
- **Supply layer:** one kind of supply with its own lead time: power and positions (places for machines in a hall), network, each machine shape, turn-up, job placement and admission.

- **Usual and cautious lead time:** how long a layer usually takes from order to arrival, and a slow case near the slow end of what's been seen. For example, if machines usually take one quarter but one order in five takes two, plan on two.
- **Turn-up:** installing and testing delivered machines so they can go live. It's a rate, set by crews, test positions and ports, not an order.
- **Ready-for-service date:** the date capacity must be live, ahead of launch by the testing the launch needs.
- **Need date:** the date a layer must be in place: the start of the layer after it, not launch day.
- **Commit point:** the last date a layer's order can still be placed: its need date minus its cautious lead time.
- **Open range (low / base / high):** how far the declaration can still move at a commit point. Here it is ten equally likely outcomes, and the plan carries the fifth-lowest as the base.
- **Driving attribute:** what sizes a layer (region, redundancy tier, hardware class and power density, data stored). Memory drives a high-memory order; region drives power and network.
- **The three choices,** when a commit point comes before the event that would size its layer, are these. 1, ask for the decision earlier; 2, buy an option that carries the commitment past the event; 3, commit to the stage's record and name the owner of the residual.
- **Decision note:** the record of which choice was made: what was known, when, by whom, and what would have changed it.
- **Shortage-to-idle ratio:** the cost of running short against the cost of holding spare. A ratio of k : 1 covers the fraction $k/(k + 1)$ of the range. Illustratively, for persistent demand in a growing business, the ratio is 1 : 1 for turn-up (the median; for turn-up it sets how much rate to staff). The same illustration gives 4 : 1 for machines (0.8, the eighth-lowest of ten), and 19 : 1 for power and shell (0.95).
- **Cover-to point:** the point in the open range a layer is bought up to, set by its ratio.
- **On hand, on order, due back, retiring, net need:** the net need is the target, minus what's on hand, on order or due back from reclaim. Then add capacity due to retire that current load depends on. Each of those is a forecast, with a date and an owner.
- **Firm, flexible, named:** the three slices of each layer. *Firm* covers up to the base, as a plain order. *Flexible* covers from the base up to the cover-to point, as options, cancelable or reschedulable orders, or a named launch reserve. *Named* covers the rest, as a bridge with an owner and a ready-by date and, at the very top, admission control.
- **Option:** a small payment now for the right, but not the duty, to buy later. For most hardware it's a cancelable or reschedulable order.
- **Launch reserve:** capacity held against a declared high scenario.
- **Stocked pool:** ready machines held in stock for many tenants. A draw on it leaves a replenishment owed, with a date and an owner.
- **Bridge:** a stopgap for approved demand that can't be sourced in time, such as machines drawn from the pool or older machines kept past decommission. It works only if arranged before it's needed, and it carries a premium.
- **Admission control:** limiting load at the very top of a peak; the last line.
- **Named gap:** a shortfall named in supply's answer: how much, by when, why, and a proposed bridge with its owner.
- **Change band:** how far size and date may move at each stage. A move inside the band is the plan working; one outside it becomes a priced event recorded against the declaration. Supply signs the same bands.
- **Supply's answer:** for each line, a *dated commit* (quantity, date, confidence, any substituted shape) or a *named gap*.
- **Pegging:** linking every order to the declaration it was placed for. Then a late delivery can be traced to the launches it hits, and a slipped launch's supply handed to another (*re-pegging*).
- **Envelope:** the capacity budget for a planning horizon. A layer committed before any declaration exists is sized on the portfolio envelope, and the launch's claim on it is recorded.

- **Declaration register, commit-point map, fulfillment ledger:** where the walk writes its answers. The register holds each declaration's owner, stage, ranges, bands and shapes. The map holds each layer's lead times, need date, commit point and instrument. The ledger holds each ask's path from approved to net, committed, allocated, delivered, live and adopted.
- **Coefficient (α):** resource used per unit of driver; here, CPU time per request.
- **Persistent and transient:** demand that lasts years, and demand that lasts days. A launch can be both, a spike on a step.
- **rps:** requests per second, the case's unit of volume; the tables give thousands.
- **Reclaim (due back):** capacity taken back from a tenant that no longer needs it, such as another tenant's cleanup. Until it arrives it is a forecast, with a date and an owner.
- **Load tests and canary:** the checks before launch; a canary is a small first release to real traffic.
- **Turn-up slot:** one unit of the staffed turn-up rate, booked ahead like an appointment.
- **Priced event:** a change outside the band, recorded against the declaration and charged under the rule agreed with finance.
- **NMU (normalized machine unit):** the walk-through's invented common unit. A standard machine of 50 cores and 200 GB is 1 NMU; a high-memory machine of 50 cores and 600 GB is 1.4.

A.1 The worksheet

Figure A.2 is the worksheet as a printable form, on two pages. Page 1 has the header, Part A and the footer; page 2 has Part B and the method's eight steps.

Worksheet for one declaration copy one per declaration · page 1 of 2

Declaration	Launch date; ready-for-service date
Declaring team and accountable lead	Stage dates (intent, dated, configured, live)
Persistent, transient, or both (two shapes)	Planner; supply counterpart; finance counterpart
Shared drivers with other declarations; shared supply dependencies	Shortfall tolerance: none / hours / days; how much
Public values to sanity-check step 1: Appendix B.1	

Part A. Read one row per supply layer

	step 1			step 2		step 3		
Layer	**Usual lead time**	**Cautious lead time**	**Source, or "illustrative"**	**Need date**	**Commit point**	**Stage at commit point**	**Open range (low / base / high)**	**Driving attribute known? If not, choice 1, 2 or 3**
Land, power, shell								
Network								
Machines (shape ...)								
Machines (shape ...)								
Turn-up								
Placement								

Footer. Bands step 7

Stage move	Size band	Date band	Signed by
Intent → dated			
Dated → configured			
Configured → live			

Choices, when the commit point comes first
(Section 6.5); write down which you chose

Choice 1 Ask for the decision earlier.

Choice 2 Buy an option that carries the commitment past the event.

Choice 3 Commit to the stage's record and name the owner of the residual.

Figure A.2 (page 1 of 2). The worksheet for one declaration: copy one per launch. The header names the declaration and its counterparts; Part A reads the situation layer by layer; the footer records the change bands.

Part B. Decide one row per supply layer, same rows as Part A · page 2 of 2

	step 4			step 5: net					step 6			step 8
Layer	Shortage : idle	Cover to	Gross target	On hand	On order	Due back (date, owner)	Retiring (date)	Net need	Firm	Flexible (instrument)	Named (bridge, owner, ready-by)	Supply answer: commit or gap
Land, power, shell												
Network												
Machines (shape ...)												
Machines (shape ...)												
Turn-up												
Placement												

The method, step by step (Section 8.2)

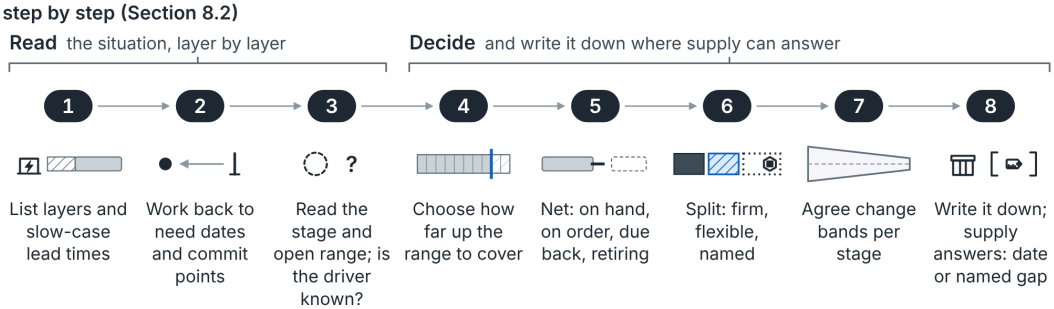


Figure A.2 (page 2 of 2). Part B of the worksheet records the decisions and supply's answer, one row per supply layer; the strip below is the method it follows.

A.2 The case

All numbers from here on are illustrative unless cited. Lead times per layer are hypothetical.

Service X is a shared document store, and Product A is its largest caller. A's peak rises from 400,000 to 500,000 requests per second: 40,000 of organic growth, and 60,000 from a launch planned for the start of the second quarter. Every 1,000 requests per second keeps one core busy, and installed cores are twice busy ones. High-memory machines usually take two quarters to arrive; standard machines come from X's stocked pool in weeks. Without the method, the high-memory commit point was −6, when the launch was dated, and nobody asked about stored data. The memory need surfaced at −3, at configuration. The high-memory machines were ordered then and arrived at +3, a quarter after launch. In between, standard machines from the pool ran for their memory as a bridge. Figure A.3 puts the whole case on one card (the paper's Section 7 tells how this launch went wrong).

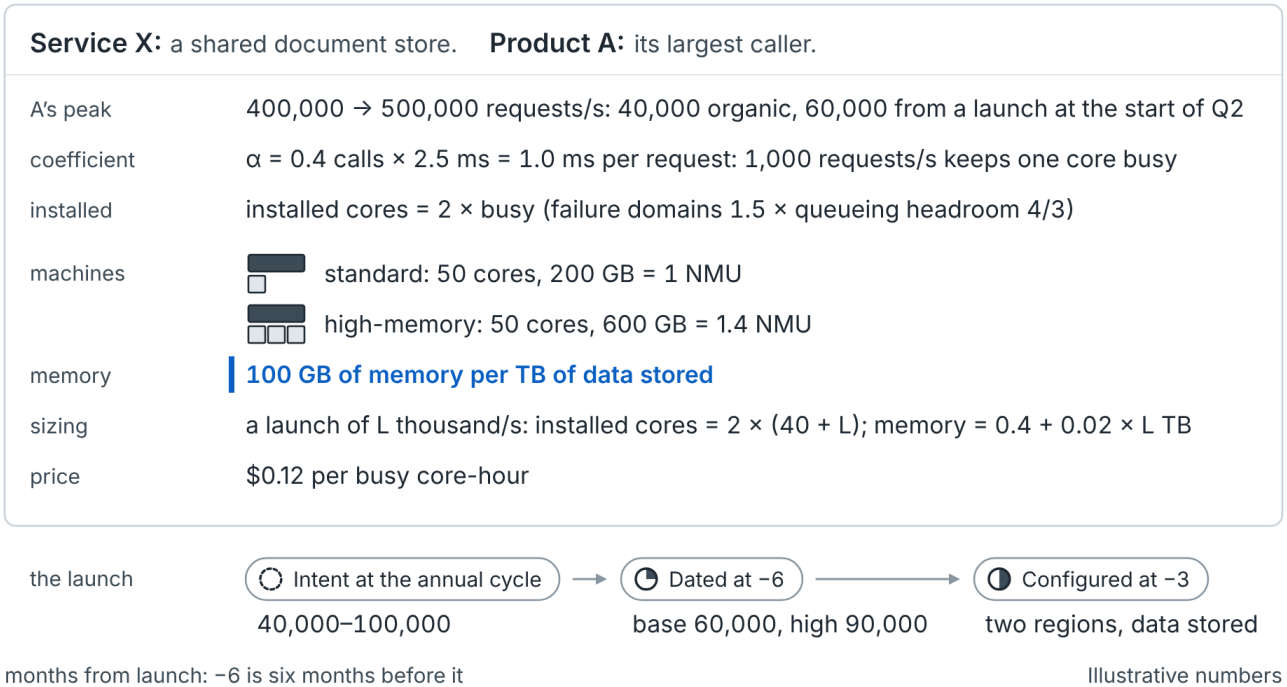


Figure A.3. The case: one launch on one shared service. Everything the walk-through needs is on this card. Illustrative numbers.

One added assumption, to make memory scale. In the case, A's data in X grows 16 TB, "most of it from the launch." Say 4 TB is organic and 12 TB is the launch's at its base of 60,000, scaling at 0.2 TB per 1,000 requests per second. At a launch volume of L thousand requests per second:

- installed cores = $2 \times (40 + L)$;
- memory = 0.4 TB + 0.02 TB × L.

At $L = 60$ that gives 200 cores and 1.6 TB, the need at the base. Sizing rule: the fewest machines that cover the cores, then the fewest high-memory machines among them that cover the memory.

Launch volume (thousand rps)	Installed cores	Memory (TB)	Machines	High-memory	Standard
40	160	1.2	4	1	3
60	200	1.6	4	2	2
70	220	1.8	5	2	3
85	250	2.1	5	3	2
90	260	2.2	6	3	3
100	280	2.4	6	3	3

To check one row: at 70, five machines with two high-memory give 250 cores against 220, and 1.2 + 0.6 = 1.8 TB against 1.8.

A.3 The stage ranges

Ten equally likely outcomes per stage, in thousand requests per second, as the stage's record might give them:

Stage (months from launch)	Ten outcomes	Fifth-lowest	Eighth-lowest	Highest
Intent (from −12)	20, 30, 40, 50, 60, 65, 75, 85, 95, 100	60	85	100
Dated (from −6)	35, 40, 45, 55, 60, 65, 70, 80, 85, 90	60	80	90
Configured (from −3)	40, 45, 50, 55, 60, 60, 65, 70, 80, 90	60	70	90

Drawn from each stage's record, they run wider at the bottom than the team's declared ranges. Each fifth-lowest is the 60,000 carried in the plan, and the actual 40,000 sits inside every list. Figure A.4 draws the three lists.

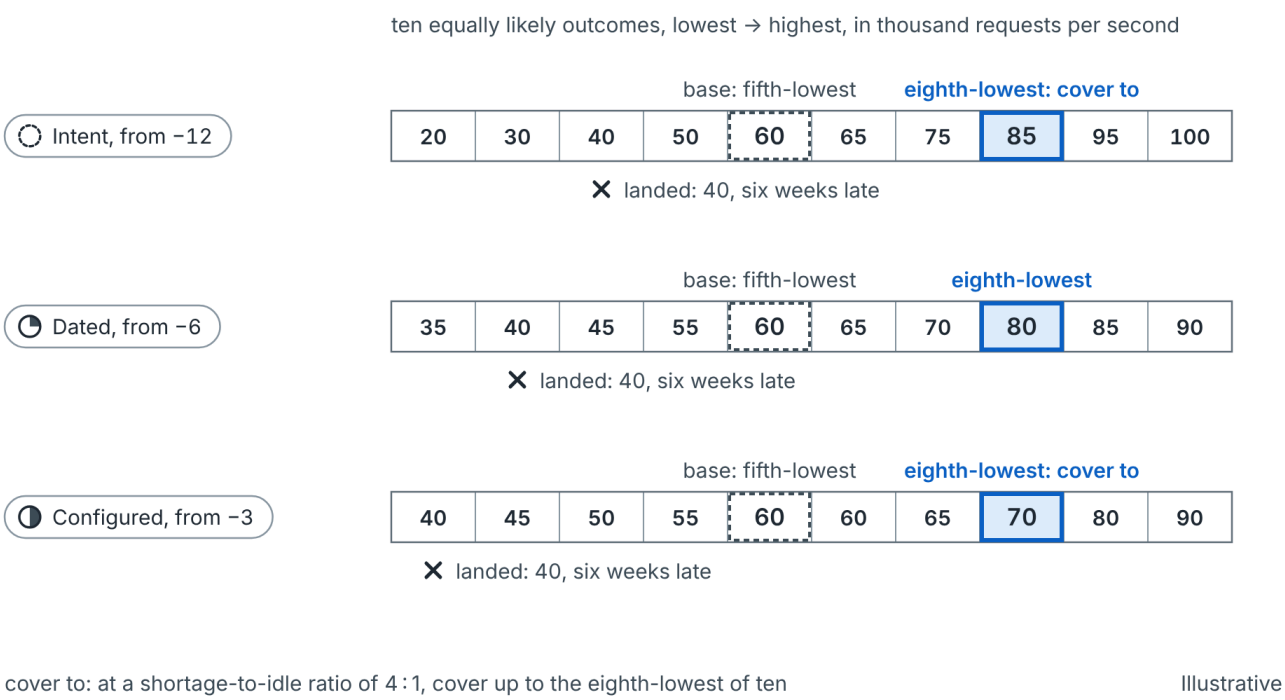


Figure A.4. Ten equally likely outcomes at each stage, from the stage's record. The base stays at 60,000 while the eighth-lowest falls from 85,000 to 70,000; the landing at 40,000 sits inside all three. Illustrative.

A.4 The walk, step by step

Step 1. Layers and lead times

Layer	Usual	Cautious	Basis
Power and positions in X's existing hall	4 quarters	6 quarters	Illustrative; adding positions inside an existing building is far faster than a new building or grid connection (Appendix B).

Layer	Usual	Cautious	Basis
High-memory machines	2 quarters (the case)	3 quarters	Illustrative: for example, if two quarters is usual and one order in five takes three.
Standard machines from X's stocked pool	2 weeks	4 weeks	Illustrative; the case says "in weeks."
Turn-up	2 weeks	1 month	Illustrative; no public value found.
Job placement and admission	Minutes to days	Days	Illustrative.

The configured launch runs in two regions, so a cross-region network layer belongs here too; the case doesn't size it, so it's left off.

Step 2. Need dates and commit points

Work back from launch (month 0):

- Ready for service at **−1**: one month of load tests and canary before launch.
- Turn-up must be done by −1. Its cautious lead time is one month, so machines must be on the dock by **−2**.
- High-memory machines: need date −2, cautious lead time 9 months, so the commit point is **−11**.
- Standard machines: need date −2, cautious lead time 1 month, so the commit point is **−3**.
- Power and positions: need date −2 (the machines need somewhere to go), cautious lead time 18 months, so the commit point is **−20**.

Without the method, the high-memory commit point was −6, at Dated: the usual lead time, counted back from launch. The memory question arrived at −3, the order went in then, and the machines landed at +3. The cautious lead time adds 3 months and the real need date 2 more: −6 − 3 − 2 = −11. Figure A.5 draws the subtraction.

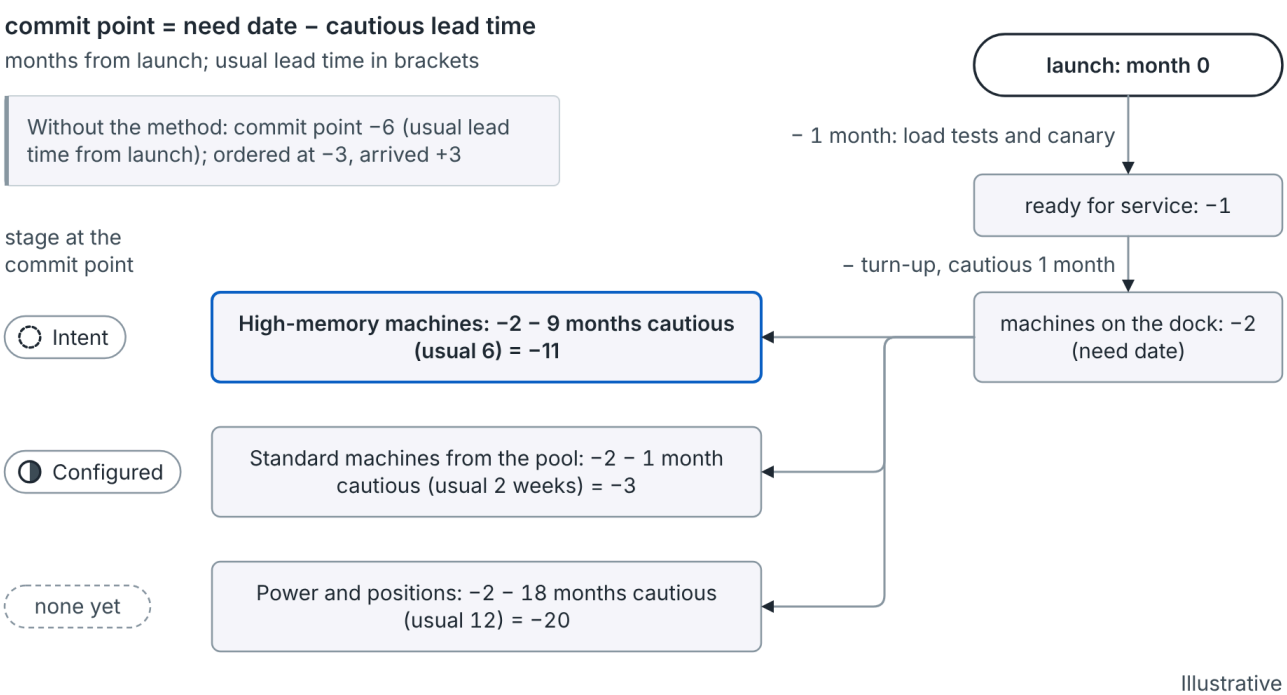


Figure A.5. Step 2 as arithmetic: each layer's need date is the next layer's start, and its commit point is that date minus a cautious lead time. High-memory machines move from −6 to −11, while the launch is still an intent. Illustrative.

Step 3. Stage and range at each commit point

Layer	Commit point	Stage then	Range (low / base / high, thousand rps)	Driving attribute	Known?
Power and positions	−20	None yet	Portfolio envelope	Positions for X's growth	Portfolio record only
High-memory machines	−11	Intent	20 / 60 / 100	Stored data	**No**: the intake asked only about CPU
Standard machines	−3	Configured	40 / 60 / 90	Request volume, regions	Yes
Turn-up	−2 (slots)	Configured	40 / 60 / 90	Machine count	Yes
Job placement and admission	0 onward	Live	Actuals	Actual traffic	Yes

The high-memory row is the miss: the stored-data question arrived at −3. Here it is caught at −11 instead. The planner takes choice 1, asking for stored data at intent as a range. Product A answers 8 to 20 TB, which fits the 0.2 TB per 1,000 requests per second assumed above (12 TB at the base). For the top of the range it also takes choice 2, an option (step 6).

Power and positions commit at −20, before the launch exists, sized for the portfolio. When the intent arrives at −12, the planner checks that it fits inside the envelope.

Step 4. How far up the range

Each layer's shortage-to-idle ratio sets its cover-to point: 19 : 1 covers the top of the ten outcomes (0.95), 4 : 1 the eighth-lowest (0.8) and 1 : 1 the median (0.5).

Layer	Shortage-to-idle ratio (illustrative)	Cover to	Point in range	Target in layer units
Power and positions	19 : 1	Highest of intent	100	6 positions, plus 1 held for the bridge in step 6 = **7**
High-memory machines	4 : 1	Eighth-lowest of intent	85	**3** high-memory
Standard machines	4 : 1	Eighth-lowest of configured	70	5 machines in all; with 2 high-memory on hand (step 5), **3** standard
Turn-up	1 : 1 (on the rate)	Median of configured	60	4 slots on the staffed rate, more on standby

Three high-memory machines also cover the top of the intent range (100 needs 3), so here the 80th percentile covers almost everything.

Step 5. Net

Layer	Target	On hand	On order	Due back	Retiring	Net need
Power and positions	7	8 free in X's hall	0	0	0	**0** to order; claim 7 of the 8

Layer	Target	On hand	On order	Due back	Retiring	Net need
High-memory machines	3	0	0	1, from another tenant's cleanup, due −4, owned by X	2 older ones due to retire at −1 (not counted; see the bridge)	**2**
Standard machines	3	0 spare in X; the stocked pool holds 8	0	0	0	**3**, drawn from the pool

The reclaim at −4 is itself a forecast, owned by X; if it fails, the layer is one machine short, and that's not a supply failure. Drawing 3 from the pool leaves 5, and the pool is owed a replenishment of 3 by, say, +3, owned by whoever runs it.

Step 6. Firm, flexible and named

Layer	Firm	Flexible	Named
Power and positions	7 positions claimed on the envelope	—	— (the bridge's position is inside the 7)
High-memory machines	1 ordered at −11 (the base needs 2; the reclaim supplies the other)	1 ordered at −11, reschedulable: it can be pushed out or re-pegged to another X tenant until −3	—
Standard machines	2 from the pool at −3 (the base)	1 from the pool as a named launch reserve, pegged to A (base 60 → 70)	Above 70 up to 90: a bridge ready by launch, owned by X: 2 more standard machines from the pool run for their memory, or 1 older high-memory machine kept past decommission if incoming hardware doesn't need its positions
Turn-up	4 slots booked	1 slot reserved	Up to 2 standby slots for the bridge
Job placement and admission	—	—	Admission control on launch day for the transient spike, owned by A with X

To check the bridge at 90: with two high-memory machines in hand, memory needs 2.2 TB. That's 1.2 from the pair plus 1.0 from five standard machines: 7 machines and 350 cores against 260. The alternative, three high-memory and three standard, gives 1.8 + 0.6 = 2.4 TB and 300 cores.

On the plan as first written, X would hold 3 high-memory and 3 standard machines for A's growth (7.2 NMU). If configuration brings the eighth-lowest to 70 (below), the second high-memory order is pushed out. By launch, X holds 3 standard and 2 high-memory: 3 + 2 × 1.4 = **5.8 NMU**. That compares with 4.8 NMU for the base alone (2 standard and 2 high-memory machines, from the sizing table). The extra 1.0 NMU is the price of covering the 80th percentile instead of the median.

Step 7. Bands

Agreed with Product A at intent:

Stage move	Size band	Date band	Outcome in this launch
Intent → dated	±50% of 60 (30–90)	±2 quarters	Dated base 60: inside
Dated → configured	±25% of 60 (45–75)	±1 month	Configured 60: inside

Stage move	Size band	Date band	Outcome in this launch
Configured → live	±10% of 60 (54–66)	±2 weeks	Live 40, six weeks late: **outside on both**

Both misses are now recorded against A's declaration, not silent costs. Who pays still follows the rule agreed with finance in advance (the paper's Sections 7 and 14).

Step 8. Artifacts and supply's answer

- **Declaration register:** A's launch, with the three ranges, the stored-data range, the bands, and "persistent, with a launch-day spike."
- **Commit-point map lines:** the five layers above, with commit points −20, −11, −3, −2 and 0.
- **Supply's answer** (illustrative): a dated commit or a named gap for each line of step 6. It carries the dates and owners step 5 set (the reclaim due −4, the pool's replenishment by +3).
- **Fulfillment ledger:** gross 6 machines (7.2 NMU) as planned, 5 (5.8 NMU) after the push-out at −3. Net 5 new as planned, 4 after the push-out, once the reclaimed high-memory machine is counted. Committed to match, plus the reclaim (owned by X); then delivered, live and adopted as they happen.

A.5 What happens when the launch lands

At configuration (−3), the eighth-lowest is 70, which needs only 2 high-memory machines, so the planner pushes the second high-memory order out and re-pegs it to another X tenant. The reclaim arrives on time. The launch goes live six weeks late, at 40,000.

At 40,000 and the planned coefficient, A's growth needs 160 installed cores (4 machines). X holds 5 machines, 250 cores, so 90 installed cores sit idle: 90 × $0.06 × 8,760 hours ≈ **$47,304 a year**. That's at $0.06 per installed core-hour (the $0.12 busy-core rate spread over twice as many installed cores). Without the method, X would hold 4 machines, 200 cores, and 40 idle installed cores would come to $21,024, so the method's extra machine accounts for about $26,280. During the six-week slip, the launch reserve could be re-pegged to another tenant. After it, the spare waits for growth inside the 4:1 ratio for persistent demand, and the band records whose declaration it waited on. (In the paper's Section 7, a release of X also raises CPU per request that year, so X needs more cores than planned. That drift would have used the spare anyway. That's luck, not the method.)

Figure A.6 shows the fulfillment ledger line the walk writes, as planned and as landed.

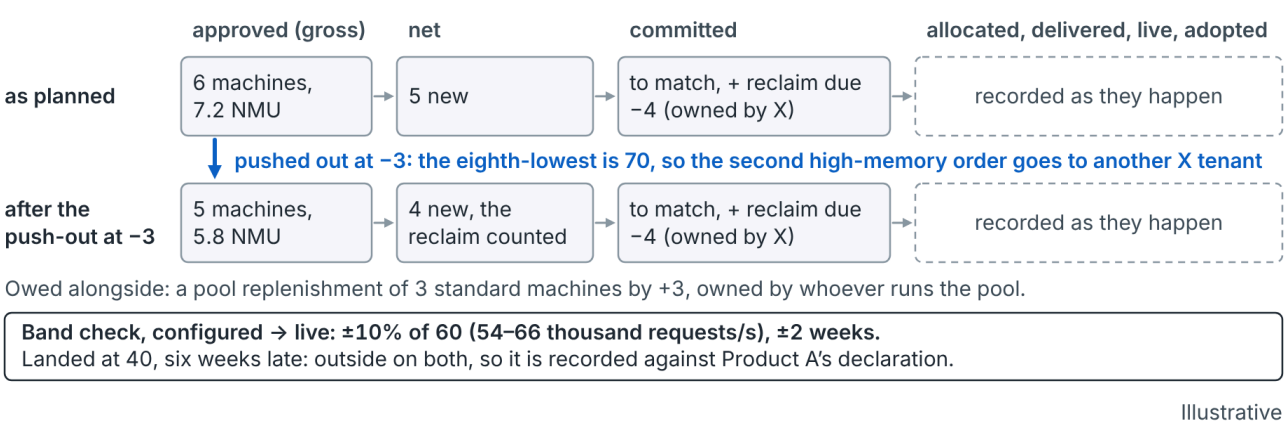


Figure A.6. The fulfillment ledger line the walk writes, before and after the second high-memory order was pushed out, and the band check when the launch landed. Illustrative.

A.6 What the walk shows, and what it doesn’t

The walk moves the stored-data question ahead of the high-memory commit point: asked at −12, not −3. It gives every surprise an owner: the reclaim, the pool’s replenishment, the bridge above 70,000 and the band breach at landing. It costs about 1.0 NMU held above the base, about $26,280 a year of idle cores at the 40,000 landing. It isn’t a model, and it is untested (the paper’s Sections 8.6 and 13).

Appendix B. What the public record says about supply

B.0 How to read this appendix

What this is. Public values for how long supply takes and how it is contracted: lead times by layer, waits for power, contract terms and capacity reported by firmness. Each value is dated and comes from its source's own context.

Who it's for. Planners, supply and finance teams checking their own numbers.

How to use it. Test your numbers against these, don't replace them. A value here describes one source's situation at one date. The charts plot only values the sources state. Ranges stay ranges. "More than" stays open; "up to" is a cap at the stated maximum, never a bar from zero; an example stays a single point. Values from different sources are never combined or averaged. Where nothing is published, the appendix says so (B.5), and your own value there is best stated as plainly hypothetical, in the "for example, if…" form. Sections 6.2, 6.4 and 6.7 of the paper use these values.

Figure B.1 keys the marks the charts use.

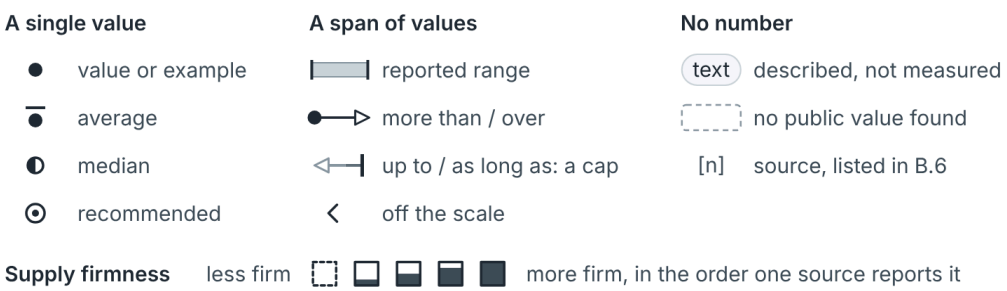


Figure B.1. How the charts in this appendix mark each kind of public value.

Terms. One line each, in the paper's words.

- **Lead time:** how long a supply layer takes from order to arrival. A cautious lead time sits near the slow end of what's been seen.
- **Supply layer:** one kind of supply with its own lead time: land, power and buildings; network; machines and parts; turn-up; placement.
- **Turn-up:** installing and testing delivered machines so they can go live.
- **Grid connection for a large load, versus interconnection of new generation:** a large new load, such as a data center, connects through the utility's own process. New generation and storage join the grid through a separate interconnection queue. Their waits are different measures.
- **Power transformer, generator step-up transformer, switchgear:** three kinds of large electrical gear, named as their source names them [61].
- **Take-or-pay:** a commitment owed whether the demand behind it arrives or not.
- **Non-cancellable order:** an order that can't be walked away from once placed.

- **Cancel window:** the last date you can still walk away from an order cheaply. Once it closes, the forecast behind the order is a liability.
- **Minimum billing demand:** a tariff's floor on the bill, set as a share of contract capacity.
- **Electric service agreement; construction and engineering letters of authorization:** the tiers under which Dominion Energy reports its data-center contract capacity, by firmness, in its own order [92]. By its account, an engineering letter pays for the utility's engineering studies and doesn't oblige the customer to proceed. A construction letter commits the customer to repay the utility's investment, and the utility reserves capacity for it. The electric service agreement is the legal document that sets the contracted capacity, and service needs one [92].
- **PJM:** the regional grid operator to which Dominion Energy submitted the documentation cited in B.4 [92].
- **Primary data center markets:** the established markets JLL's outlook groups as primary; its wait figure covers those markets, not every location [60].
- **Residual value guarantee:** a lessee's promise to make up any shortfall if the leased asset is worth less than an agreed amount at the end of the lease.
- **Contracted versus active power:** power an operator has signed for, against power it reports as active. The paper asks each power figure to say which power it means: contracted, provisioned or drawn.
- **Bridge:** a stopgap for demand that supply can't meet in time. For power, it means temporary on-site generation.
- **Firmness:** how far supply has moved toward use. Supply firms up in stages of its own: planned, contracted, under construction, delivered, live.

B.1 Lead times

Layer	Public value	Source
Your own buildings	Could be "five years" (an example) if you erect your own buildings; "10 minutes" for a small cloud service	Hixson and Guliani, 2015 [6]
Land, utility, buildings	"planned years in advance"	Olavson, 2019 [8]
Grid connection, primary markets	JLL puts "the average wait time for a grid connection in primary data center markets" at more than four years	JLL, January 2026 [60]
Power transformers; generator step-up transformers; switchgear	128 weeks, 143 weeks and 44 weeks on average, in the second quarter of 2025	Wood Mackenzie and American Clean Power [61]
High-capacity transformers	"as long as four years"	[62]
Backbone network	"months or even years"	Ahuja and colleagues, 2021 [63]
Components	"over 6 months for some components"	Olavson, 2019 [8]
Accelerators (the vendor's own supply)	"more than 12 months"	NVIDIA, 2025 [64]
Cloud capacity reservations	Requested 5 to 120 days ahead; 56 recommended	AWS documentation [109]
Turn-up	No public value found	— (B.5)

Figure B.2 plots the table, each value as its source states it; weeks convert at 7 days and months at 30.4.

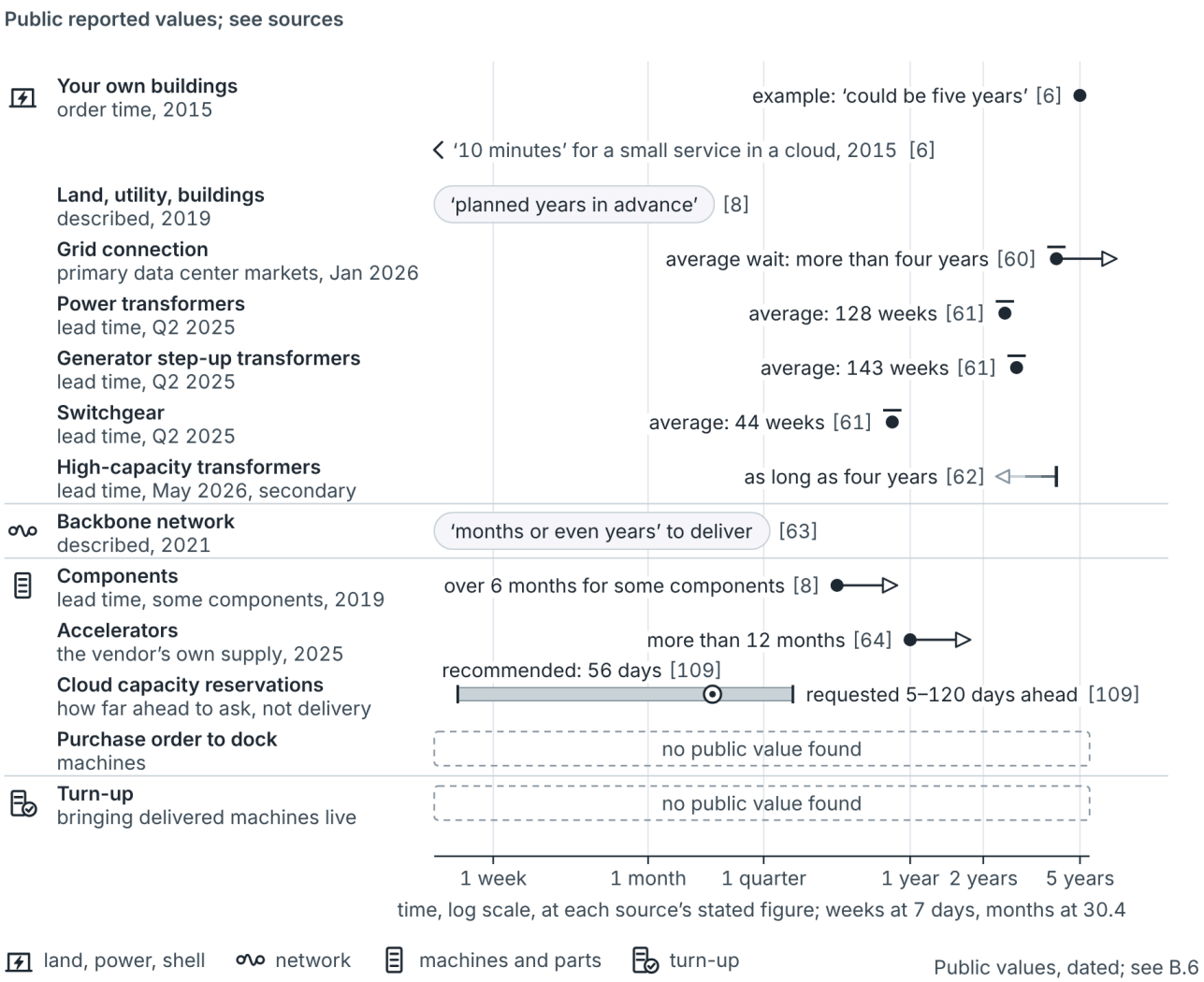


Figure B.2. Public reported lead times by supply layer, each from one source at one date; see sources. Ranges stay ranges, "more than" stays open and "as long as" is a cap, not a range; an example is a single dot. No two sources are combined. Turn-up and order-to-dock times have no public value.

B.2 Power for a large new load

Power's numbers need care. For new *generation*, the median wait from interconnection request to signed agreement was "well over 3 years in 2025" [112], but a large new load goes through the utility's own process. For new loads, one transmission owner cited by Lawrence Berkeley National Laboratory expects studies to take "6 to 18 months," and any construction requiring a line extension "at least 18 months and potentially as long as 60 months"; the same report notes "limited publicly available information on large load interconnection timelines" [65]. Power can be bridged, too: JLL expects natural gas to play a role "both for temporary bridge power and increasingly for permanent on-site power generation" [60]. Figure B.3 keeps the three measures apart.

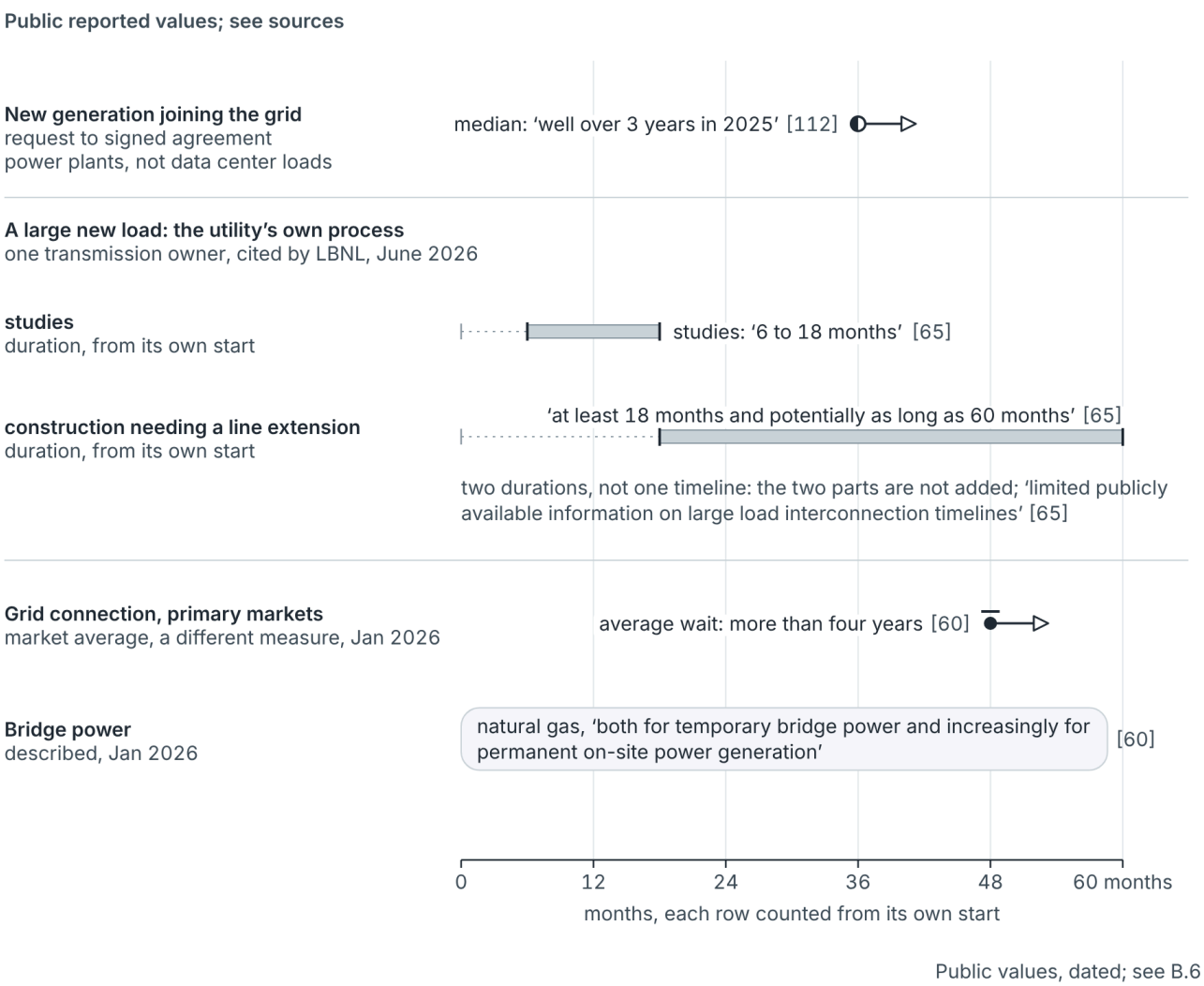


Figure B.3. A large new load waits in the utility's own process, not the queue for new power plants. Public reported values, each in its own source's terms; the two parts of the load path are durations, each from its own start, not one timeline, and are not added. See sources.

B.3 Contracts

The accelerator vendor in B.1 reports having "paid premiums, provided deposits, and entered into long-term supply agreements and capacity commitments," and placing "non-cancellable and non-returnable purchase orders" ahead of its historical lead times [64]. Microsoft, for example, signed a 20-year power purchase agreement tied to restarting a nuclear unit expected online in 2028 when the agreement was announced [113]. AEP Ohio's data center tariff (for loads of 25 MW or more) sets a load ramp of up to four years. In it, the contract capacity in effect steps through no less than 50, 65, 80 and 90% of the full contract capacity in years one to four. Separately, its minimum billing demand cannot exceed 85% of contract capacity, a cap the largest loads reach. The initial term is the ramp plus eight years; after the fifth post-ramp year, a customer may leave by paying "minimum charges for 36 months after notice of termination" [114].

Leases run long, too. Microsoft calls its data center leases "very long-lived assets," at "15 to 20 years," while for short-lived chips "the lifetimes of these and the lifetimes of the contracts are very similar" [72]. Meta leases one campus,

Hyperion, on “a four-year initial term with options to extend,” backed by a residual value guarantee for the first 16 years [75].

The structure runs on to customers: CoreWeave sells capacity “on a take-or-pay basis” under contracts averaging “approximately five years,” and buys from its own suppliers “according to stated lead times” [68]. Oracle states the mismatch risk plainly: if it overestimates demand it “could be locked into multi-year commitments for excess data center space,” and its long-term data center leases “typically do not align with the duration and pricing of customer contracts” [69]. Figure B.4 lines up the commitment lengths, and the tariff’s load ramp on a scale of its own.

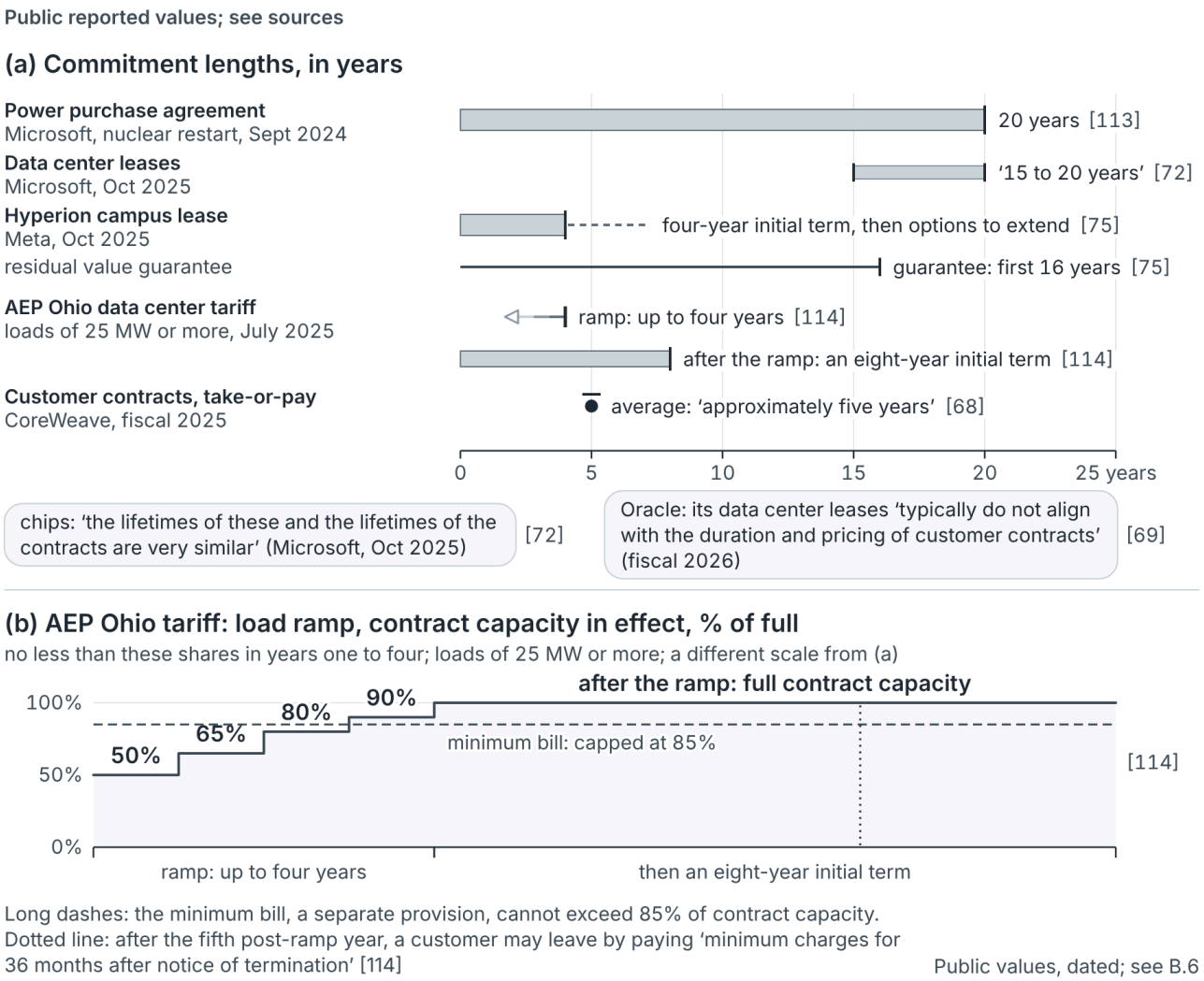


Figure B.4. Supply commitments often run longer than the customer contracts behind them. (a) Commitment lengths, each from its own source; (b) one utility tariff’s load ramp, with its cap on the minimum bill drawn apart. Public reported values; different scales; see sources.

B.4 Supply reported by firmness

Dominion Energy reports the capacity under its data-center contracts by firmness. It has 9.8 GW under electric service agreements and 7.1 GW under construction letters of authorization. Another 30.1 GW sits under engineering letters of authorization, which it labels “potential future load” [92]. On the operator side, CoreWeave reported about 3.1 GW of

contracted power at the end of 2025, against more than 850 MW active [68]. Figure B.5 draws the two sources side by side, each on its own scale.

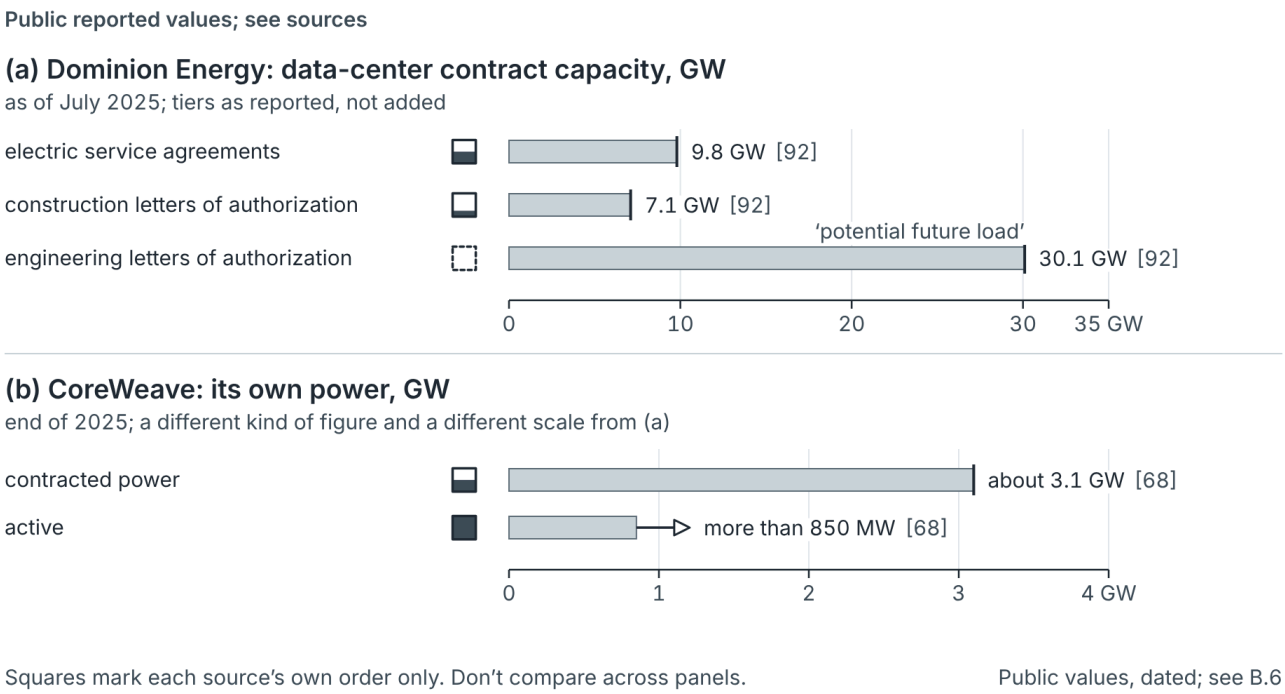


Figure B.5. Power reported by firmness: contracted far ahead of what is under construction or active. Two sources, two kinds of figure, two scales. Public reported values; see sources.

B.5 What isn't public

In the sources read for the paper, these have no public value:

- **Purchase order to dock** for machines: no published time found.
- **Turn-up:** no published duration found. For your own, use a plainly hypothetical value, in the "for example, if…" form.
- **Hardware contract terms** of the cancel-schedule kind (what you owe if you cancel, by date): none found.
- **Waits for power for large new loads:** Lawrence Berkeley National Laboratory notes "limited publicly available information on large load interconnection timelines" [65].

The capacity planning chapter of the second edition of *Site Reliability Engineering* (September 2026) [7] doesn't publish these either. It does name the constraints behind them: power grids, specialized hardware lead times and labor shortages put "a structural ceiling on construction speed" [7].

B.6 Sources for this appendix

Numbered as in the paper's reference list, so the two can be read side by side. Accessed 2026-10-04 unless noted. **[Secondary]** = summary, encyclopedia entry, news report or third-party record used in place of the primary source.

6. Hixson, D., & Guliani, K. "Capacity Planning." *USENIX ;login:* 40(1):32–38, February 2015. https://www.usenix.org/system/files/login/articles/login_feb15_07_hixson.pdf

7. Gordon, A., Kirby, C., Hixson, D., et al. “Capacity Planning” (chapter 15). In Beyer, B., Jones, C., Leng, C., Huska, D., Petoff, J., & Murphy, N. R. (eds.), *Site Reliability Engineering*, 2nd ed. O’Reilly, September 2026. https://www.oreilly.com/library/view/site-reliability-engineering/9798341607675/ch15.html — *Full chapter read in the O’Reilly online edition, 7 October 2026.*

8. Olavson, T. “Humans-in-the-loop forecasting: integrating data science and business planning.” The Unofficial Google Data Science Blog, 4 December 2019. https://www.unofficialgoogledatascience.com/2019/12/humans-in-loop-forecasting-integrating.html (accessed 2026-10-05).

60. JLL. *2026 Global Data Center Outlook*, 5 January 2026. https://www.jll.com/en-us/insights/market-outlook/data-center-outlook (accessed 2026-10-06).

61. Wood Mackenzie & American Clean Power. *Making the Connection: Meeting the Electric T&D Supply Chain Challenge*, executive summary, September 2025. Hosted by POWER magazine: https://www.powermag.com/wp-content/uploads/2026/01/making-the-connection-executive-summary-september-2025.pdf (accessed 2026-10-06; figures are Q2 2025 averages).

62. Kennedy, R. “U.S. transformer market faces severe supply constraints as lead times extend to four years.” pv magazine USA, 11 May 2026 (quoting PwC analysts). https://pv-magazine-usa.com/2026/05/11/u-s-transformer-market-faces-severe-supply-constraints-as-lead-times-extend-to-four-years/ **[Secondary]**

63. Ahuja, S. S., Gupta, V., Dangui, V., Bali, S., Gopalan, A., Zhong, H., Lapukhov, P., Xia, Y., & Zhang, Y. “Capacity-Efficient and Uncertainty-Resilient Backbone Network Planning with Hose.” *Proc. ACM SIGCOMM 2021*, August 2021. https://research.facebook.com/file/311440033651798/Capacity-Efficient-and-Uncertainty-Resilient-Backbone-Network-Planning-with-Hose.pdf (accessed 2026-10-06).

64. NVIDIA Corporation. Annual Report on Form 10-K for the fiscal year ended 26 January 2025 (risk factors on supply). U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/1045810/000104581025000023/nvda-20250126.htm

65. Kahrl, F., & Mims Frick, N. *Speed to Power: Solutions for Accelerating Large Load Connections*. Lawrence Berkeley National Laboratory, June 2026. https://eta-publications.lbl.gov/sites/default/files/2026-06/lbnl_large_loads_speed_to_power_final_1.pdf (accessed 2026-10-06; the 6–18 and 18–60 month figures are the report’s citation of one transmission owner, p. 4, fn. 9).

68. CoreWeave, Inc. Annual Report on Form 10-K for the fiscal year ended 31 December 2025. U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/1769628/000176962826000104/crwv-20251231.htm (accessed 2026-10-06).

69. Oracle Corporation. Annual Report on Form 10-K for the fiscal year ended 31 May 2026 (risk factors). U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/0001341439/000119312526277521/orcl-20260531.htm (accessed 2026-10-06).

72. Microsoft Corp. FY26 First Quarter Earnings Conference Call, 29 October 2025 (speaker: S. Nadella, Chairman and CEO). https://www.microsoft.com/en-us/investor/events/fy-2026/earnings-fy-2026-q1 (transcript: aka.ms/transcriptfy26q1).

75. Meta. “Meta Announces Joint Venture With Funds Managed by Blue Owl Capital to Develop Hyperion Data Center.” Meta Newsroom, 21 October 2025. https://about.fb.com/news/2025/10/meta-blue-owl-capital-develop-hyperion-data-center/ (accessed 2026-10-06).

92. Dominion Energy. Load forecast documentation submitted to PJM, 6 January 2026. https://www.pjm.com/-/media/DotCom/planning/res-adeq/load-forecast/dominion-documentation.pdf (accessed 2026-10-06; figures as of July 2025).

109. Amazon Web Services. “Capacity Reservation concepts” (future-dated Capacity Reservations). Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/cr-concepts.html (accessed 2026-10-06).

112. Lawrence Berkeley National Laboratory. “Backlog of power plants seeking transmission grid connection eased somewhat in 2025 amidst high withdrawals” (*Queued Up* 2026 edition), news release, 1 July 2026. https://emp.lbl.gov/news/backlog-power-plants-seeking-transmission-grid-connection-eased-somewhat-2025-amidst (measures generation and storage joining the grid, not data center loads).

113. Constellation Energy. “Constellation to Launch Crane Clean Energy Center, Restoring Jobs and Carbon-free Power to the Grid.” Press release, 20 September 2024 (20-year power purchase agreement with Microsoft; Three Mile Island Unit 1 expected online in 2028 at announcement). https://www.constellationenergy.com/newsroom/2024/Constellation-to-Launch-Crane-Clean-Energy-Center-Restoring-Jobs-and-Carbon-Free-Power-to-The-Grid.html (accessed 2026-10-10); also via Business Wire: https://www.businesswire.com/news/home/20240920487805/en

114. Ohio Power Company (AEP Ohio). Data Center Tariff (Schedule DCT), effective 23 July 2025; approved by the Public Utilities Commission of Ohio, order of 9 July 2025. https://www.aepohio.com/company/about/rates/data-center-tariff ; reported by POWER: https://www.powermag.com/regulator-approves-aep-ohios-landmark-data-center-tariff/ (accessed 2026-10-06).

About the author. Guillermo Martinez worked in infrastructure capacity planning and resource governance at a hyperscale operator from 2015 to 2024. Views are his own.

Disclaimer. This paper reflects the author’s own views. It is not endorsed by, and does not represent, any current or former employer. It uses no confidential information and describes no organization’s internal practice beyond what the cited public sources state. All examples and figures not attributed to a source are invented for illustration, including the normalized machine unit, its weights and every ratio marked illustrative. Several cited works are by authors at the author’s former employer; they are cited only for what they publish, alongside published work from other operators. Company, product and system names are trademarks of their respective owners, used only to identify published work; no endorsement is implied.

References

Accessed 2026-10-04 unless noted. **[Preprint]** = not peer reviewed. **[Secondary]** = summary, encyclopedia entry, news report or third-party record used in place of the primary source. **[Not re-consulted]** = cited from its verified bibliographic record; text not re-read for this draft.

1. Helstroom, D., Weir, T., Leonard, E., & Delimon, K. “Software Engineering in SRE” (chapter 18, describing Auxon). In Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (eds.), *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly, 2016. https://sre.google/sre-book/software-engineering-in-sre/
2. Eriksen, M., et al. “Global Capacity Management With Flux.” *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’23)*, Boston, MA, 10–12 July 2023. https://www.usenix.org/system/files/osdi23-eriksen.pdf
3. Hax, A. C., & Meal, H. C. “Hierarchical Integration of Production Planning and Scheduling.” In M. A. Geisler (ed.), *Logistics*, TIMS Studies in the Management Sciences, vol. 1, pp. 53–69. Amsterdam: North-Holland, 1975. Earlier version: MIT Sloan School of Management Working Paper 656-73, 1973, https://ideas.repec.org/p/mit/sloanp/1868.html **[Not re-consulted]** (bibliographic record verified via RePEc and a Springer reference list, https://link.springer.com/chapter/10.1007/978-3-642-51693-1_11 ; text not read).
4. Welbourne, T. M., & Gomez-Mejia, L. R. “Gainsharing: A Critical Review and a Future Research Agenda.” *Journal of Management* 21(3):559–609, June 1995. https://asu.elsevierpure.com/en/publications/gainsharing-a-critical-review-and-a-future-research-agenda **[Not re-consulted]** (abstract and record only).
5. TBM Council. “TBM Taxonomy,” v5.0.1, released 18 July 2025. https://www.tbmcouncil.org/taxonomy
6. Hixson, D., & Guliani, K. “Capacity Planning.” *USENIX ;login:* 40(1):32–38, February 2015. https://www.usenix.org/system/files/login/articles/login_feb15_07_hixson.pdf
7. Gordon, A., Kirby, C., Hixson, D., et al. “Capacity Planning” (chapter 15). In Beyer, B., Jones, C., Leng, C., Huska, D., Petoff, J., & Murphy, N. R. (eds.), *Site Reliability Engineering*, 2nd ed. O’Reilly, September 2026. https://www.oreilly.com/library/view/site-reliability-engineering/9798341607675/ch15.html — *Full chapter read in the O’Reilly online edition, 7 October 2026.*
8. Olavson, T. “Humans-in-the-loop forecasting: integrating data science and business planning.” The Unofficial Google Data Science Blog, 4 December 2019. https://www.unofficialgoogledatascience.com/2019/12/humans-in-loop-forecasting-integrating.html (accessed 2026-10-05).
9. Kahneman, D., & Lovallo, D. “Timid Choices and Bold Forecasts: A Cognitive Perspective on Risk Taking.” *Management Science* 39(1):17–31, 1993. https://ideas.repec.org/a/inm/ormnsc/v39y1993i1p17-31.html (abstract consulted, 2026-10-05).
10. Flyvbjerg, B. “Curbing Optimism Bias and Strategic Misrepresentation in Planning: Reference Class Forecasting in Practice.” *European Planning Studies* 16(1):3–21, 2008. https://ideas.repec.org/a/taf/eurpls/v16y2006i1p3-21.html (abstract consulted, 2026-10-05; the RePEc record’s identifier carries 2006, the volume is 2008).
11. “Cost driver.” Wikipedia (definitions of resource and activity drivers). https://en.wikipedia.org/wiki/Cost_driver **[Secondary]** The definition of a cost driver in the text is the author’s paraphrase of standard usage.
12. Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure.” Google Technical Report dapper-2010-1, April 2010. https://static.googleusercontent.com/media/research.google.com/en//archive/papers/dapper-2010-1.pdf
13. Farrow, R. “Capacity Engineering: An Interview with Rick Boone.” *USENIX ;login:* 43(3), Fall 2018. https://www.usenix.org/system/files/login/articles/login_fall18_08_boone.pdf

14. Schroder, C. “Twitter’s Chargeback System Measures Resource Use and Sends Out a Bill.” Linux.com, 7 November 2016, reporting a LinuxCon North America 2016 talk by V. Charanya and M. Benedict. https://www.linux.com/audience/enterprise/twitters-chargeback-system-measures-resource-use-and-sends-out-bill/ **[Secondary]**
15. Sutherland, I. E. “A Futures Market in Computer Time.” *Communications of the ACM* 11(6):449–451, 1968. https://dl.acm.org/doi/10.1145/363347.363396
16. Stokely, M., Winget, J., Keyes, E., Grimes, C., & Yolken, B. “Using a Market Economy to Provision Compute Resources Across Planet-wide Clusters.” *2009 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, Rome, 2009. https://research.google/pubs/using-a-market-economy-to-provision-compute-resources-across-planet-wide-clusters/ ; arXiv copy posted 22 March 2025: https://arxiv.org/abs/2503.17691
17. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., & Wilkes, J. “Large-scale cluster management at Google with Borg.” *Proc. EuroSys ’15*, Bordeaux, April 2015. doi:10.1145/2741948.2741964. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43438.pdf
18. Sivan, B., Paes Leme, R., Tiuca, M., McFarlane, I., Gkatzelis, V., Mehta, N., Hassas Yeganeh, S., Mirrokni, V., & Vahdat, A. “Quota Marketplace: Dynamic Pricing for Efficient Allocation of ML Training Resources.” arXiv:2607.09802v1, 9 July 2026. https://arxiv.org/html/2607.09802 **[Preprint]**
19. IT Process Maps. “Capacity Management (ITIL).” IT Process Wiki. https://wiki.en.it-processmaps.com/index.php/Capacity_Management **[Secondary]** summary of ITIL V3/2011 and ITIL 4.
20. FinOps Foundation. “FinOps Personas.” FinOps Framework. https://www.finops.org/framework/personas/
21. FinOps Foundation. “Allocation” (cost allocation capability). FinOps Framework. https://www.finops.org/framework/capabilities/allocation/
22. FinOps Foundation. “Data Center” (technology category; introduced as a scope in the 2025 Framework update, 20 March 2025). FinOps Framework. https://www.finops.org/framework/technology-categories/data-center/ ; https://finops.org/insights/2025-finops-framework
23. Wallace, T. F., & Stahl, R. A. *Sales & Operations Planning: The How-To Handbook*, 3rd ed. T. F. Wallace & Co., 2008. **[Not re-consulted]**
24. Allspaw, J. *The Art of Capacity Planning*. O’Reilly, 2008; Kejariwal, A., & Allspaw, J. *The Art of Capacity Planning*, 2nd ed. O’Reilly, September 2017. https://www.oreilly.com/library/view/the-art-of/9781491939192/ (scope assessed from the publisher’s table of contents).
25. Cockcroft, A., & Walker, B. *Capacity Planning for Internet Services*. Sun BluePrints. Sun Microsystems Press / Prentice Hall PTR, June 2001. ISBN 978-0-13-089402-1. https://informit.com/store/capacity-planning-for-internet-services-9780130894021 (publisher’s description consulted, 2026-10-07).
26. Gunther, N. J. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Berlin/Heidelberg: Springer, 2007. doi:10.1007/978-3-540-31010-5. https://link.springer.com/book/10.1007/978-3-540-31010-5 (publisher’s description consulted, 2026-10-07).
27. Sonney, K., & Dufresne, M.-A. “Capacity Constraints Unveiled: Navigating Cloud Scaling Realities.” SREcon24 Americas, 18 March 2024. https://www.usenix.org/conference/srecon24americas/presentation/sonney (abstract consulted, 2026-10-07).
28. Han, J., & Phadnis, P. “Cloud Efficiency at Netflix.” Netflix TechBlog, 17 December 2024. https://netflixtechblog.com/cloud-efficiency-at-netflix-f2a142955f83
29. Eriksen, M. “The journey of real-life industry work behind an OSDI paper — global capacity management for millions of servers.” ACM SIGOPS Blog, 5 February 2024. https://www.sigops.org/2024/the-journey-of-real-life-industry-work-behind-an-osdi-paper-global-capacity-management-for-millions-of-servers/ (accessed 2026-10-07).
30. Quesada Torres, L., & Colish, D. “SRE Best Practices for Capacity Management.” *USENIX ;login:* 45(4), Winter 2020. https://sre.google/static/pdf/login_winter20_10_torres.pdf (accessed 2026-10-07).

31. Rai, R., Ho, G., & Huang, F. “Meta Capacity Planning for AI Training Hardware.” Talk at AI Infra @Scale 2024, 22 August 2024. Talk page and summary: https://atscaleconference.com/meta-capacity-planning-for-ai-training-hardware/ (accessed 2026-10-05).
32. Vora, M., Deek, L., & Livengood, E. “Content Popularity for Open Connect.” Netflix TechBlog, 20 June 2017. https://netflixtechblog.com/content-popularity-for-open-connect-b86d56f613b (accessed 2026-10-05).
33. Jiang, J. “Capacity Planning for Alibaba’s Double 11 Shopping Festival.” Alibaba Cloud Blog, 9 November 2018. https://www.alibabacloud.com/blog/capacity-planning-for-alibabas-double-11-shopping-festival_594164 (accessed 2026-10-05).
34. Gupta, A. “Stop asking engineers how many machines they need.” LeadDev, 16 September 2026. https://leaddev.com/technical-direction/stop-asking-engineers-how-many-machines-they-need (accessed 2026-10-05).
35. “Shadow price.” Wikipedia. https://en.wikipedia.org/wiki/Shadow_price **[Secondary]** (defines the shadow price as a constraint’s dual variable; the zero-for-slack and integer-placement caveats in the text are standard duality results not stated on this page).
36. Hirshleifer, J. “On the Economics of Transfer Pricing.” *Journal of Business* 29(3):172–184, 1956. https://ideas.repec.org/r/ucp/jnlbus/v29y1956p172.html **[Not re-consulted]**
37. Thomas, A. L. *The Allocation Problem: Part Two*. Studies in Accounting Research No. 9, American Accounting Association, 1974. As discussed in Grinyer, J. R., & Kouhy, R. “Matching and Incorrigibility: Reconsideration and Proposals.” *British Accounting Review* 25(4):309–323, December 1993. https://rke.abertay.ac.uk/en/publications/matching-and-incorrigibility-reconsideration-and-proposals/ **[Secondary]**
38. TBM Council. “Allocation Methods” (cost allocation guidance). Web page, updated 4 September 2025. https://www.tbmcouncil.org/learn-tbm/tbm-modeling/allocation-methods/
39. Shapley, L. S. “A Value for n-Person Games.” In H. W. Kuhn & A. W. Tucker (eds.), *Contributions to the Theory of Games, Vol. II*, Annals of Mathematics Studies 28, pp. 307–317. Princeton, NJ: Princeton University Press, 1953. **[Not re-consulted]** (record confirmed via https://mathworld.wolfram.com/ShapleyValue.html).
40. Aumann, R. J., & Shapley, L. S. *Values of Non-Atomic Games*. Princeton, NJ: Princeton University Press, 1974. Library record: https://ci.nii.ac.jp/ncid/BA0299117X **[Not re-consulted]**
41. Billera, L. J., & Heath, D. C. “Allocation of Shared Costs: A Set of Axioms Yielding a Unique Procedure.” *Mathematics of Operations Research* 7(1):32–39, 1982. https://pubsonline.informs.org/doi/10.1287/moor.7.1.32 (abstract consulted).
42. Young, H. P. (ed.). *Cost Allocation: Methods, Principles, Applications*. Amsterdam: North-Holland, 1985. Library record: https://ci.nii.ac.jp/ncid/BA00407282 **[Not re-consulted]**
43. Boiteux, M. “La tarification des demandes en pointe.” *Revue Générale de l’Électricité*, 1949. English translation: “Peak-Load Pricing,” *Journal of Business*, April 1960. **[Not re-consulted]** (record confirmed via a secondary reference list: https://eml.berkeley.edu/~woroch/lumpy%20investment.pdf).
44. Steiner, P. O. “Peak Loads and Efficient Pricing.” *Quarterly Journal of Economics* 71(4):585–610, 1957. **[Not re-consulted]** (record confirmed via RePEc citation: https://ideas.repec.org/p/ind/cdswpp/346.html).
45. Miller, R. “Meta Will Run its Servers For Up to 5 Years.” Data Center Frontier, 2 February 2023 (URL slug reflects an earlier headline). https://www.datacenterfrontier.com/hyperscale/article/21548840/meta-will-abandon-some-data-center-builds-run-servers-longer **[Secondary]** (reporting Meta’s disclosed change in useful-life assumptions: 4 to 4.5 years in Q2 2022, to 5 years two quarters later, for non-AI servers).
46. Sharwood, S. “Low-quality hardware, server supply chain kinks, slow AWS AI.” The Register, 7 February 2025 (reporting Amazon’s Q4 2024 results and useful-life study). https://www.theregister.com/software/2025/02/07/low-quality-hardware-server-supply-chain-kinks-slow-aws-ai/502483 **[Secondary]** (Amazon’s FY2024 Form 10-K not consulted).

47. Costello, K. W., & Hemphill, R. C. “Electric Utilities’ ‘Death Spiral’: Hyperbole or Reality?” *The Electricity Journal* 27(10):7–26, December 2014. Copy consulted: https://energia.pr.gov/wp-content/uploads/sites/7/2016/07/Attachment-RH-1-Costello-Hemphill-Death-Spiral-2014_Final-Pub.pdf
48. Cooper, R., & Kaplan, R. S. “Activity-Based Systems: Measuring the Costs of Resource Usage.” *Accounting Horizons* 6(3):1–13, September 1992. Summary consulted (archived): http://web.archive.org/web/20240714192553/https://maaw.info/ArticleSummaries/ArtSumCooperKaplan1992.htm **[Secondary]**
49. Kaplan, R. S., & Anderson, S. R. “Time-Driven Activity-Based Costing.” *Harvard Business Review* 82(11):131–138, November 2004. https://hbr.org/2004/11/time-driven-activity-based-costing . Summary consulted (archived): http://web.archive.org/web/20220605144333/https://maaw.info/ArticleSummaries/ArtSumKaplanAnderson2004.htm **[Secondary]** (pages per MAAW summary).
50. Google Cloud. “Get key spending insights for your GKE resource allocation and cluster costs” (GKE cost allocation). Documentation. https://docs.cloud.google.com/kubernetes-engine/docs/how-to/cost-allocations
51. Kubecost. “Allocations Dashboard.” Kubecost documentation. https://docs.kubecost.com/using-kubecost/navigating-the-kubecost-ui/cost-allocation
52. Ang, B. W. “Decomposition Analysis for Policymaking in Energy: Which Is the Preferred Method?” *Energy Policy* 32(9):1131–1139, 2004. https://ideas.repec.org/a/eee/enepol/v32y2004i9p1131-1139.html **[Not re-consulted]** (record verified; its recommendation of the logarithmic mean Divisia index confirmed via citing literature).
53. Radovanovic, A., Koningstein, R., Schneider, I., Chen, B., Duarte, A., Roy, B., Xiao, D., Haridasan, M., Hung, P., Care, N., Talukdar, S., Mullen, E., Smith, K., Cottman, M., & Cirne, W. “Carbon-Aware Computing for Datacenters.” arXiv:2106.11750, 11 June 2021. https://arxiv.org/abs/2106.11750 (accessed 2026-10-07). **[Preprint]**
54. Newell, A., Skarlatos, D., Fan, J., Kumar, P., Khutornenko, M., Pundir, M., et al. “RAS: Continuously Optimized Region-Wide Datacenter Resource Allocation.” *Proc. ACM SOSP ’21*, October 2021. https://research.facebook.com/file/4561236690664587/RAS-Continuously-Optimized-Region-Wide-Datacenter-Resource-Allocation.pdf
55. Box, G. E. P., & Tiao, G. C. “Intervention Analysis with Applications to Economic and Environmental Problems.” *Journal of the American Statistical Association* 70(349):70–79, 1975. **[Not re-consulted]**
56. Hyndman, R. J., & Athanasopoulos, G. *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021, §5.2 “Some simple forecasting methods.” https://otexts.com/fpp3/simple-methods.html
57. Li, H., Berger, D. S., Novakovic, S., Hsu, L., Ernst, D., Zardoshti, P., Shah, M., Rajadnya, S., Lee, S., Agarwal, I., Hill, M. D., Fontoura, M., & Bianchini, R. “Pond: CXL-Based Memory Pooling Systems for Cloud Platforms.” *Proc. ASPLOS ’23*, Vancouver, March 2023. arXiv:2203.00241. https://arxiv.org/abs/2203.00241 (accessed 2026-10-07).
58. Microsoft. “On-demand capacity reservation.” Azure Virtual Machines documentation, Microsoft Learn, updated 16 June 2026. https://learn.microsoft.com/en-us/azure/virtual-machines/capacity-reservation-overview (accessed 2026-10-07).
59. van der Vaart, L. (poster), presenting guidance by S. Tolan (Google Cloud Technical Account Manager). “Managing Capacity, Quota, and Stockouts in the Cloud: Concepts and Tips.” Google Cloud Community, 9 September 2022. https://discuss.google.dev/t/managing-capacity-quota-and-stockouts-in-the-cloud-concepts-and-tips/92468 (community article; accessed 2026-10-07).
60. JLL. *2026 Global Data Center Outlook*, 5 January 2026. https://www.jll.com/en-us/insights/market-outlook/data-center-outlook (accessed 2026-10-06).
61. Wood Mackenzie & American Clean Power. *Making the Connection: Meeting the Electric T&D Supply Chain Challenge*, executive summary, September 2025. Hosted by POWER magazine: https://www.powermag.com/wp-

content/uploads/2026/01/making-the-connection-executive-summary-september-2025.pdf (accessed 2026-10-06; figures are Q2 2025 averages).

62. Kennedy, R. “U.S. transformer market faces severe supply constraints as lead times extend to four years.” pv magazine USA, 11 May 2026 (quoting PwC analysts). https://pv-magazine-usa.com/2026/05/11/u-s-transformer-market-faces-severe-supply-constraints-as-lead-times-extend-to-four-years/ **[Secondary]**
63. Ahuja, S. S., Gupta, V., Dangui, V., Bali, S., Gopalan, A., Zhong, H., Lapukhov, P., Xia, Y., & Zhang, Y. “Capacity-Efficient and Uncertainty-Resilient Backbone Network Planning with Hose.” *Proc. ACM SIGCOMM 2021*, August 2021. https://research.facebook.com/file/311440033651798/Capacity-Efficient-and-Uncertainty-Resilient-Backbone-Network-Planning-with-Hose.pdf (accessed 2026-10-06).
64. NVIDIA Corporation. Annual Report on Form 10-K for the fiscal year ended 26 January 2025 (risk factors on supply). U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/1045810/000104581025000023/nvda-20250126.htm
65. Kahrl, F., & Mims Frick, N. *Speed to Power: Solutions for Accelerating Large Load Connections*. Lawrence Berkeley National Laboratory, June 2026. https://eta-publications.lbl.gov/sites/default/files/2026-06/lbnl_large_loads_speed_to_power_final_1.pdf (accessed 2026-10-06; the 6–18 and 18–60 month figures are the report’s citation of one transmission owner, p. 4, fn. 9).
66. De Chant, T. “Altman and Nadella need more power for AI, but they’re not sure how much.” TechCrunch, 3 November 2025, reporting S. Nadella (Microsoft, Chairman and CEO) on the BG2 podcast. https://www.techcrunch.com/2025/11/03/altman-and-nadella-need-more-power-for-ai-but-theyre-not-sure-how-much ; the same remark reported by DatacenterDynamics: https://www.datacenterdynamics.com/en/news/microsoft-has-ai-gpus-sitting-in-inventory-because-it-lacks-the-power-necessary-to-install-them (accessed 2026-10-06; DCD’s transcription reads “it’s actually the fact”). **[Secondary]**
67. Lee, H. L., Padmanabhan, V., & Whang, S. “Information Distortion in a Supply Chain: The Bullwhip Effect.” *Management Science* 43(4):546–558, 1997. https://ideas.repec.org/a/inm/ormnsc/v43y1997i4p546-558.html (abstract consulted).
68. CoreWeave, Inc. Annual Report on Form 10-K for the fiscal year ended 31 December 2025. U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/1769628/000176962826000104/crwv-20251231.htm (accessed 2026-10-06).
69. Oracle Corporation. Annual Report on Form 10-K for the fiscal year ended 31 May 2026 (risk factors). U.S. Securities and Exchange Commission. https://www.sec.gov/Archives/edgar/data/0001341439/000119312526277521/orcl-20260531.htm (accessed 2026-10-06).
70. Tsay, A. A. “The Quantity Flexibility Contract and Supplier-Customer Incentives.” *Management Science* 45(10):1339–1358, October 1999. https://ideas.repec.org/a/inm/ormnsc/v45y1999i10p1339-1358.html (abstract consulted, 2026-10-06).
71. Tsay, A. A., & Lovejoy, W. S. “Quantity Flexibility Contracts and Supply Chain Performance.” *Manufacturing & Service Operations Management* 1(2):89–111, 1999. https://pubsonline.informs.org/doi/10.1287/msom.1.2.89 . Abstract consulted via the IAOR record, https://www.iaorifors.com/paper/29240 ; the narrowing of tolerated change toward delivery as described by the University of Michigan Ross School of Business, https://michiganross.umich.edu/about/100-years/our-impact/1999/quantity-flexibility-contracts-and-supply-chain-performance (accessed 2026-10-06). **[Secondary]**
72. Microsoft Corp. FY26 First Quarter Earnings Conference Call, 29 October 2025 (speaker: S. Nadella, Chairman and CEO). https://www.microsoft.com/en-us/investor/events/fy-2026/earnings-fy-2026-q1 (transcript: aka.ms/transcriptfy26q1).

73. Feitzinger, E., & Lee, H. L. “Mass Customization at Hewlett-Packard: The Power of Postponement.” *Harvard Business Review*, January–February 1997. https://hbr.org/1997/01/mass-customization-at-hewlett-packard-the-power-of-postponement (accessed 2026-10-06). **[Not re-consulted]** (bibliographic record and opening summary only; body not read).
74. Meta Platforms, Inc. Q3 2025 Earnings Call Transcript, 29 October 2025 (speakers: M. Zuckerberg, CEO; S. Li, CFO). https://s21.q4cdn.com/399680738/files/doc_financials/2025/q3/META-Q3-2025-Earnings-Call-Transcript.pdf (accessed 2026-10-06).
75. Meta. “Meta Announces Joint Venture With Funds Managed by Blue Owl Capital to Develop Hyperion Data Center.” Meta Newsroom, 21 October 2025. https://about.fb.com/news/2025/10/meta-blue-owl-capital-develop-hyperion-data-center/ (accessed 2026-10-06).
76. Dixit, A. K., & Pindyck, R. S. *Investment under Uncertainty*. Princeton University Press, 1994. https://ideas.repec.org/b/pup/pbooks/5474.html **[Not re-consulted]**
77. Jordan, W. C., & Graves, S. C. “Principles on the Benefits of Manufacturing Process Flexibility.” *Management Science* 41(4):577–594, April 1995. https://ideas.repec.org/a/inm/ormnsc/v41y1995i4p577-594.html (abstract consulted, 2026-10-06).
78. MIT OpenCourseWare. 15.772J *D-Lab: Supply Chains*, Fall 2014, Lecture 14, “Inventory Management, Part II.” https://ocw.mit.edu/courses/15-772j-d-lab-supply-chains-fall-2014/0d50c5c77382852102ee30b98f1d4657_MIT15_772JF14_Lec14.pdf
79. Treynor Sloss, B. “Introduction” (chapter 1). In Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (eds.), *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly, 2016. https://sre.google/sre-book/introduction/ (accessed 2026-10-07).
80. Luss, H. “Operations Research and Capacity Expansion Problems: A Survey.” *Operations Research* 30(5):907–947, October 1982. doi:10.1287/opre.30.5.907. https://ideas.repec.org/a/inm/oropre/v30y1982i5p907-947.html (abstract consulted).
81. Eppen, G. D. “Effects of Centralization on Expected Costs in a Multi-Location Newsboy Problem.” *Management Science* 25(5):498–501, May 1979. https://ideas.repec.org/a/inm/ormnsc/v25y1979i5p498-501.html
82. Bimpikis, K., & Markakis, M. G. “Inventory Pooling under Heavy-Tailed Demand.” *Management Science* 62(6):1800–1813, June 2016. https://www.gsb.stanford.edu/faculty-research/publications/inventory-pooling-under-heavy-tailed-demand
83. Weinman, J. “Cloudonomics: A Rigorous Approach to Cloud Benefit Quantification.” CSIAC (Cybersecurity & Information Systems Information Analysis Center), 15 November 2016. https://csiac.dtic.mil/articles/cloudonomics-a-rigorous-approach-to-cloud-benefit-quantification/ (accessed 2026-10-07).
84. Halfin, S., & Whitt, W. “Heavy-Traffic Limits for Queues with Many Exponential Servers.” *Operations Research* 29(3):567–588, 1981. doi:10.1287/opre.29.3.567. https://ideas.repec.org/a/inm/oropre/v29y1981i3p567-588.html (abstract consulted).
85. Bashir, N., Deng, N., Rządca, K., Irwin, D., Kodakara, S., & Jnagal, R. “Take it to the Limit: Peak Prediction-driven Resource Overcommitment in Datacenters.” *Proc. EuroSys ’21*, 2021. https://research.google/pubs/take-it-to-the-limit-peak-prediction-driven-resource-overcommitment-in-datacenters/
86. Fan, X., Weber, W.-D., & Barroso, L. A. “Power Provisioning for a Warehouse-sized Computer.” *Proc. 34th ISCA*, 2007. https://research.google/pubs/power-provisioning-for-a-warehouse-sized-computer/
87. Zhang, C., et al. “Flex: High-Availability Datacenters With Zero Reserved Power.” *Proc. ISCA*, June 2021. https://www.microsoft.com/en-us/research/publication/flex-high-availability-datacenters-with-zero-reserved-power/
88. Tirmazi, M., Barker, A., Deng, N., Haque, M. E., Qin, Z. G., Hand, S., Harchol-Balter, M., & Wilkes, J. “Borg: the Next Generation.” *Proc. EuroSys ’20*, Heraklion, April 2020. https://research.google/pubs/borg-the-next-generation/ (accessed 2026-10-07).

89. Jevons, W. S. *Money and the Mechanism of Exchange*. London: Henry S. King, 1875. Chapter I, “Barter,” consulted in the 1876 edition: https://www.econlib.org/library/YPDBooks/Jevons/jvnMME1.html
90. Alphabet Inc. Q4 2025 Earnings Call Transcript, 4 February 2026 (speakers: S. Pichai, CEO; A. Ashkenazi, CFO). https://s206.q4cdn.com/479360582/files/doc_events/2026/Feb/04/2025_Q4_Earnings_Transcript.pdf
91. Meta Platforms, Inc. Second Quarter 2026 Results Conference Call, 29 July 2026 (speakers: M. Zuckerberg, CEO; S. Li, CFO). https://s21.q4cdn.com/399680738/files/doc_financials/2026/q2/META-Q2-2026-Earnings-Call-Transcript.pdf (accessed 2026-10-07).
92. Dominion Energy. Load forecast documentation submitted to PJM, 6 January 2026. https://www.pjm.com/-/media/DotCom/planning/res-adeq/load-forecast/dominion-documentation.pdf (accessed 2026-10-06; figures as of July 2025).
93. Crusoe. “Crusoe, Blue Owl Capital, and Primary Digital Infrastructure enter second phase of $15 billion joint venture to fund AI data center in Abilene, Texas.” Press release, 21 May 2025. https://www.crusoe.ai/newsroom/crusoe-blue-owl-capital-and-primary-digital-infrastructure-enter-joint-venture/ (accessed 2026-10-07).
94. Alberta Electric System Operator (AESO). “Large Load Projects” (Large Load Integration, Phase 1). https://www.aeso.ca/grid/connecting-to-the-grid/large-load-projects (accessed 2026-10-07).
95. Jensen, M. C. “Paying People to Lie: The Truth about the Budgeting Process.” *European Financial Management* 9(3):379–406, 2003. https://ideas.repec.org/a/bla/eufman/v9y2003i3p379-406.html
96. Kerr, S. “On the Folly of Rewarding A, While Hoping for B.” *Academy of Management Journal* 18(4):769–783, 1975. **[Not re-consulted]**
97. Ostrom, E. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. Design principles as summarized at https://wiki.p2pfoundation.net/Governing_the_Commons **[Secondary]**
98. Anandamohan, J. “Managing Clouds from the Ground Up: Cost Engineering at Spotify.” Spotify Engineering, 29 September 2020. https://engineering.atspotify.com/2020/09/managing-clouds-from-the-ground-up-cost-engineering-at-spotify/
99. FinOps Foundation. “Invoicing & Chargeback” (capability). FinOps Framework. https://www.finops.org/framework/capabilities/invoicing-chargeback/
100. Tran, T., & Zetune, M. “Capacity Efficiency at Meta: How Unified AI Agents Optimize Performance at Hyperscale.” Engineering at Meta, 16 April 2026. https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/
101. Johnson, O. “Amazon can’t build AI capacity fast enough, throws another $200B at the problem.” The Register, 6 February 2026 (reporting Amazon’s Q4 2025 earnings call; quoting A. Jassy, CEO). https://www.theregister.com/off-prem/2026/02/06/aws-to-spend-200-billion-to-double-capacity-by-end-of-2027/4524861 (accessed 2026-10-10) **[Secondary]** (no Amazon-hosted text transcript was found).
102. Alphabet Inc. Q2 2026 Earnings Call Transcript, 22 July 2026 (speakers: S. Pichai, CEO; A. Ashkenazi, CFO). https://s206.q4cdn.com/479360582/files/doc_events/2026/Jul/22/2026_Q2_Earnings_Transcript.pdf (accessed 2026-10-07).
103. Weng, Q., Yang, L., Yu, Y., Wang, W., Tang, X., Yang, G., & Zhang, L. “Beware of Fragmentation: Scheduling GPU-Sharing Workloads with Fragmentation Gradient Descent.” *Proc. USENIX ATC ’23*, July 2023. https://www.usenix.org/conference/atc23/presentation/weng (abstract consulted, 2026-10-07).
104. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., & Zhang, H. “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving.” *Proc. OSDI ’24*, 2024. https://arxiv.org/abs/2401.09670

105. Wongpanich, A., Oguntebi, T., Baiocchi Paredes, J., Wang, Y. E., Phothilimthana, P. M., Mitra, R., Zhou, Z., Kumar, N., & Janapa Reddi, V. “Machine Learning Fleet Efficiency: Improving TPU Systems at Scale with ML Productivity Goodput.” *Proc. MLSys 2026* (Industry Track). https://proceedings.mlsys.org/paper_files/paper/2026/file/fbe2b2f74a2ece8070d8fb073717bda6-Paper-Conference.pdf ; earlier version arXiv:2502.06982, https://arxiv.org/pdf/2502.06982
106. Storment, J. R. “Token Economics: The Atomic Unit of AI Value.” FinOps Foundation, 10 May 2026. https://finops.org/insights/token-economics-the-atomic-unit-of-ai-value/ (the throughput-per-megawatt figure it cites is a vendor claim).
107. Jevons, W. S. *The Coal Question*. London: Macmillan, 1865. Via “Jevons paradox,” Wikipedia: https://en.wikipedia.org/wiki/Jevons_paradox **[Secondary]**
108. FinOps Foundation Special Interest Group. “Accurate Cloud Forecasts,” updated 13 March 2026. https://www.finops.org/wg/forecasting-cloud-costs/
109. Amazon Web Services. “Capacity Reservation concepts” (future-dated Capacity Reservations). Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/cr-concepts.html (accessed 2026-10-06).
110. Schaevitz, S. “Three months, 30x demand: How we scaled Google Meet during COVID-19.” Google Workspace blog, 6 August 2020. https://workspace.google.com/blog/productivity-collaboration/keeping-google-meet-ahead-of-usage-demand-during-covid-19 (accessed 2026-10-07).
111. Hadary, O., Marshall, L., Menache, I., Pan, A., Greeff, E. E., Dion, D., Dorminey, S., Joshi, S., Chen, Y., Russinovich, M., & Moscibroda, T. “Protean: VM Allocation Service at Scale.” *Proc. OSDI ’20*, November 2020. https://www.usenix.org/conference/osdi20/presentation/hadary (accessed 2026-10-07).
112. Lawrence Berkeley National Laboratory. “Backlog of power plants seeking transmission grid connection eased somewhat in 2025 amidst high withdrawals” (*Queued Up* 2026 edition), news release, 1 July 2026. https://emp.lbl.gov/news/backlog-power-plants-seeking-transmission-grid-connection-eased-somewhat-2025-amidst (measures generation and storage joining the grid, not data center loads).
113. Constellation Energy. “Constellation to Launch Crane Clean Energy Center, Restoring Jobs and Carbon-free Power to the Grid.” Press release, 20 September 2024 (20-year power purchase agreement with Microsoft; Three Mile Island Unit 1 expected online in 2028 at announcement). https://www.constellationenergy.com/newsroom/2024/Constellation-to-Launch-Crane-Clean-Energy-Center-Restoring-Jobs-and-Carbon-Free-Power-to-The-Grid.html (accessed 2026-10-10); also via Business Wire: https://www.businesswire.com/news/home/20240920487805/en
114. Ohio Power Company (AEP Ohio). Data Center Tariff (Schedule DCT), effective 23 July 2025; approved by the Public Utilities Commission of Ohio, order of 9 July 2025. https://www.aepohio.com/company/about/rates/data-center-tariff ; reported by POWER: https://www.powermag.com/regulator-approves-aep-ohios-landmark-data-center-tariff/ (accessed 2026-10-06).

Sources added in version 7.9

Cited as [A1]–[A9] until the next full renumbering by first appearance. Accessed 2026-10-10.

A1. O’Connor, J. J., & Robertson, E. F. “Agner Krarup Erlang.” MacTutor History of Mathematics, University of St Andrews (describing Erlang’s 1917 paper on automatic telephone exchanges). https://mathshistory.st-andrews.ac.uk/Biographies/Erlang/ **[Secondary]**

A2. Liu, R. P., Mellou, K., Gong, X.-Y., Li, B., Coffee, T., Pathuri, J., Simchi-Levi, D., & Menache, I. “Efficient Cloud Server Deployment Under Demand Uncertainty.” *Manufacturing & Service Operations Management* 27(2), 2025.

https://www.microsoft.com/en-us/research/publication/efficient-cloud-server-deployment-under-demand-uncertainty/ (abstract consulted).

A3. Kumbhare, A. G., Azimi, R., Manousakis, I., Bonde, A., Frujeri, F., Mahalingam, N., Misra, P. A., Javadi, S. A., Schroeder, B., Fontoura, M., & Bianchini, R. “Prediction-Based Power Oversubscription in Cloud Platforms.” *Proc. USENIX ATC ’21*, 2021. https://www.usenix.org/system/files/atc21-kumbhare.pdf

A4. Embrechts, P., McNeil, A., & Straumann, D. “Correlation and Dependence in Risk Management: Properties and Pitfalls.” ETH Zürich working paper, 1999; published in M. Dempster (ed.), *Risk Management: Value at Risk and Beyond*, Cambridge University Press, 2002. https://www.casact.org/abstract/correlation-and-dependence-risk-management-properties-and-pitfalls-0 (abstract consulted).

A5. Cachon, G. P., & Lariviere, M. A. “Capacity Choice and Allocation: Strategic Behavior and Supply Chain Performance.” *Management Science* 45(8):1091–1108, 1999. https://ideas.repec.org/a/inm/ormnsc/v45y1999i8p1091-1108.html (abstract consulted).

A6. Orrick. “FERC Finalizes Rule Adopting Interconnection Reforms” (on FERC Order No. 2023), August 2023. https://www.orrick.com/en/Insights/2023/08/FERC-Finalizes-Rule-Adopting-Interconnection-Reforms **[Secondary]**

A7. Aubakirova, M., Atallah, A., Clark, C., Summerville, J., & Midha, A. “State of AI: An Empirical 100 Trillion Token Study with OpenRouter.” arXiv:2601.10088, January 2026. https://arxiv.org/abs/2601.10088 **[Preprint]**

A8. OpenRouter. “DeepSeek V4 Is Earning Agentic Token Share.” OpenRouter blog, 30 June 2026. https://openrouter.ai/blog/insights/deepseek-v4-adoption/

A9. Google. “Batch Mode in the Gemini API.” Google Developers Blog, 7 July 2025. https://developers.googleblog.com/en/scale-your-ai-workloads-batch-mode-gemini-api/